

УНИВЕРСАЛЬНОЕ ПОСОБИЕ ПО АЛГОРИТМАМ И СТРУКТУРАМ
ПОЛНЫЙ ИСХОДНЫЙ КОД ПРОЕКТА (ВСЕ ФАЙЛЫ, ВСЕ

Дата генерации: 04.09.2026, 22:24:19
Файлов исходного кода: 78
Суммарный объём кода: 1000.5 KB
Глав/билетов в пособии: 30

ОГЛАВЛЕНИЕ: ГЛАВЫ И БИЛЕТЫ

1. 1. Дерево отрезков с операциями на отрезках [id: segment-tree]
2. 2. Двумерная разреженная матрица (Sparse Table)
3. 3. Декартово дерево (Treap) [id: treap]
4. 4. Splay-дерево [id: splay-tree]
5. 5. Двумерная матрица префиксных сумм [id: prefix-sums]
6. Планарность: K5, K3,3 и формула Эйлера [id: planarity]
7. 6. Графы. Представление, DFS, BFS [id: graph-basics]
8. 7. Графы. Планарные. Покраска [id: graph-planarity]
9. 8. Графы. Компоненты связности [id: graph-components]
10. 9. Графы. Топологическая сортировка [id: topological-sort]
11. 10. Графы. Компоненты сильной связности (SCC)
12. 11. Графы. Мосты [id: graph-bridges]
13. 12. Графы. Точки сочленения [id: graph-articulation-points]
14. 13. Графы. Эйлеров цикл [id: graph-euler]
15. Билет 14. Дейкстра [id: dijkstra]
16. Билет 15. Форд-Беллман [id: bellman-ford]
17. Билет 16. Флойд-Уоршелл [id: floyd]
18. 17. Графы. Ускорения поиска [id: pathfinding-optimizations]
19. 17. Графы. Алгоритм Джонсона (Фокус с перевзвешиванием)
20. 18. Графы. Остовное дерево. Краскал [id: mst-kruskal]
21. 19. Графы. Остовное дерево. Прима [id: mst-prims]
22. 20. Графы. Остовное дерево. Борувка [id: mst-boruvka]
23. 21. Алгоритм Кнута-Морриса-Пратта (KMP) [id: kmp]
24. 22. Z-функция [id: string-z-function]
25. Билет 23. Алгоритм Ахо-Корасик [id: aho-corasick]

```
22. src/data/tickets/ticket1_5.ts
23. src/data/tickets/ticket14_20.ts
24. src/data/tickets/ticket21_24.ts
25. src/data/tickets/ticket6_13.ts
```

Интерактивные компоненты и симуляторы

```
26. src/components/ArticulationPointsViz.tsx
27. src/components/BellmanFordViz.tsx
28. src/components/BfsViz.tsx
29. src/components/ChapterImage.tsx
30. src/components/ChapterNav.tsx
31. src/components/ChapterView.tsx
32. src/components/DfsBridgesSimulator.tsx
33. src/components/DijkstraViz.tsx
34. src/components/DPVisualizer.tsx
35. src/components/EulerSimulator.tsx
36. src/components/ExpressQuiz.tsx
37. src/components/FloydViz.tsx
38. src/components/GraphTraversalViz.tsx
39. src/components/HeapViz.tsx
40. src/components/hints/demoKit.tsx
41. src/components/hints/demosExtra.tsx
42. src/components/hints/demosGraph.tsx
43. src/components/hints/demosStruct.tsx
44. src/components/hints/MiniDemo.tsx
45. src/components/hints/SimulatorModal.tsx
46. src/components/hints/TermPopover.tsx
47. src/components/JohnsonViz.tsx
48. src/components/KosarajuViz.tsx
49. src/components/KruskalSimulator.tsx
50. src/components/MnemonicCards.tsx
51. src/components/MobileToc.tsx
52. src/components/Navbar.tsx
53. src/components/PlanarityDemo.tsx
54. src/components/QueueViz.tsx
55. src/components/SalmonAutomatonWidget.tsx
56. src/components/SegmentTreeVisualizer.tsx
57. src/components/Sidebar.tsx
```

Универсальное пособие • полный исходный код

ФАЙЛ 1/78: AGENTS.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 12 | РАЗМЕР: 700

AI Agent Custom Instructions

You are the author of a Universal Educational Guide (Универсальное

****CRITICAL DIRECTIVE****:

Before you start creating new chapters, editing content or using the `view_file` tool to read the skill instructions located at `.ai_skills/universal_guide_authoring/SKILL.md``

This file contains the mandatory rich HTML templates, styles and components required to maintain the high quality of this universal guide.

Do NOT generate plain markdown chapters or invent new HTML elements.

ФАЙЛ 2/78: GEMINI.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 16 | РАЗМЕР: 2000

Gemini App Canvas Instructions

Ты – AI-ассистент, работающий в Gemini App Canvas (или Gemini App Canvas). При запросе пользователя создать или сгенерировать новую страницу ****строго**** придерживаться готовых шаблонов разметки, стилей и компонентов.

Правила для малых страниц (Билетов / Глав)

- **Единый стиль:**** Всегда используй Tailwind CSS и HTML-шаблоны.
- **Структура JSON/TS:**** Каждая новая малая страница должна быть описана в файле `src/data/content.ts` с использованием `{ id: '...', content: ... }` для вставки в `src/data/content.ts`.
- **Строгая структура контента:****
 - Заголовок с цветным бейджем (например, ``bg-indigo-100``).
 - Определение/правило в центре (``border-blue-500`` или ``border-blue-500``).

ФАЙЛ 5/78: package.json

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 41 | РАЗМЕР: 1

```
{
  "name": "algv0-exam-guide",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview",
    "audit:content": "node scripts/audit-coverage.mjs",
    "audit:terms": "node scripts/audit-terms.mjs",
    "export:txt": "node scripts/export/collect-code.mjs",
    "export:png": "node scripts/export/render-pages.mjs",
    "export:pdf": "node scripts/export/build-pdf.mjs",
    "export": "npm run export:txt && npm run export:png",
  },
  "dependencies": {
    "clsx": "2.1.1",
    "lucide-react": "^1.21.0",
    "motion": "^12.40.0",
    "pdfkit": "^0.20.1",
    "react": "19.2.6",
    "react-dom": "19.2.6",
    "tailwind-merge": "3.4.0"
  },
  "devDependencies": {
    "@tailwindcss/vite": "4.1.17",
    "@types/node": "22.19.17",
    "@types/pdfkit": "^0.17.6",
    "@types/react": "19.2.7",
    "@types/react-dom": "19.2.3",
```

Универсальное пособие • полный исходный код

| `npm run export` | всё сразу | txt + png + pdf |

Скрипты лежат в `scripts/export/`. Требуется ImageMagick

```
```bash
sudo apt-get install -y imagemagick fonts-dejavu-core
```
```

Настройки через переменные окружения:

- * `EXPORT\_DENSITY=200` — DPI растеризации (по умолчанию 300)
- * `EXPORT\_GRAYSCALE=1` — оттенки серого вместо 1-битного

Автоматизация (CI)

`github/workflows/rebuild-pdf.yml` запускается **на каждой новой коммита** и пересобирает TXT → PNG → PDF, кладёт PNG-страницы и документ в PDF в папку `public/export/full\_code\_all\_pages` и коммитит обновлённые `public/export/full\_code\_all\_pages` (с меткой `[skip ci]`, чтобы не заиклиться).

Приложение отдаёт эти файлы по ссылкам `/export/full\_code\_all\_pages/`

Аудит контента

```
```bash
node scripts/audit-coverage.mjs # какие темы без аудита
node scripts/audit-coverage.mjs --md # markdown-таблица
```
```

Актуальный отчёт: [`TOPICS\_AUDIT.md`](./TOPICS_AUDIT.md)

Как устроен интерфейс

- * **Содержание** — на десктопе боковая панель с поиском, на телефоне — липкая строка «Содержание · N из M» и вкладки
- * **Переходы между темами** — кнопки «Назад / Вперёд» в шапке, «Предыдущая / Следующая тема» под текстом; на клавиатуре — стрелки
- * **Всплывающие подсказки по терминам** — текст главы появляется при наведении на термин

k).

- * Таким образом, при извлечении текстового содержимого технически остаются в валидном Markdown-формате латеха (и

Картинки и визуализации (Схемы)

Вместо обычных `<img>` в приложении используются векторы

- * Они помещены внутрь оберток с классом `<div class="img">`
- * Все отрисовки находятся внутри векторного тега `<svg>`
- * Подпись к картинке, если она есть, идет сразу после SVG

Списки

- * Стандартные списки оформлены через `<ul class="list-n">`

Валидный экспорт в Markdown (встроенный)

Экспорт перехватывает основные структурные компоненты HTML

1. Заголовки `<h2>` и `<h3>` заменяются на `##` и `###`
2. Элементы `.analogy-badge` получают выделение маркера
3. Теги `<svg>` заменяются строкой-заглушкой `[ Иллюстрация ]`
4. Элементы списка `<li>` получают маркер `-`
5. Результат извлекается в виде текста и чистится от лишнего

ФАЙЛ 7/78: TOPICS_AUDIT.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 106 | РАЗМЕР: 1.5 КБ

Аудит покрытия тем: где нет «объяснения на пальцах» и

> Сгенерировано скриптом `node scripts/audit-coverage.m`

> Критерии:

- > * **Аналогия** — в HTML главы есть блок «Аналогия ...»
- > * **2D** — к главе привязан интерактивный визуализатор
- >
- > Всего глав в пособии: **25**.

```

т «на пальцах». |
| 16. Флойд (Флойд–Уоршелл) | `floyd` | Есть матричный
| 17. Алгоритм Джонсона | `johnson-algo` | Перевзвешива
| Обзор: Кратчайшие пути и Графы | `intro` | Вводная стр
| Эйлер: Путь ≠ Цикл | `euler-path-vs-cycle` | Ключевая

```

Уровень 4. Есть аналогия, но НЕТ 2D-визуализации (

> Замкнуто: «5. Двумерная матрица префиксных сумм» полн

```

Глава	id	Чего не хватает
3. Декартово дерево (Treap)	`treap`	Нет ни визуализ
8. Графы. Компоненты связности	`graph-components`	
24. Классы сложности, сведение задач	`complexity-cl	

```

Сводная таблица по всем главам

| # | Глава | Аналогий | 2D-виз. | SVG | Картинка |
|----|---|----------|---------|-----|----------|
| 2 | 2. Двумерная разреженная матрица (Sparse Table) | | | | |
| 3 | 3. Декартово дерево (Treap) | 2 | – | – | – |
| 4 | 4. Splay-дерево | 2 | | – | – |
| 5 | 5. Двумерная матрица префиксных сумм | 1 | | | – |
| – | Планарность: K5, K3,3 и формула Эйлера | – | | | – |
| 6 | 6. Графы. Представление, DFS, BFS | 4 | | – | – |
| 8 | 8. Графы. Компоненты связности | 1 | – | – | – |
| 9 | 9. Графы. Топологическая сортировка | 1 | | | – |
| 10 | 10. Графы. Компоненты сильной связности (SCC) | | | | – |
| 12 | 12. Графы. Точки сочленения | 1 | | – | – |
| 14 | 14. Графы. Кратчайший путь. Дейкстра | 3 | | | – |
| 15 | 15. Графы. Кратчайший путь. Форд–Беллман | 1 | | | – |
| 16 | 16. Графы. Кратчайший путь. Флойд | – | | | – |

```
    /* Bundler mode */
    "moduleResolution": "bundler",
    "allowImportingTsExtensions": true,
    "resolveJsonModule": true,
    "isolatedModules": true,
    "noEmit": true,
    "jsx": "react-jsx",

    /* Path mapping */
    "baseUrl": ".",
    "paths": {
      "@/*": ["src/*"]
    },

    /* Linting */
    "strict": true,
    "noUnusedLocals": true,
    "noUnusedParameters": true,
    "noFallthroughCasesInSwitch": true
  },
  "include": ["src", "vite.config.ts"]
}
```

ФАЙЛ 9/78: vite.config.ts

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 33 | РАЗМЕР: 812 Б

```
import path from "path";
import { fileURLToPath } from "url";
import tailwindcss from "@tailwindcss/vite";
import react from "@vitejs/plugin-react";
import { defineConfig } from "vite";
import { viteSingleFile } from "vite-plugin-singlefile";

const filename = fileURLToPath(import.meta.url);
```

```

-----
import { Sidebar } from "../components/Sidebar";
import { MobileToc } from "../components/MobileToc";
import { ChapterView } from "../components/ChapterView";
import { ChapterNav } from "../components/ChapterNav";
import { SimulatorModal } from "../components/hints/Simu
import { SimulatorHub } from "../components/SimulatorHub

export default function App() {
  const [activeTab, setActiveTab] = useState<"guide" |
  const [activeChapterId, setActiveChapterId] = useStat
  const [modalVizId, setModalVizId] = useState<string |

  const index = Math.max(
    0,
    chapters.findIndex((c) => c.id === activeChapterId)
  );
  const activeChapter = chapters[index] ?? chapters[0];
  const prev = index > 0 ? chapters[index - 1] : undefin
  const next = index < chapters.length - 1 ? chapters[in

  const goTo = useCallback((id: string) => {
    setActiveChapterId(id);
    window.scrollTo({ top: 0, behavior: "smooth" });
  }, []);

  // Стрелки ← → листают темы (как на обучающих сайтах)
  useEffect(() => {
    const onKey = (e: KeyboardEvent) => {
      const target = e.target as HTMLElement | null;
      if (target && /^(INPUT|TEXTAREA|SELECT)$/.test(ta
      if (e.metaKey || e.ctrlKey || e.altKey) return;
      if (e.key === "ArrowRight" && next) goTo(next.id)
      if (e.key === "ArrowLeft" && prev) goTo(prev.id);
    };
    window.addEventListener("keydown", onKey);
    return () => window.removeEventListener("keydown",
  }, [goTo, next, prev]);

```

```

        className="h-full bg-gradient-to-r from-blue-500 to-purple-500"
        style={{ width: `${progress}%` }}
      />
    </div>

    <ChapterView
      key={activeChapter.id}
      chapter={activeChapter}
      prev={prev}
      next={next}
      index={index}
      total={chapters.length}
      onGo={goTo}
      onOpenSimulator={setModalVizId}
    />
  </main>
</div>
</>
) : (
  <main className="px-4 sm:px-6 lg:px-10 py-6 lg:py-8" >
    <SimulatorHub
      onOpen={setModalVizId}
      onGoChapter={({id}) => {
        setActiveTab("guide");
        goTo(id);
      }}
    />
  </main>
)}

<SimulatorModal vizId={modalVizId} onClose={() => {}} />

<footer className="p-8 text-center text-slate-600" >
  © 2026 Universal Educational Guide. Интерактивное
</footer>
</div>
);
}

```

```

*/
export function annotateTerms(root: HTMLElement): void {
  if (!cachedRegex) cachedRegex = buildRegex();
  const re = cachedRegex;

  const perTerm = new Map<string, number>();
  const perSurface = new Map<string, number>();
  const walker = document.createTreeWalker(root, NodeFilter.FILTER_ACCEPT,
    acceptNode(node) {
      const parent = (node as Text).parentElement;
      if (!parent) return NodeFilter.FILTER_REJECT;
      if (parent.closest(SKIP_SELECTOR)) return NodeFilter.FILTER_REJECT;
      if (!node.nodeValue || node.nodeValue.trim().length === 0) return NodeFilter.FILTER_REJECT;
      return NodeFilter.FILTER_ACCEPT;
    },
  );

  const targets: Text[] = [];
  let current = walker.nextNode();
  while (current) {
    targets.push(current as Text);
    current = walker.nextNode();
  }

  for (const textNode of targets) {
    const text = textNode.nodeValue ?? "";
    re.lastIndex = 0;
    if (!re.test(text)) continue;
    re.lastIndex = 0;

    const frag = document.createDocumentFragment();
    let last = 0;
    let m: RegExpExecArray | null;

    while ((m = re.exec(text)) !== null) {
      const surface = m[1].toLowerCase();
      const id = labelToId.get(surface);
      if (!id) continue;

```

```
    } catch {
      /* если браузер не умеет lookbehind – просто оста
    }
  };

  // при смене главы
  // eslint-disable-next-line react-hooks/exhaustive-dep
  useEffect(run, deps);

  // страховка: контент перерисовали, а подсказок в нём
  useEffect(() => {
    const el = ref.current;
    if (!el) return;
    if (!el.querySelector(".term-hint")) run();
  });
}
```

ФАЙЛ 12/78: src/index.css

КАТЕГОРИЯ: Ядро приложения | СТРОК: 165 | РАЗМЕР: 3.1 K

```
@import "tailwindcss";

@layer utilities {
  .neon-glow-blue {
    box-shadow: 0 0 15px rgba(59, 130, 246, 0.5);
  }
  .neon-glow-emerald {
    box-shadow: 0 0 15px rgba(16, 185, 129, 0.5);
  }
  .neon-glow-rose {
    box-shadow: 0 0 15px rgba(244, 63, 94, 0.5);
  }
  .neon-glow-yellow {
    box-shadow: 0 0 20px rgba(234, 179, 8, 0.8);
  }
}
```

```
        transform: translateY(4px);
    }
    to {
        opacity: 1;
        transform: translateY(0);
    }
}

.term-popover {
    animation: hintIn 0.14s ease-out;
}

/* Интерактивные термины в тексте главы (превью как в Википедии)
.term-hint {
    cursor: help;
    border-bottom: 1px dashed rgba(129, 140, 248, 0.75);
    color: #c7d2fe;
    border-radius: 3px;
    padding: 0 1px;
    transition:
        background-color 0.15s ease,
        color 0.15s ease;
}
.term-hint:hover,
.term-hint:focus-visible {
    background: rgba(99, 102, 241, 0.22);
    color: #fff;
    outline: none;
}
@media (hover: none) {
    .term-hint {
        border-bottom-style: solid;
        background: rgba(99, 102, 241, 0.12);
    }
}

/* Контент главы приходит готовым HTML — чиним его под себя
.chapter-body {
```

```
    line-height: 1.3 !important;
  }
  .chapter-body :where(h3) {
    font-size: 1.05rem !important;
    line-height: 1.35 !important;
  }
  .chapter-body :where(.p-6, .p-8) {
    padding: 0.9rem !important;
  }
  .chapter-body :where(.text-3xl) {
    font-size: 1.4rem !important;
  }
  .chapter-body :where(.text-2xl) {
    font-size: 1.2rem !important;
  }
  .chapter-body :where(.text-xl) {
    font-size: 1.05rem !important;
  }
}

/* Custom scrollbar styling for dark theme */
::-webkit-scrollbar {
  width: 8px;
  height: 8px;
}
::-webkit-scrollbar-track {
  background: #0f172a;
}
::-webkit-scrollbar-thumb {
  background: #334155;
  border-radius: 4px;
}
::-webkit-scrollbar-thumb:hover {
  background: #475569;
}
```

```
declare module '*.jpg' {
  const src: string;
  export default src;
}
declare module '*.jpg?url' {
  const src: string;
  export default src;
}
declare module '*.jpeg' {
  const src: string;
  export default src;
}
declare module '*.png' {
  const src: string;
  export default src;
}
```

РАЗДЕЛ: КОНТЕНТ И ДАННЫЕ ПОСОБИЯ

ФАЙЛ 16/78: src/data/additionalTickets.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 646 | РАЗМ

```
import { Chapter } from "../types";
```

```
export const additionalTickets: Chapter[] = [
  {
    id: "sparse-table",
    title: "2. Двумерная разреженная матрица (Sparse Table)",
    type: "html",
    description: "Разреженная таблица для идемпотентных",
    category: "Продвинутые структуры",
    content: `
<section id="sparse-table" class="mb-12 scroll-mt-10">
```



```

{
  id: "splay-tree",
  title: "4. Splay-дерево",
  type: "html",
  description: "Дерево поиска, которое самобалансируется",
  category: "Продвинутые структуры",
  content: `
<section id="splay-tree" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-indigo-600 text-white px-4 py-1">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
      <h3 class="text-xl font-bold text-blue-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-blue-300 italic">
        <p class="text-xl font-mono text-white">
        (Zig, Zig-Zig, Zig-Zag), гарантируя амортизацию
      </div>
      <div class="grid grid-cols-1 xl:grid-cols-2">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
            Аналогия 1: VIP-лифт
          </div>
          <p class="text-slate-300 text-sm mb-4">
            вится корнем дерева (VIP-статус). Если ты член клуба,
            ка почти не требуется времени!</p>
          </div>
          <div class="bg-slate-900 p-6 rounded-lg">
            <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
              Аналогия 2: Балансировка черешка
            </div>
            <p class="text-slate-300 text-sm mb-4">
              и становиться кривым как бамбук. Но как толк
            </p>
          </div>
        </div>
      </div>
    </div>
  </div>
`

```

```

<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl border"
    <h3 class="text-xl font-bold text-emerald-400"
    <div class="bg-slate-900 p-4 rounded-lg mb-1"
      <p class="text-lg text-emerald-300 italic"
      <p class="text-xl font-mono text-white"
        , y1...y2) = P[x2][y2] - P[x1-1][y2] - P[x2
    </div>
    <div class="grid grid-cols-1 xl:grid-cols-2"
      <div class="bg-slate-800 p-6 rounded-lg"
        <div class="absolute -top-3 left-4 l
        order border-emerald-500 shadow-md">
          ≈ Аналогия 1: Вырезание прямоу
        </div>
        <p class="text-slate-300 text-sm mb
          з него маленький целевой прямоугольник в цел
          ый верхний угол отрезается дважды! Поэтому
        </div>
      </div>
    </div>
  </div>
</section>`,
  },
  {
    id: "graph-planar-colors",
    title: "7. Графы. Планарные. Покраска",
    type: "html",
    description: "Формула Эйлера и теорема о 4 красках.",
    category: "Графы. Теория",
    content: `
<section id="graph-planar-colors" class="mb-12 scroll-m
  <div class="flex items-center mb-6 flex-wrap gap-3"
    <span class="bg-blue-600 text-white px-4 py-1 r
    <h2 class="text-2xl sm:text-3xl font-bold text-
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl bord
    <h3 class="text-xl font-bold text-blue-400

```

```

        перед.</p>
      </div>
      <div class="grid grid-cols-1 xl:grid-cols-2" >
        <div class="bg-slate-800 p-6 rounded-lg" >
          <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md" >
            Аналогия 1: Учеба в вузе
          </div>
          <p class="text-slate-300 text-sm mb-2">
            еская сортировка – это расписание, которое
          >
        </div>
        <div class="bg-slate-900 p-6 rounded-lg" >
          <div class="absolute -top-3 left-4 border border-rose-500 shadow-md" >
            Ограничения: Циклы
          </div>
          <p class="text-slate-300 text-sm mb-2">
            получить опыт, нужна работа". Алгоритм выяв
          </div>
        </div>
      </div>
    </div>
  </section>`,
  },
  {
    id: "scc-kosaraju",
    title: "10. Графы. Компоненты сильной связности (SCC)",
    type: "html",
    description: "Разбиение орграфа на макро-вершины. Алгоритм Косарая",
    category: "Графы. Продвинутое",
    content: `
<section id="scc-kosaraju" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
      <h2 class="text-2xl sm:text-3xl font-bold text-white">
    </div>
    <div class="space-y-8">

```

```

        <p class="text-lg text-rose-300 italic">
        <p class="text-xl font-mono text-white">
и fup[v] > tin[u].</p>
</div>
<div class="grid grid-cols-1 gap-6">
    <div class="bg-slate-800 p-6 rounded-lg">
        <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
            Аналогия 1: Единственный мост
        </div>
        <p class="text-slate-300 text-sm mb
этот мост, экономикой обоих кусков придет
        </div>
    </div>
</div>
</div>
</section>`,
    },
    {
        id: "graph-articulation",
        title: "12. Графы. Точки сочленения",
        type: "html",
        description: "Вершины, удаление которых убивает свя
        category: "Графы. Продвинутое",
        content: `
<section id="graph-articulation" class="mb-12 scroll-mt
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-blue-600 text-white px-4 py-1 r
        <h2 class="text-2xl sm:text-3xl font-bold text-v
    </div>
    <div class="space-y-8">
        <div class="bg-slate-700/50 p-6 rounded-xl bord
            <h3 class="text-xl font-bold text-rose-400 m
            <div class="bg-slate-900 p-4 rounded-lg mb-
                <p class="text-lg text-rose-300 italic m
                <p class="text-xl font-mono text-white">
тей > 1).</p>
            </div>

```

```

        </div>
        <p class="text-slate-300 text-sm mb-4">
            е сходится нечетное число линий, значит, из
            а везде должно быть четное число (зашел-вышел)
        </div>
    </div>
</div>
</div>
</section>`,
    },
    {
        id: "pathfinding-accel",
        title: "17. Графы. Ускорения поиска",
        type: "html",
        description: "",
        category: "Графы. Пути",
        content: `
<section id="pathfinding-accel" class="mb-12 scroll-mt-12">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-blue-600 text-white px-4 py-1 rounded">
            <h2 class="text-2xl sm:text-3xl font-bold text-white">
        </div>
    <div class="space-y-8">
        <div class="bg-slate-700/50 p-6 rounded-xl border">
            <h3 class="text-xl font-bold text-blue-400">
            <div class="bg-slate-900 p-4 rounded-lg mb-4">
                <p class="text-lg text-blue-300 italic">
                <p class="text-xl font-mono text-white">
                    d-длина.</p>
            </div>
            <div class="grid grid-cols-1 gap-6">
                <div class="bg-slate-800 p-6 rounded-lg">
                    <div class="absolute -top-3 left-4 border
                        order border-emerald-500 shadow-md">
                        Аналогия: Копаем туннель под
                    </div>
                <p class="text-slate-300 text-sm mb-4">
                    ней земли. Но если начать копать одновременно

```

```

        </div>
    </div>
</section>`,
    },
    {
        id: "mst-prima",
        title: "19. Графы. Остовное дерево. Прима",
        type: "html",
        description: "",
        category: "Графы. Остовные",
        content: `
<section id="mst-prima" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-blue-600 text-white px-4 py-1 rounded">
            <h2 class="text-2xl sm:text-3xl font-bold text-white">
        </div>
    <div class="space-y-8">
        <div class="bg-slate-700/50 p-6 rounded-xl border">
            <h3 class="text-xl font-bold text-emerald-400">
            <div class="bg-slate-900 p-4 rounded-lg mb-4">
                <p class="text-lg text-emerald-300 italic">
                <p class="text-xl font-mono text-white">
                    которое торчит из посещенных в непосещенных
            </div>
            <div class="grid grid-cols-1 gap-6">
                <div class="bg-slate-800 p-6 rounded-lg">
                    <div class="absolute -top-3 left-4 border
                    rder border-emerald-500 shadow-md">
                        ✦ Аналогия: Заражение вирусом
                    </div>
                    <p class="text-slate-300 text-sm mb-4">
                        ли вирус). Мы стартуем из одного города, и
                        Сеть растет только из одного связного куска
                    </div>
                </div>
            </div>
        </div>
    </div>
</section>`,
    },

```

```

    title: "21. Алгоритм Кнута-Морриса-Пратта (КМП)",
    type: "html",
    description: "",
    category: "Строки",
    content: `
<section id="string-kmp" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-indigo-600 text-white px-4 py-1">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
      <h3 class="text-xl font-bold text-blue-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-blue-300 italic">
        <p class="text-xl font-mono text-white">
        i]), который одновременно является её суффиксом.
      </div>
      <div class="grid grid-cols-1 xl:grid-cols-2">
        <!-- Аналогия 1 -->
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
            Аналогия 1: Бумеранг (Префикс и суффикс)
          </div>
          <p class="text-slate-300 text-sm mb-4">
            идишь "АБРА". Если ты ошибешься при поиске
            концовка "АБРА" совпадает с началом "АБРА",
            мы избежим перепроверок!</p>
        </div>
      </div>
      <details class="bg-slate-900/40 rounded-lg border">
        <summary class="p-4 font-bold text-slate-300">
          Код (Скрыто)
        </summary>
        <div class="p-5 text-sm text-slate-300">
          <pre class="bg-slate-950 p-4 rounded">

```

```

        </div>
        <p class="text-slate-300 text-sm mb-6">
            в тексте кричит: "Эй! Начиная с этой буквы
            т "окно" [L, R] самой дальней найденной копии
        </div>
    </div>
    <details class="bg-slate-900/40 rounded-lg border
        <summary class="p-4 font-bold text-slate-300
        -b border-slate-700/50">
            Код (Скрыто)
        </summary>
        <div class="p-5 text-sm text-slate-300">
            <pre class="bg-slate-950 p-4 rounded" style="font-family: monospace; font-size: 0.9em; margin: 0; padding: 10px 0 10px 20px;">
int l = 0, r = 0;
for (int i = 1; i < n; i++) {
    if (i <= r) z[i] = min(r - i + 1, z[i - l]);
    while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
    if (i + z[i] - 1 > r) { l = i; r = i + z[i] - 1; }
}
</pre>
        </div>
    </details>
</div>
</div>
</section>`,
    },
    {
        id: "complexity-classes",
        title: "24. Классы сложности, сведение задач",
        type: "html",
        description: "",
        category: "Теория сложности",
        content: `
<section id="complexity-classes" class="mb-12 scroll-mt-12">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-purple-600 text-white px-4 py-1 rounded">
            <h2 class="text-2xl sm:text-3xl font-bold text-white">
                </div>
    </div>
    <div class="space-y-8">

```

```
export interface Chapter {
  id: string;
  title: string;
  type: "html";
  content: string;
}

export const chapters: Chapter[] = [
  {
    id: "euler-path-vs-cycle",
    title: "Эйлер: Путь ≠ Цикл (ты путаешь!)",
    type: "html",
    content: `<section id="euler-path-vs-cycle" class="
<div class="flex items-center mb-6">
  <span class="bg-indigo-600 text-white px-4 py-1
  <h2 class="text-3xl font-bold text-white">Не бы,
</div>

<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl bord
    <h3 class="text-xl font-bold text-blue-400 m

    <div class="bg-slate-900 p-4 rounded-lg mb-4
      <p class="text-lg text-blue-300 italic m
      <div class="text-left font-mono text-sm
        <p><strong class="text-white">Эйлер
ин раз. Вершины повторять можно.</p>
        <p><strong class="text-white">Эйлер
        <div class="p-3 bg-slate-950 rounde
          <p class="text-sm text-blue-300
          <p><strong class="text-white">П
          <p><strong class="text-white">Ц
        </div>
      </div>
    </div>
  </div>
</div>
```

```

    </div>

    <details class="bg-slate-900/40 rounded-lg border
      <summary class="p-4 font-bold text-slate-400
        order-slate-700/50">
        Душный режим: Формальное доказательство
      </summary>
      <div class="p-5 text-sm text-slate-300 space-y-4">
        <p class="text-blue-400">Теорема Эйлера
        <p>
          Связный граф содержит эйлеров цикл ⇔
          <b>Эйлеров путь (не цикл) ⇔ ровно 2 вершины с нечётной степенью
        </p>
        <p class="text-rose-400">Почему Гамильтон?
        <p>
          Задача коммивояжёра (TSP) – это взломать комбинаторный экспонент.
          Доказано, что любой полиномиальный алгоритм решает TSP за экспоненту.
          В общем, не парься. Просто запомни.
        </p>
      </div>
    </details>
  </div>
</section>`,
  },
  {
    id: "bridges-code",
    title: "Мосты в графах: код + визуализация",
    type: "html",
    content: `<section id="bridges-code" class="mb-20 space-y-4">
      <div class="flex items-center mb-6">
        <span class="bg-emerald-600 text-white px-4 py-2 rounded">Код
        <h2 class="text-3xl font-bold text-white">Мосты
      </div>

      <div class="space-y-8">
        <div class="bg-slate-700/50 p-6 rounded-xl border">
          <h3 class="text-xl font-bold text-emerald-400">Мосты

```

```
<p class="text-slate-300 text-sm mb-4">
  Граф – это кровеносная система. Ребра (артериалы и вены) – это не мост, живём.
  Если же перерезать единственную артерию, то наступит катастрофа.
  Алгоритм DFS ищет такие «критические» ребра.
```

```
</p>
</div>
</div>
```

```
<details class="bg-slate-900/40 rounded-lg border border-slate-700/40" >
  <summary class="p-4 font-bold text-slate-400" >
    Полный код на C++ (Скрыто)
  </summary>
```

```
<div class="p-5 text-sm text-slate-300 space-y-4">
  <p>Классическая реализация – <code class="text-slate-300">
    оходов.</p>
```

```
<div class="bg-slate-950 p-4 rounded-lg">
  <pre class="text-xs font-mono text-slate-300">
```

```
#include <vector>
#include <algorithm>

using namespace std;

int n; // число вершин
vector<vector<int>>> adj; // список смежности
vector<bool> visited;
vector<int> tin, low;
int timer;

void dfs(int v, int p = -1) {
    visited[v] = true;
    tin[v] = low[v] = timer++; // устанавливаем время

    for (int to : adj[v]) {
        if (to == p) continue; // не идём назад
        dfs(to, v);
        low[v] = min(low[v], low[to]);
    }
}
```

```
<div class="flex items-center mb-6">
  <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-3xl font-bold text-white">Планар
  </div>
```

```
<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl border" style="border-color: #000000; border-width: 1px;">
    <h3 class="text-xl font-bold text-blue-400">Графы

    <div class="bg-slate-900 p-4 rounded-lg mb-4">
      <p class="text-lg text-blue-300 italic">Граф — это набор вершин и ребер.
      <p class="text-3xl font-mono text-white">
        <div class="text-sm text-slate-400 space-y-2">
          <p><span class="text-blue-400 font-mono">V</span> — вершины,
          <p><span class="text-emerald-400 font-mono">E</span> — ребра.
          <p><span class="text-rose-400 font-mono">G</span> — граф.
        </div>
      </div>

      <p class="text-slate-300 text-sm">
        <strong>Следствие:</strong> Для планарного графа:
        <math>E \leq 3V - 6</math>.
        Если ребер больше — граф гарантированно не планарен.
      </p>
    </div>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
  <div class="bg-slate-800 p-6 rounded-lg border" style="border-color: #000000; border-width: 1px;">
    <div class="absolute -top-3 left-4 bg-slate-800 border border-color: #000000; border-width: 1px; padding: 2px 5px;
      order-green-500 shadow-md">
        K5: Полный граф на 5 вершинах
      </div>
      <p class="text-slate-300 text-sm mb-4">
        У K5: <code class="text-white">V=5, E=10</code> — БАХ, непланарен!
        Все 5 вершин соединены со всеми. Напомню:
        Теорема Куратовского: K5 — эталон непланарности.
        Если внутри графа спрятан K5 — забудь про планарность.
      </p>
    </div>
  </div>
```

```

</section>`,
  },
  {
    id: "graph-articulation",
    title: "12. Графы. Точки сочленения",
    type: "html",
    content: `<section id="graph-articulation" class="mb-6">
<div class="flex items-center mb-6">
  <span class="bg-rose-600 text-white px-4 py-1 rounded">
  <h2 class="text-3xl font-bold text-white">Точки
</div>

<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl border">
    <h3 class="text-xl font-bold text-rose-400">
    <div class="bg-slate-900 p-4 rounded-lg mb-4">
      <p class="text-lg text-rose-300 italic">
      <div class="text-left font-mono text-sm">
        <p><strong class="text-white">Точка
рой увеличивает число компонент связности.</p>
        <div class="p-3 bg-slate-950 rounded">
          <span class="text-rose-400 font">
          <p class="text-xl font-bold tex">
          <p class="text-xs text-slate-400">
        </div>
      </div>
    </div>
  </div>

  <p class="text-slate-300 text-sm">
    <strong>Важно:</strong> Это работает то
еются к точкам сочленения. То есть, <strong>
ег - это тупик со степенью 1).
  </p>
</div>

<div class="grid grid-cols-1 xl:grid-cols-2 gap">
  <div class="bg-slate-800 p-6 rounded-lg bor`

```



```

    }
  });

  // Add the new ones
  if (newChapters) {
    newChapters.forEach(c => dedup.set(c.id, c));
  }

  const finalChapters = Array.from(dedup.values());

  finalChapters.sort((a, b) => {
    const numA = parseInt(a.title.match(/(\d+)/)?.[0] ||
    const numB = parseInt(b.title.match(/(\d+)/)?.[0] ||
    return numA - numB;
  });

  export const chapters: Chapter[] = finalChapters;

  export const activeVizIds = [
    "euler-path-vs-cycle", "bridges-code", "planarity-euler",
    "graph-dfs-bfs", "stack-dfs", "queue-bfs", "heap-beam-se",
    "intro", "dijkstra", "bellman-ford", "floyd", "aho-corasi",
    "mst-kruskal", "mst-prima", "mst-boruvka", "string-kmp",
    "string-z-func", "segment-trees", "sparse-table", "dynam
  ];

```

ФАЙЛ 19/78: src/data/glossary.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 626 | РАЗМ

```

import type { DemoKind } from "../components/hints/Mini

/**
 * Глоссарий «википедийных» подсказок.
 *
 * Любое слово из `labels`, встреченное в тексте главы,

```

```

    "Прёшь вперёд до упора, упёрся — откатываешься на
formal:
    "Стек (или рекурсия). Из DFS растут: времена вход
complexity: "O(V + E)",
demo: "dfs",
simulator: "stack-dfs",
},
{
    id: "heap",
    labels: ["куча", "кучу", "кучи", "куче", "кучей", "куча"],
    title: "Куча (priority queue)",
    short:
        "Не отсортированный список, а «мешок, из которого
formal:
        "Полное бинарное дерево в массиве: дети узла i ле
complexity: "вставка/извлечение O(log n), «посмотре
demo: "heap",
simulator: "heap-beam-search",
},
{
    id: "stack",
    labels: ["стек", "стека", "стеке", "стеком", "LIFO"],
    title: "Стек (LIFO)",
    short: "Стопка тарелок: кладёшь сверху и берёшь свер
formal: "push / pop / top за O(1). На стеке живёт р
demo: "stack",
simulator: "stack-dfs",
},
{
    id: "queue",
    labels: ["очередь", "очереди", "очередью", "FIFO"],
    title: "Очередь (FIFO)",
    short: "Очередь в столовой: кто первым встал, тот п
formal: "push_back / pop_front за O(1). Основа BFS
demo: "queue",
simulator: "queue-bfs",
},
{

```

```

        о есть строго по уровням.",
    formal:
        "Кладём в очередь детей корня (их ссылка = корень
        кладём с в очередь.",
    complexity: "O(суммарной длины словаря × алфавит)",
    demo: "trie-bfs",
    simulator: "aho-corasick",
},
{
    id: "suffix-link",
    labels: ["суффиксная ссылка", "суффиксные ссылки",
    title: "Суффиксная ссылка",
    short:
        "«Куда откатиться, если следующая буква не подошла
        строки.",
    formal: "link(v) = go(link(parent(v)), ch). Считает
    demo: "suffix-link",
    simulator: "aho-corasick",
},
{
    id: "toposort",
    labels: ["топологическая сортировка", "топсорт", "топосорт",
    title: "Топологическая сортировка",
    short:
        "Порядок «сначала штаны, потом ботинки»: каждая с
        циклов.",
    formal: "DFS + вершины в обратном порядке выхода, л
    complexity: "O(V + E)",
    demo: "toposort",
    simulator: "top-sort",
},
{
    id: "relaxation",
    labels: ["релаксация", "релаксацию", "релаксации",
    title: "Релаксация ребра",
    short:
        "«А через соседа не быстрее?» Если известный путь
        ,

```

```

    formal: "Строится за  $O(n)$ , запрос и обновление —  $O(1)$ ",
    demo: "segment-tree",
    simulator: "segment-trees",
},
{
    id: "bridge",
    labels: ["мост", "моста", "мосты", "мостов", "мостовые"],
    title: "Мост",
    short:
        "Ребро, после удаления которого граф разваливается",
    formal: "Ребро  $(u, v)$  — мост  $\iff \text{low}[v] > \text{tin}[u]$ , где  $\text{tin}[u]$  — время, когда зашло ребро в родителя — нет).",
    complexity: " $O(V + E)$  одним DFS",
    demo: "bridge",
    simulator: "bridges-code",
},
{
    id: "articulation",
    labels: ["точка сочленения", "точки сочленения", "точки разрыва"],
    title: "Точка сочленения",
    short:
        "Вершина-незаменимый-сотрудник: увольь её — и коллапс",
    formal: "Для не-корня:  $v$  — точка сочленения  $\iff \exists \text{ребро } (v, u) \text{ такое, что } \text{low}[u] \geq \text{tin}[v]$ ",
    demo: "articulation",
    simulator: "graph-articulation",
},
{
    id: "mst",
    labels: [
        "остовное дерево", "остовного дерева", "остовное",
        "минимальное остовное дерево", "MST",
    ],
    title: "Минимальное остовное дерево (MST)",
    short:
        "Связать все точки проводами так, чтобы суммарная длина была минимальной",
    formal: " $V-1$  ребро, никаких циклов. Краскал (жадно).",
    demo: "mst",

```

```

    },
    {
      id: "scc",
      labels: ["компонента сильной связности", "компоненты"],
      title: "Компоненты сильной связности",
      short:
        "Группы вершин, где из каждой можно дойти до каждой",
      formal: "Косарайю: DFS с сохранением порядка выхода",
      complexity: "O(V + E)",
      demo: "scc",
      simulator: "scc-kosaraju",
    },
    {
      id: "prefix-function",
      labels: [
        "префикс-функция", "префикс-функции", "префикс-функцию",
        "префикс", "префикса", "суффиксом", "суффикс", "суффиксов",
        "подстроки", "подстроку", "подстрока",
      ],
      title: "Префикс-функция  $\pi$  (КМП)",
      short:
        "Для каждой позиции запоминаем: какой кусок начался в этой позиции и повторяется до конца строки.",
      formal:
        " $\pi[i]$  — длина наибольшего собственного префикса  $s[0..i-1]$ , который совпадает с суффиксом  $s[0..i-1]$ , возвращая указатель по тексту.",
      complexity: "O(n + m)",
      demo: "prefix-function",
      simulator: "string-kmp",
    },
    {
      id: "z-function",
      labels: ["Z-функция", "Z-функции", "Z-функцию", "z-функция"],
      title: "Z-функция",
      short:
        "Для каждой позиции: сколько символов подряд, начиная с этой позиции, совпадает с началом строки.",
      formal:

```

```

    formal:
      "Форд–Беллман: если на V-й итерации ещё удастся с
demo: "relax",
simulator: "bellman-ford",
},
{
  id: "greedy",
  labels: [
    "жадный алгоритм", "жадный", "жадно", "жадная",
    "самое дешёвое ребро", "самое дешёвое ребро", "са
  ],
  title: "Жадный алгоритм",
  short:
    "На каждом шаге хватаем то, что лучше прямо сейчас
  formal:
    "Корректность обычно доказывается «леммой о разре
demo: "mst",
simulator: "mst-kruskal",
},
{
  id: "euler",
  labels: ["эйлеров цикл", "эйлеров путь", "эйлерова
  title: "Эйлеров путь и цикл",
  short:
    "Пройти по каждому ребру ровно один раз, не отрыв
  formal:
    "Связный граф: эйлеров цикл    все степени чётные;
simulator: "euler-path-vs-cycle",
},
{
  id: "planarity",
  labels: ["планарный", "планарность", "планарного",
  title: "Планарность",
  short:
    "Граф планарен, если его можно нарисовать на листе
  ,
  formal:
    " $V - E + F = 2$  для связного планарного графа. Кур

```

```

    },
    {
      id: "bellman-ford",
      labels: ["Форд-Беллман", "Форда-Беллмана", "Беллман"],
      title: "Алгоритм Форда–Беллмана",
      short:
        "Тупой, но честный: V–1 раз проходим по ВСЕМ рёбрам",
      formal:
        "После k-й итерации гарантированно верны все кратчайшие пути, которые не содержат отрицательный цикл.",
      complexity: "O(V · E)",
      demo: "relax",
      simulator: "bellman-ford",
    },
    {
      id: "floyd",
      labels: [
        "Флойд", "Флойда", "Флойд-Уоршелл", "Флойда-Уоршелла",
        "промежуточная вершина", "промежуточную вершину",
      ],
      title: "Флойд–Уоршелл: разрешаем вершину k",
      short:
        "Внешний цикл – это не «шаг», а «какие пересадки сделать, чтобы посадить пересадку в k?",
      formal:
        "
$$D[i][j] = \min(D[i][j], D[i][k] + D[k][j])$$
. Порядок вершин произвольный.",
      complexity: "O(V³) времени, O(V²) памяти",
      demo: "floyd",
      simulator: "floyd",
    },
    {
      id: "prim",
      labels: ["Прима", "Прим", "Prim"],
      title: "Алгоритм Прима",
      short:
        "Остов растёт как плесень из одной точки: на каждом шаге добавляем ребро, которое не создаст цикла",
    }
  ]
}

```

```

    short:
        "Граф-архипелаг: острова, между которыми нет мостов.",
    formal:
        "Считаются одним проходом: пока есть непосещённая",
    complexity: "O(V + E)",
    demo: "components",
    simulator: "graph-dfs-bfs",
},
{
    id: "graph",
    labels: [
        "граф", "графа", "графе", "графы", "графов", "граф",
        "вершины", "вершина", "вершин", "ребра с весом",
    ],
    title: "Граф",
    short:
        "Точки (вершины) и связи между ними (рёбра). Города",
    formal:
        "G = (V, E). Ребро бывает ориентированным (улицы)",
    demo: "graph",
    simulator: "graph-dfs-bfs",
},
{
    id: "coord-compress",
    labels: ["сжатие координат", "сжатия координат", "сжатие", "сжатия"],
    title: "Сжатие координат",
    short:
        "Координаты бывают гигантские и дробные, а нам нужно",
        "орядковым номером.",
    formal:
        "sort + unique + binary search: значение → индекс",
    complexity: "O(n log n)",
    demo: "coord-compress",
},
{
    id: "treap",

```

```

{
  id: "reduction",
  labels: ["сведение", "сведения", "сведении", "сведён"],
  title: "Сведение задач",
  short:
    "«Я не умею решать A, зато умею B». Если A можно решить, то и B можно решить.",
  formal:
    "A ≤p B: есть полиномиальная функция, переводящая A в B.",
  demo: "np",
},
{
  id: "range",
  labels: ["отрезок", "отрезка", "отрезке", "отрезки"],
  title: "Запрос на отрезке",
  short:
    "Спрашиваем не про один элемент, а про кусок массива.",
  formal:
    "Суммы — префиксные суммы O(1). Минимум без изменений.",
  demo: "prefix-sum",
  simulator: "prefix-sums-2d",
},
{
  id: "string",
  labels: ["строки", "строке", "строку", "строка", "строки"],
  title: "Строка как массив символов",
  short:
    "Строковые алгоритмы держатся на трёх вопросах: где встречается слово (Z-функция) и где встречаются слова словаря (бор).",
  formal:
    "Наивный поиск подстроки — O(N·M). Префикс-функция — O(N).",
  demo: "prefix-function",
  simulator: "string-kmp",
},
{
  id: "rotation",
  labels: ["поворот", "повороты", "поворота", "поворо"],

```

```
    short:
      "Узлы идут «змейкой»: p — левый сын g, а x — правый
        p и g становятся двумя детьми x.",
    formal: "Эквивалентно двойному повороту AVL-дерева",
    demo: "zig-zag",
    simulator: "splay-rotations",
  },
];
```

```
/** Быстрый доступ по id. */
export const GLOSSARY_BY_ID = new Map(GLOSSARY.map((e) => [e.id, e]));

/** Все метки, отсортированные от длинных к коротким (ч
export const GLOSSARY_LABELS: { label: string; id: string }[] =
  e.labels.map((label) => ({ label, id: e.id }))
).sort((a, b) => b.label.length - a.label.length);
```

ФАЙЛ 20/78: src/data/graphContent.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРӨК: 275 | РАЗМЕР: 1.5 КБ

```
import { Chapter } from "../types";

export const graphChapters: Chapter[] = [
  {
    id: "intro",
    title: "Обзор: Кратчайшие пути и Графы",
    type: "html",
    category: "Графы",
    content: `<section id="intro-content" class="mb-12">
<div class="flex items-center mb-6">
  <span class="bg-indigo-600 text-white px-4 py-1">Обзор
  <h2 class="text-3xl font-bold text-white">Обзор
</div>
<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl border`
  }
];
```

```

        (нельзя поехать и заработать). Он всегда в
        <div id="slot-chapter-image-dijkstra"
        </div>

        <!-- Аналогия 2 -->
        <div class="bg-slate-900 p-6 rounded-lg"
            <div class="absolute -top-3 left-4 border
border-rose-500 shadow-md">
                Ограничения
            </div>
            <p class="text-slate-300 text-sm mb-2">
охождение улицы), Дейкстра сломается, так как
            </div>
        </div>

        <details class="bg-slate-900/40 rounded-lg"
            <summary class="p-4 font-bold text-slate-300
-b border-slate-700/50">
                Строгое доказательство (Скрыто)
            </summary>
            <div class="p-5 text-sm text-slate-300"
                <p>Алгоритм использует приоритетную
g V). Гарантирует корректность только для не
            </div>
        </details>

        <div id="slot-dijkstra-viz" class="mt-8"></div>
    </div>
</div>
</section>`
    },
    {
        id: "bellman-ford",
        title: "Билет 15. Форд-Беллман",
        type: "html",
        category: "Графы",
        content: `<section id="bellman-ford-content" class="
<div class="flex items-center mb-6">

```

```

    </div>
</section>`
  },
  {
    id: "floyd",
    title: "Билет 16. Флойд-Уоршелл",
    type: "html",
    category: "Графы",
    content: `<section id="floyd-content" class="mb-12">
<div class="flex items-center mb-6">
  <span class="bg-blue-600 text-white px-4 py-1 rounded">
  <h2 class="text-3xl font-bold text-white">Флойд
</div>

<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl border">
    <h3 class="text-xl font-bold text-indigo-400">
    <div class="bg-slate-900 p-4 rounded-lg mb-4">
      <p class="text-lg text-indigo-300 italic">
      <p class="text-xl font-mono text-white">
    </div>

    <div class="grid grid-cols-1 xl:grid-cols-2">
      <!-- Аналогия 1 -->
      <div class="bg-slate-800 p-6 rounded-lg">
        <div class="absolute -top-3 left-4 border
rder border-emerald-500 shadow-md">
          Аналогия 1: Автомагистрали
        </div>
        <p class="text-slate-300 text-sm mb-4">
евле долететь из Москвы в Париж, если мы сд
        <div id="slot-chapter-image-floyd" class="
        </div>

      <!-- Аналогия 2 -->
      <div class="bg-slate-900 p-6 rounded-lg">
        <div class="absolute -top-3 left-4

```

```

<div class="grid grid-cols-1 xl:grid-cols-2 gap-6">
  <div class="bg-slate-800 p-6 rounded-lg border bo
    <div class="absolute -top-3 left-4 bg-slate-700
      emerald-500 shadow-md">
        Аналогия 1: Паутина (Бор)
      </div>
    <p class="text-slate-300 text-sm mb-4">Представ
      мы движемся по веткам. Если нужного продолж
        ("нити страховки") в самый длинный из извест
    </div>

    <div class="bg-slate-900 p-6 rounded-lg border-2 l
      <div class="absolute -top-3 left-4 bg-rose-900
        -500 shadow-md">
          Аналогия 2: Матёшка (Терминальные)
        </div>
      <p class="text-slate-300 text-sm mb-4">Представ
        нашли и "he", потому что оно спрятано внутри
        минальные ссылки</b> (term_link) – прямые мос
      <p>
    </div>
  </div>

  <details class="bg-slate-900/40 rounded-lg border b
    <summary class="p-4 font-bold text-slate-400 outli
      pen:border-slate-700/50">
        Внутренняя структура автомата и код BFS (Скры
      </summary>
      <div class="p-5 text-sm text-slate-300 space-y-4">
        <p>Каждая вершина имеет таблицу переходов <code>
          ссылку <code>term_link</code> (ближайшая мен
        <div class="bg-black/50 p-4 rounded-lg font-mon
struct Node {
  map<char, int> go;          // Предвычисленные пер
  int link = 0;              // Суффиксная ссылка: куда
  int term_link = 0;         // Терминальная ссылка: матр
  bool is_terminal = false;
  vector<string> words;

```

```
    id: "alg-map",
    title: "Алгоритмы на карте",
    type: "html",
    category: "Графы",
    content: `

<div class="flex items-center mb-6">
        <span class="bg-purple-600 text-white px-4 py-1">
          <h2 class="text-3xl font-bold text-white">Алгоритмы на карте
        </div>

        <div class="space-y-8">
          <div class="bg-slate-700/50 p-6 rounded-xl border">
            <h3 class="text-xl font-bold text-purple-400">Алгоритмы на карте
            <p class="text-slate-300 text-sm mb-4">Когда мы имеем дело с графами, которые могут
              содержать большие числа (дольше, чем 10 цифр), мы часто хотим
              использовать большие числа (дольше, чем 10 цифр), и уже по ним строим графы.</p>
          </div>
        </div>
      </section>`
  }
];


```

ФАЙЛ 21/78: src/data/quizzes.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 106 | РАЗМЕР: 1.5 КБ

```
export interface QuizQuestion {
  question: string;
  options: string[];
  correctIndex: number;
  explanation: string;
}

export const quizzes: Record<string, QuizQuestion[]> = {
  "euler-path-vs-cycle": [
    {
      question: "Какой из следующих путей является простым?",
      options: [
        "Путь, который не содержит повторяющихся вершин",
        "Путь, который не содержит повторяющихся ребер",
        "Путь, который не содержит повторяющихся ребер и вершин",
        "Путь, который не содержит повторяющихся вершин и ребер",
      ],
      correctIndex: 3,
      explanation: "Простой путь — это путь, который не содержит повторяющихся вершин и ребер.",
    },
  ],
}
```

```

        "Нет, low[u] должно быть меньше или равно",
        "Только если граф связный и неориентированный"
    ],
    correctIndex: 0,
    explanation: "Условие моста: low[u] > tin[v]. В н
},
{
    question: "Для чего нужен массив low[v]?",
    options: [
        "Для того чтобы было больше переменных в коде",
        "Чтобы знать минимальный tin предка, до которого",
        "Чтобы хранить глубину рекурсии"
    ],
    correctIndex: 1,
    explanation: "low[v] хранит минимальное время входа
        поиска мостов и точек сочленения."
},
{
    question: "Какова временная сложность алгоритма поиска мостов",
    options: [
        "O(V^2) — ищем мосты в квадрате",
        "O(V + E) — каждый элемент один раз",
        "O(2^V) — полный перебор"
    ],
    correctIndex: 1,
    explanation: "DFS обходит каждую вершину и каждое ребро"
},
],
"planarity-euler-formula": [
    {
        question: "Планарный граф имеет V=6, E=12. Возможна ли такая конфигурация?",
        options: [
            "Да, если это двудольный граф",
            "Нет, нарушается неравенство E ≤ 3V - 6",
            "Да, если нет треугольных граней"
        ],
        correctIndex: 1,
        explanation: "E ≤ 3V - 6 ⇒ 12 ≤ 12 — на грани простоя"
    }
]

```

```
{
  id: "segment-trees",
  title: "1. Дерево отрезков с операциями на отрезках",
  type: "html",
  description: "Дерево отрезков – структура данных для...",
  category: "Оптимизации и Продвинутое структурирование",
  content: `
<section id="segment-trees" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-indigo-600 text-white px-4 py-1">
      <h2 class="text-2xl sm:text-3xl font-bold text-white">
    </div>

    <div class="space-y-8">
      <div id="intro" class="bg-slate-700/50 p-6 rounded">
        <h3 class="text-xl font-bold text-blue-400">

          <div class="bg-slate-900 p-4 rounded-lg mb-4">
            <p class="text-lg text-blue-300 italic">
              <p class="text-xl font-mono text-white">
                omise (lazy).</p>
              </div>

            <div class="grid grid-cols-1 xl:grid-cols-2">
              <div class="bg-slate-800 p-6 rounded-lg">
                <div class="absolute -top-3 left-4 l
                  rder border-emerald-500 shadow-md">
                    Аналогия 1: Корпоративная пре
                  </div>
                <p class="text-slate-300 text-sm mb">
                  ы рассылать 10 000 писем, он пишет одно пис
                  счетах сотрудников только тогда, когда сотр
                  женное обновление).</p>
                </div>

                <div class="bg-slate-900 p-6 rounded-lg">
                  <div class="absolute -top-3 left-4 l
                    border-rose-500 shadow-md">
```

```
<section id="sparse-table" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-indigo-600 text-white px-4 py-1">
    <h2 class="text-2xl sm:text-3xl font-bold text-slate-800">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-800">
      <h3 class="text-xl font-bold text-blue-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-blue-300 italic">
        <p class="text-xl font-mono text-white">
          ых отрезков:  $\min(\text{ST}[L][k], \text{ST}[R - 2^k + 1][k])$ 
      </div>
      <div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md">
            Аналогия 1: Школьные линейки
          </div>
          <p class="text-slate-300 text-sm mb-4">
            зок длины 7. Ты берешь две заготовленные заготовки
            ь отрезок длины 7 (перекрывая друг друга по 1 см)
            ничего складывать, просто пересекаем куски.
          </div>
          <div class="bg-slate-900 p-6 rounded-lg">
            <div class="absolute -top-3 left-4 right-4 bottom-4 border border-rose-500 shadow-md">
              Аналогия 2: Ступени
            </div>
            <p class="text-slate-300 text-sm mb-4">
              ужно покрыть отрезок, мы берем две самые большие
            </div>
          </div>
        <div class="bg-slate-900/40 rounded-lg p-4">
          <summary class="p-4 font-bold text-slate-200">
            -b border-slate-700/50">
              Код (Скрыто)
            </summary>
          </div>
        </div>
      </div>
    <div class="bg-slate-900/40 rounded-lg p-4">
      <summary class="p-4 font-bold text-slate-200">
        -b border-slate-700/50">
          Код (Скрыто)
        </summary>
      </div>
    </div>
  </div>
```

```

        </div>
        <div class="bg-slate-900 p-6 rounded-lg" style="border: 1px solid #2d3748; border-radius: 0.5em; padding: 0.5em 0.75em 0.75em 0.5em;">
            <div class="absolute -top-3 left-4 right-4 bottom-4 border-rose-500 shadow-md" style="border: 1px solid #f47920; border-radius: 0.5em; padding: 0.25em 0.5em 0.5em 0.25em;">
                Аналогия 2: Слияние капель (Merge Sort)
            </div>
            <p class="text-slate-300 text-sm mb-0">
                вого). Мы просто смотрим на корни: у кого Y
                как потомок.</p>
            </div>
        </div>
        <details class="bg-slate-900/40 rounded-lg border border-slate-700/50" style="border: 1px solid #2d3748; border-radius: 0.5em; padding: 0.5em 0.75em 0.75em 0.5em;">
            <summary class="p-4 font-bold text-slate-300" style="border-bottom: 1px solid #2d3748; padding: 0.5em 0.75em 0.5em 0.5em;">
                Код (Скрыто)
            </summary>
            <div class="p-5 text-sm text-slate-300" style="padding: 0.5em 0.75em 0.75em 0.5em;">
                <pre class="bg-slate-950 p-4 rounded" style="background-color: #1a202c; padding: 0.5em 0.75em 0.75em 0.5em;">
pair<Node*, Node*> split(Node* t, int x) {
    if (!t) return {nullptr, nullptr};
    if (t->val <= x) {
        auto [L, R] = split(t->right, x);
        t->right = L;
        return {t, R};
    } else {
        auto [L, R] = split(t->left, x);
        t->left = R;
        return {L, t};
    }
}</pre>
            </div>
        </details>
    </div>
</div>
</section>`
    },
    {
        id: "splay-tree",

```

```
<details class="bg-slate-900/40 rounded-lg"
  <summary class="p-4 font-bold text-slate-300"
    -b border-slate-700/50">
      Код (Скрыто)
    </summary>
    <div class="p-5 text-sm text-slate-300"
      <pre class="bg-slate-950 p-4 rounded"
// Zig, Zag
function rotate(x) {
  let p = x.parent;
  if (p.left === x) { // Right rotate
    p.left = x.right;
    x.right = p;
  } else { // Left rotate
    p.right = x.left;
    x.left = p;
  }
}</pre>

    </div>
  </details>

  <div class="bg-indigo-950/60 p-6 rounded-xl"
    <h4 class="text-lg font-bold text-indigo-300">
      Как это работает?
    </h4>
    <div class="space-y-4 text-sm text-slate-300"
      <div>
        <strong class="text-white block">
          корень?</strong>
        <p>Оно поднимет узел, на котором
          ска. За счет сдвига этого листа в корень, д
        </p>
      </div>
      <div>
        <strong class="text-white block">
          а?</strong>
        <p>Если агент постоянно переключ
          находящимися на разных концах дерева, его
          ащая дерево в вытянутый бамбук для другого.
```

```
-----
        <div class="absolute -top-3 left-4
            rder border-emerald-500 shadow-md">
            * Аналогия 1: Вырезание прямоу
        </div>
        <p class="text-slate-300 text-sm mb
            з него маленький целевой прямоугольник в це
            ый верхний угол отрезается дважды! Поэтому
        </div>
    </div>
</div>
</section>`
    }
};
```

ФАЙЛ 23/78: src/data/tickets/ticket14_20.ts
КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 248 | РАЗМ

```
-----

import { Chapter } from '../types';

export const tickets14to20: Chapter[] = [
    {
        id: "dijkstra",
        title: "14. Графы. Поиск кратчайшего пути. Дейкстра",
        type: "html",
        category: "Графы. Пути",
        content: `
<section id="dijkstra-content" class="mb-12 scroll-mt-12">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-blue-600 text-white px-4 py-1 rounded">
        <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">
    </div>
    <div class="space-y-8">
        <div class="bg-slate-700/50 p-6 rounded-xl border-emerald-400">
            <h3 class="text-xl font-bold text-emerald-400">

```

```

        <h2 class="text-2xl sm:text-3xl font-bold text-rose-400">
</div>
<div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border-emerald-500 shadow-md">
        <h3 class="text-xl font-bold text-rose-400">
            <div class="bg-slate-900 p-4 rounded-lg mb-4">
                <p class="text-lg text-rose-300 italic">
                    <p class="text-xl font-mono text-white">
            </div>
            <div class="grid grid-cols-1 xl:grid-cols-2">
                <div class="bg-slate-800 p-6 rounded-lg">
                    <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
                        ♂ Аналогия 1: Параноик
                    </div>
                    <p class="text-slate-300 text-sm mb-4">
                        еряет ВСЕ возможные дороги V-1 раз подряд.
                    </div>
                    <div class="bg-slate-900 p-6 rounded-lg">
                        <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
                            Отрицательный цикл
                        </div>
                        <p class="text-slate-300 text-sm mb-4">
                            опали во временную петлю.</p>
                    </div>
                </div>
                <div id="slot-bellman-viz" class="mt-8"></div>
            </div>
        </div>
</section>`
    },
    {
        id: "floyd",
        title: "16. Графы. Поиск кратчайшего пути. Флойд",
        type: "html",
        category: "Графы. Пути",
        content: `

```

```

{
  id: "johnson-algo",
  title: "17. Графы. Алгоритм Джонсона (Фокус с перев.",
  type: "html",
  category: "Графы. Пути",
  content: `
<section id="johnson-algo" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-slate-800">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
      <h3 class="text-xl font-bold text-amber-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-xl font-mono text-white">
      </div>
      <div class="grid grid-cols-1 gap-6 mb-6">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 right-4 bottom-4
er border-amber-500 shadow-md">
            ⚡ Суть перевзвешивания
          </div>
          <p class="text-slate-300 text-sm mb-4">
            Дейкстра (Билет 14) быстрая, но
            ает очень медленно. Мы хотим сделать <b>Джонсо
            е пути остались теми же.
          </p>
          <ol class="text-sm text-slate-300 space-y-4">
            <li>Добавляем <b>фиктивную вершину</b>
            <li>Запускаем от неё <b>медленный поиск</b> по
            <b>«потенциалам»</b> <code class="text-pink-400">
            <li>Меняем старые веса: новое расстояние = старое
            <li><b>Магия!</b> Все веса теперь являются минимальными
            ренные потенциалы сокращаются).</li>
            <li>Теперь смело запускаем <b>быстрый поиск</b> по
          </ol>
        </div>
      </div>
    </div>
  </div>
`

```

```

</section>`
  },
  {
    id: "mst-prima",
    title: "19. Графы. Остовное дерево. Прима",
    type: "html",
    category: "Графы. Остовные",
    content: `
<section id="mst-prima" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
      <h3 class="text-xl font-bold text-emerald-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-emerald-300 italic">
        <p class="text-xl font-mono text-white">
        которое торчит из посещенных в непосещенных
      </div>
      <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border
rder border-emerald-500 shadow-md">
            ✦ Аналогия: Заражение вирусом
          </div>
          <p class="text-slate-300 text-sm mb-4">
ли вирус). Мы стартуем из одного города, и
Сеть растет только из одного связного куска
        </div>
      </div>
    </div>
  </div>
</section>`
  },
  {
    id: "mst-boruvka",

```

```
import { Chapter } from '../..types';

export const tickets21to24: Chapter[] = [
  {
    id: "string-kmp",
    title: "21. Префикс-функция (π) – Взгляд назад (КМП)",
    type: "html",
    category: "Строки",
    content: `
<section id="string-kmp" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-indigo-600 text-white px-4 py-1">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
      <h3 class="text-xl font-bold text-blue-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-blue-300 italic">
        <p class="text-xl font-mono text-white">
        сом, оканчивающимся в позиции i.</p>
      </div>

      <div class="grid grid-cols-1 xl:grid-cols-2">
        <!-- Аналогия 1 -->
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
            ♂ Аналогия 1: Поиск без отка
          </div>
          <p class="text-slate-300 text-sm mb">
            Мы идём по строке слева направо.
            строки СЕЙЧАС закончился под моим пальцем?»
            Представь, ты ищешь в тексте слово
            x</code>. Тупой алгоритм начнет искать за но
            </code>, а это начало нашего слова! Не надо
```

```

        </div>
    </details>
    <details class="bg-slate-900/40 rounded-lg border-1
        <summary class="p-4 font-bold text-slate-300
        -b border-slate-700/50">
            Код C++ (Скрыто)
        </summary>
        <div class="p-5 text-sm text-slate-300
            <pre class="bg-slate-950 p-4 rounded
vector<int> pi(s.length());
for (int i = 1; i < s.length(); i++) {
    int j = pi[i-1];
    while (j > 0 && s[i] != s[j]) j = pi[j-1];
    if (s[i] == s[j]) j++;
    pi[i] = j;
}</pre>
        </div>
    </details>
</div>
</div>
</section>`
    },
    {
        id: "string-z-func",
        title: "22. Z-функция — Сдвиг и наложение",
        type: "html",
        category: "Строки",
        content: `
<section id="string-z-func" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-indigo-600 text-white px-4 py-1">
        <h2 class="text-2xl sm:text-3xl font-bold text-white">
    </div>
    <div class="space-y-8">
        <div class="bg-slate-700/50 p-6 rounded-xl border">
            <h3 class="text-xl font-bold text-emerald-400">
            <div class="bg-slate-900 p-4 rounded-lg mb-4">
                <p class="text-lg text-emerald-300 italic">

```

```

<div class="p-5 text-sm text-slate-300" style="border: 1px solid #444; padding: 10px;">
  <p>Тот же пример: abcabcd</p>
  <ul class="list-disc pl-5 space-y-2">
    <li><code>Z[0]</code> – не считаем, так как <b>a</b> не совпало с <b>a</b>
    <li><code>Z[1]</code> (<b>bcabcd</b> совпало с <b>b</b> в позиции 1)
    <li><code>Z[2]</code> (<b>cabcd</b> совпало с <b>c</b> в позиции 2)
    <li><code>Z[3]</code> (<b>abcd</b> совпало с <b>a</b> в позиции 3)
    <li><code>Z[4], Z[5], Z[6]</code> совпали с пустой строкой.
  </ul>
</div>
</details>
<details class="bg-slate-900/40 rounded-lg p-5" style="border: 1px solid #444;">
  <summary class="p-4 font-bold text-slate-300" style="border: 1px solid #444; border-bottom: none;">
    Код C++ (Скрыто)
  </summary>
  <div class="p-5 text-sm text-slate-300" style="border: 1px solid #444; border-top: none;">
    <pre class="bg-slate-950 p-4 rounded" style="border: 1px solid #444; border-top: none;">
int l = 0, r = 0;
for (int i = 1; i <= n; i++) {
  if (i <= r) z[i] = min(r - i + 1, z[i - l]);
  while (i + z[i] <= n && s[z[i]] == s[i + z[i]]) z[i]++;
  if (i + z[i] - 1 > r) { l = i; r = i + z[i] - 1; }
}
</pre>
</div>
</details>
</div>
</div>
</section>`
},
{
  id: "aho-corasick",
  title: "23. Алгоритм Ахо-Корасик",
  type: "html",
  category: "Строки",

```

</div>

<details class="bg-slate-900/40 rounded-lg border border-slate-700/40">
 <summary class="p-4 font-bold text-slate-400 outline-slate-700/50">

Код (Скрыто)

</summary>

<div class="p-5 text-sm text-slate-300 space-y-2">

<pre class="bg-slate-950 p-4 rounded overflow-x-auto">

```
int get_link(int v) {
  if (t[v].link == -1) {
    if (v == 0 || t[v].p == 0) t[v].link = 0;
    else t[v].link = go(get_link(t[v].p), t[v].pch);
  }
  return t[v].link;
}
```

```
int go(int v, char c) {
  if (t[v].go[c] == -1) {
    if (t[v].next[c] != -1) t[v].go[c] = t[v].next[c];
    else t[v].go[c] = v == 0 ? 0 : go(get_link(v), t[v].pch);
  }
  return t[v].go[c];
}</pre>
```

</div>

</details>

</div>

</section>

},

{

id: "complexity-classes",

title: "24. Классы сложности, сведение задач",

type: "html",

category: "Теория сложности",

content: `

<section id="complexity-classes" class="mb-12 scroll-mt-1">

<div class="flex items-center mb-6 flex-wrap gap-3">

ФАЙЛ 25/78: src/data/tickets/ticket6_13.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 486 | РАЗМЕР: 10.5 КБ

```
import { Chapter } from "../../types";

export const tickets6to13: Chapter[] = [
  {
    id: "graph-dfs-bfs",
    title: "6. Графы. Представление, DFS, BFS",
    type: "html",
    description: "Представление графов и базовые обходы",
    category: "Графы. База",
    content: `
<section id="graph-dfs-bfs" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">Билет 6-13
    <h2 class="text-2xl sm:text-3xl font-bold text-slate-800">Графы
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-700">
      <h3 class="text-xl font-bold text-slate-300">Представление графов
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-slate-300 italic">Граф — это набор вершин V и набор рёбер E.
        <p class="text-xl font-mono text-white">G = (V, E).</p>
      </div>
      <div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
            Аналогия: Матрица = Таблица
          </div>
          <p class="text-slate-300 text-sm mb-4">Пример: Кто знает кого?
          <p class="text-slate-300 text-sm mb-4">Глупо, но проверка "знает ли Вася Петю" м
        </div>
      </div>
    </div>
  </div>
`
  }
]
```

```

<div class="bg-slate-900 p-6 rounded-lg" style="border: 1px solid #000; border-radius: 10px; padding: 10px;">
  <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md">
    BFS (Поиск в ширину) = Волна
  </div>
  <p class="text-slate-300 text-sm mb-0">
    <b>Что делает:</b> Концентрические волны от источника.
    Работает через Очередь (FIFO). Продвигается в 4 стороны.
  </p>
  <ul class="text-xs text-emerald-300 list-disc list-inside">
    <li><b>Где используется:</b></li>
    <li>Кратчайший путь в Невзвешенном графе</li>
    <li>Алгоритм Ахо-Корасик (сборка слов)</li>
    <li>Алгоритм Форда-Фалкерсона / Поиск в ширину</li>
    <li>Двунаправленный поиск для ускорения</li>
  </ul>
</div>
</div>

<details class="bg-slate-900/40 rounded-lg" style="border: 1px solid #000; border-radius: 10px; padding: 10px;">
  <summary class="p-4 font-bold text-slate-300" style="border: 1px solid #000; border-radius: 10px; padding: 5px;">
    -b border-slate-700/50" style="border: 1px solid #000; border-radius: 10px; padding: 5px;">
    Код обходов C++ (Основа)
  </summary>
  <div class="grid grid-cols-1 md:grid-cols-2">
    <pre class="bg-slate-950 p-3 rounded" style="border: 1px solid #000; border-radius: 10px; padding: 5px;">
// DFS (Рекурсия)
vector<bool> vis;
vector<vector<int>>> adj;

void dfs(int v) {
    vis[v] = true;
    for (int u : adj[v]) {
        if (!vis[u]) dfs(u);
    }
}</pre>

    <pre class="bg-slate-950 p-3 rounded" style="border: 1px solid #000; border-radius: 10px; padding: 5px;">
// BFS (Очередь)
vector<int> q;
vector<bool> vis;
vector<vector<int>>> adj;

void bfs(int start) {
    q.push_back(start);
    vis[start] = true;
    while (!q.empty()) {
        int v = q.front(); q.pop();
        for (int u : adj[v]) {
            if (!vis[u]) {
                vis[u] = true;
                q.push_back(u);
            }
        }
    }
}</pre>

```

```

        </div>
        <div class="grid grid-cols-1 gap-6">
            <div class="bg-slate-800 p-6 rounded-lg">
                <div class="absolute -top-3 left-4
                rder border-emerald-500 shadow-md">
                    Аналогия 1: Карта мира
                </div>
                <p class="text-slate-300 text-sm mb-6">
                    . Границы не пересекаются в одной точке по-
                    ы не сливались цветами, Всегда можно всего `
                </div>
            </div>
        </div>
    </div>
</section>`,
    },
    {
        id: "graph-components",
        title: "8. Графы. Компоненты связности",
        type: "html",
        description: "Нахождение кусков графа",
        category: "Графы. База",
        content: `
<section id="graph-components" class="mb-12 scroll-mt-1">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-blue-600 text-white px-4 py-1 rounded">
            <h2 class="text-2xl sm:text-3xl font-bold text-white">
        </div>
    <div class="space-y-8">
        <div class="bg-slate-700/50 p-6 rounded-xl border">
            <h3 class="text-xl font-bold text-emerald-400">
            <div class="bg-slate-900 p-4 rounded-lg mb-6">
                <p class="text-lg text-emerald-300 italic">
                <p class="text-xl font-mono text-white">
                ество компонент.</p>
            </div>
        <div class="grid grid-cols-1 xl:grid-cols-2">
            <div class="bg-slate-800 p-6 rounded-lg">

```

```

        </div>
        <p class="text-slate-300 text-sm mb-4">
            еская сортировка – это расписание, которое
        >
    </div>
    <div class="bg-slate-900 p-6 rounded-lg">
        <div class="absolute -top-3 left-4 border-rose-500 shadow-md">
            Ограничения: Циклы
        </div>
        <p class="text-slate-300 text-sm mb-4">
            получить опыт, нужна работа". Алгоритм выяв
        </div>
    </div>

    <details class="bg-slate-900/40 rounded-lg">
        <summary class="p-4 font-bold text-slate-300">
            Душный мод: Код на C++ (Скрыто)
        </summary>
        <div class="p-5 text-sm text-slate-300">
            <p class="text-indigo-400 font-bold">
                <div class="bg-slate-950 p-4 rounded">
                    <pre class="text-xs font-mono">
vector<int> adj[MAXN];
bool visited[MAXN];
vector<int> ans;

void dfs(int v) {
    visited[v] = true;
    for (int to : adj[v]) {
        if (!visited[to]) {
            dfs(to);
        }
    }
    ans.push_back(v); // Добавляем в момент ВЫХОДА (pos
}

```

```
<p class="text-slate-300 text-sm mb-4">
  <strong>«Это просто цикл?» – Нет!</strong>
</p>
```

```
<ul class="list-disc list-inside text-sm te
  <li>Если у тебя есть цикл <code>А → Б →
  <li>Если ты добавишь вершину <strong>Г</strong>,
етыре вершины – одна большая SCC</strong>.</li>
  <li>Правило: <b>Если два цикла имеют хо
</ul>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2
  <div class="bg-slate-800 p-6 rounded-lg
    <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
```

Аналогия: Интриги в офисе

```
</div>
```

```
<p class="text-slate-300 text-sm mb
  Сильная связность – это "клуб с
ни в одной SCC. Добавить курьера Гошу, котор
йти наружу (к директору), но обратно в клуб
</p>
```

```
</div>
```

```
<div class="bg-slate-900 p-6 rounded-lg
  <div class="absolute -top-3 left-4 l
border-rose-500 shadow-md">
```

Связь с Билетом 12

```
</div>
```

```
<p class="text-slate-300 text-sm mb
  <b>SCC (Билет 10)</b> склеивает
</br><br>
```

```
  <b>Точки сочленения (Билет 12)</b>
ость</strong> монолита. Вырвешь точку сочле
</p>
```

```
</div>
```

```
</div>
```

```
<details class="bg-slate-900/40 rounded-lg l
```

```

и fup[v] > tin[u].</p>
</div>
<div class="grid grid-cols-1 gap-6">
  <div class="bg-slate-800 p-6 rounded-lg">
    <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
      Аналогия 1: Единственный мост
    </div>
    <p class="text-slate-300 text-sm mb-4">
      ЭТОТ МОСТ, ЭКОНОМИКОЙ ОБОИХ КУСКОВ ПРИДЕТ :
    </div>
  </div>
</div>
</div>
</section>`,
  },
  {
    id: "graph-articulation",
    title: "12. Графы. Точки сочленения",
    type: "html",
    description: "Вершины, удаление которых убивает свя
category: "Графы. Продвинутое",
    content: `
<section id="graph-articulation" class="mb-12 scroll-mt">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-rose-400">
  </div>

  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
      <h3 class="text-xl font-bold text-rose-400">

    <div class="bg-slate-900 p-4 rounded-lg mb-4">
      <p class="text-lg text-rose-300 italic">
      <div class="text-left font-mono text-sm">
        <p><strong class="text-white">Точка
рой увеличивает число компонент связности.</p>

```

```

</div>

<details class="bg-slate-900/40 rounded-lg border-1 border-slate-700/50">
  <summary class="p-4 font-bold text-slate-400">
    Душный мод: Код и корень DFS (Скрыто)
  </summary>
  <div class="p-5 text-sm text-slate-300 space-y-4">
    <p class="text-blue-400 font-bold">Особый случай (если не брать в расчёт то выше неё прыгать некуда). Поэтому для него у него <strong>больше 1 независимого ребенка</strong>.
    <pre class="text-xs font-mono text-slate-300">
void dfs(int v, int p = -1) {
    visited[v] = true;
    tin[v] = low[v] = timer++;
    int children = 0;

    for (int to : adj[v]) {
        if (to == p) continue;
        if (visited[to]) {
            low[v] = min(low[v], tin[to]);
        } else {
            dfs(to, v);
            low[v] = min(low[v], low[to]);
            if (low[to] >= tin[v] && p != -1)
                IS_CUTPOINT(v);
            ++children;
        }
    }
    if (p == -1 && children > 1)
        IS_CUTPOINT(v);
}
</pre>
    </div>
  </div>

```

```

        <p class="text-lg text-indigo-300 italic">
        <p class="text-xl font-mono text-white">
    </div>
    <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg">
            <div class="absolute -top-3 left-4 l
            rder border-emerald-500 shadow-md">
                Аналогия 1: Рисование не отры
            </div>
            <p class="text-slate-300 text-sm mb
е сходится нечетное число линий, значит, из
а везде должно быть четное число (зашел-выш
            </div>
        </div>
    </div>
</section>`,
    },
];
```

РАЗДЕЛ: ИНТЕРАКТИВНЫЕ КОМПОНЕНТЫ И СИМУЛЯТОРЫ

ФАЙЛ 26/78: src/components/ArticulationPointsViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

```
import React, { useState, useEffect } from "react";
import { Play, Pause, RotateCcw, Info } from "lucide-re
```

```
interface Node {
  id: number;
  x: number;
  y: number;
  label: string;
```

```

interface StepInfo {
  desc: string;
  visited: Set<number>;
  ap: Set<number>;
  currentNode: number | null;
  tin: number[];
  low: number[];
}

export const ArticulationPointsViz: React.FC = () => {
  const [steps, setSteps] = useState<StepInfo[]>([]);
  const [currentStepIndex, setCurrentStepIndex] = useState(0);
  const [isPlaying, setIsPlaying] = useState(false);

  useEffect(() => {
    const newSteps: StepInfo[] = [];
    let timer = 0;
    const visited = new Set<number>();
    const ap = new Set<number>();
    const currentTin = Array(nodes.length).fill(-1);
    const currentLow = Array(nodes.length).fill(-1);

    const recordStep = (desc: string, u: number | null) => {
      newSteps.push({
        desc,
        visited: new Set(visited),
        ap: new Set(ap),
        currentNode: u,
        tin: [...currentTin],
        low: [...currentLow],
      });
    };

    recordStep("Начало DFS (Тарьян) для поиска точек со

    const dfs = (v: number, p: number = -1) => {
      visited.add(v);
      currentTin[v] = currentLow[v] = timer++;
    };
  }, []);
};

```

```

    });
  } else if (p === -1) {
    recordStep(
      `Корень дерева DFS ${v} имеет <=1 ребенка, он
      v,
    );
  }
};

for (let i = 0; i < nodes.length; i++) {
  if (!visited.has(i)) {
    dfs(i);
  }
}

recordStep("Алгоритм завершен. Найдены все точки со
setSteps(newSteps);
}, []));

const currentStep = steps[currentStepIndex] || {
  desc: "",
  visited: new Set(),
  ap: new Set(),
  currentNode: null,
  tin: [],
  low: [],
};

useEffect(() => {
  let interval: NodeJS.Timeout;
  if (isPlaying && currentStepIndex < steps.length -
    interval = setInterval(() => {
      setCurrentStepIndex((prev) => prev + 1);
    }, 1500);
  } else if (currentStepIndex >= steps.length - 1) {
    setIsPlaying(false);
  }
  return () => clearInterval(interval);

```

```
</div>
```

```
<div className="grid grid-cols-1 lg:grid-cols-3 g
```

```
  <div className="lg:col-span-2 relative h-96 lg:h-
```

```
    <svg className="w-full h-full" viewBox="0 0 8
```

```
    {/* Edges */}
```

```
    {edges.map((edge, i) => {
```

```
      const u = nodes.find((n) => n.id === edge
```

```
      const v = nodes.find((n) => n.id === edge
```

```
      const isBridge =
```

```
        (edge.u === 3 && edge.v === 4) ||
```

```
        (edge.u === 4 && edge.v === 3);
```

```
      return (
```

```
        <line
```

```
          key={"edge-" + i}
```

```
          x1={u.x}
```

```
          y1={u.y}
```

```
          x2={v.x}
```

```
          y2={v.y}
```

```
          stroke={isBridge ? "#f43f5e" : "#475568"}
```

```
          strokeWidth={isBridge ? "4" : "2"}
```

```
          strokeDasharray={isBridge ? "8,4" : ""}
```

```
        />
```

```
      );
```

```
    ])}
  )}
```

```
{/* Nodes */}
```

```
{nodes.map((node) => {
```

```
  const isCurrent = currentStep.currentNode
```

```
  const isVisited = currentStep.visited.has
```

```
  const isAp = currentStep.ap.has(node.id);
```

```
  const fill = isCurrent
```

```
    ? "#3b82f6"
```

```
    : isAp
```

```
      ? "#e11d48"
```

```

        <text
          x={node.x}
          y={node.y - r - 15}
          textAnchor="middle"
          fill="#94a3b8"
          fontSize="10px"
        >
          tin:{" "}
          {currentStep.tin[node.id] >= 0
            ? currentStep.tin[node.id]
            : "-"}
        </text>
        <text
          x={node.x}
          y={node.y + r + 20}
          textAnchor="middle"
          fill="#94a3b8"
          fontSize="10px"
        >
          low:{" "}
          {currentStep.low[node.id] >= 0
            ? currentStep.low[node.id]
            : "-"}
        </text>
      </>
    )}
  </g>
);
}}
</svg>
</div>

```

```

<div className="bg-slate-950 p-6 border-t lg:bo
  <h4 className="text-sm font-bold text-slate-4
    Статус алгоритма
  </h4>

```

```

<div className="bg-slate-900 border border-sl

```

```

        xt-slate-500"}` }
        >
        {step.desc}
      </div>
    )))
  </div>
</div>
</div>
<div className="mt-4 shrink-0">
  <div className="w-full bg-slate-800 h-2 rounded" style={{
    <div
      className="bg-rose-500 h-full transition-all duration-500"
      style={{
        width: `${(currentStepIndex / Math.max(steps.length, 1)) * 100}%`
      }}
    />
  </div>
</div>
</div>
</div>
</div>
</div>
  );
};

```

ФАЙЛ 27/78: src/components/BellmanFordViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 10

```

import { useState, useEffect } from 'react';

interface Node { id: string; x: number; y: number; name: string; }
interface Edge { u: string; v: string; w: number; }
interface Step {
  iteration: number;
  checkingEdge: { u: string; v: string; w: number } | null;
  distances: { [key: string]: number };
}

```

```

    });

    const edges = getEdges(hasCycle);

    const getAdjList = (ed: Edge[]) => {
        const list: { [key: string]: { v: string; w: number } } = {};
        nodes.forEach(n => (list[n.id] = []));
        ed.forEach(e => list[e.u].push({ v: e.v, w: e.w }));
        return list;
    };
    const adjList = getAdjList(edges);

    const codeLines = [
        { line: 1, text: 'function bellman_ford(graph, start)' },
        { line: 2, text: '    dist = {v: ∞}; dist[start] = 0;' },
        { line: 3, text: '    for i from 1 to V - 1: // |V| - 1 iterations' },
        { line: 4, text: '        for (u, v, weight) in E: // E - edges' },
        { line: 5, text: '            if dist[u] + weight < dist[v]' },
        { line: 6, text: '                dist[v] = dist[u] + weight;' },
        { line: 7, text: '    for (u, v, weight) in E: // Проверка на отрицательный цикл' },
        { line: 8, text: '        if dist[u] + weight < dist[v]' },
        { line: 9, text: '            return "ОБНАРУЖЕН ОТРИЦАТЕЛЬНЫЙ ЦИКЛ"' },
        { line: 10, text: '    return dist' },
    ];

    useEffect(() => {
        const simulationSteps: Step[] = [];
        let dist: { [key: string]: number } = { S: 0, A: 999 };
        let prev: { [key: string]: string | null } = { S: null };

        simulationSteps.push({
            iteration: 0, checkingEdge: null,
            distances: { ...dist }, previous: { ...prev },
            log: '    Фура заведена. Начинаем с Базы (S). Расстояние до А: 999',
            activeCodeLine: 2,
        });

        const vCount = nodes.length;
    }, []);

```

```

        cycleFound = true;
        simulationSteps.push({
            iteration: vCount, checkingEdge: { u: edge.u, v: edge.v },
            distances: { ...dist }, previous: { ...prev },
            log: `    ВНИМАНИЕ! Проверка цикла: Ребро ${edge.u} -> ${edge.v}
            activeCodeLine: 9,
        });
        break;
    }
}

if (!cycleFound) {
    simulationSteps.push({
        iteration: vCount, checkingEdge: null,
        distances: { ...dist }, previous: { ...prev },
        log: '    Проверка завершена. Ни одно ребро не содержит цикл.
        activeCodeLine: 10,
    });
}

setSteps(simulationSteps);
setCurrentStepIndex(0);
setIsPlaying(false);
}, [hasCycle]));

useEffect(() => {
    let timer: any = null;
    if (isPlaying && currentStepIndex < steps.length - 1) {
        timer = setTimeout(() => setCurrentStepIndex((p) => p + 1), 1000);
    } else if (currentStepIndex >= steps.length - 1) {
        setIsPlaying(false);
    }
    return () => clearTimeout(timer);
}, [isPlaying, currentStepIndex, steps.length]);

const currentStep = steps[currentStepIndex] || null;
if (!currentStep) return <div>Загрузка...</div>;

```

```
</div>
```

```
{currentStep.hasNegativeCycle && (
  <div className="absolute top-3 right-3 bg-r
    animate-pulse shadow-lg shadow-rose-600/50
    ▲ Отрицательный цикл!
  </div>
)}
```

```
<svg className="w-full h-auto max-h-[500px]"
  <defs>
    <marker id="arrow-bf-normal" markerWidth=
    <path d="M0,0 L6,3 L0,6 z" fill="#64748b" />
    <marker id="arrow-bf-active" markerWidth=
    <path d="M0,0 L6,3 L0,6 z" fill="#f59e0b" />
    <marker id="arrow-bf-cycle" markerWidth=
    path d="M0,0 L6,3 L0,6 z" fill="#e11d48" />
  </defs>
```

```
{edges.map((edge, idx) => {
  const uNode = nodes.find((n) => n.id === u)
  const vNode = nodes.find((n) => n.id === v)
  const isChecking = currentStep.checkingEdges
  v;
  const isNeg = edge.w < 0;
  const isCyclePart = hasCycle && ((edge.u === u
  && edge.v === 'D'));
```

```
let strokeColor = '#475569';
let marker = 'url(#arrow-bf-normal)';
if (isChecking) { strokeColor = '#f59e0b'
else if (currentStep.hasNegativeCycle &&
```

```
return (
  <g key={idx}>
    <line x1={uNode.x} y1={uNode.y} x2={vNode.x} y2={vNode.y} stroke={strokeColor} stroke-width={
    asNegativeCycle && isCyclePart) ? 4 : 2} marker={isChecking ? '4,4' : 'none'} />
```

```

        <div key={u} className="flex flex-wrap gap-2
        -colors">
          <span className="font-bold text-white">
            {adjList[u].map((edge, idx) => {
              const isChk = currentStep.checkingEdge === edge
              return (
                <span key={idx} className={`px-2 py-1
                edge.w < 0 ? 'bg-rose-900/80 text-rose-300' : 'bg-slate-800/40 text-slate-300'}>
                  {edge.v} ({edge.w})
                </span>
              )
            })}
          </div>
        </div>
      </div>
    </div>
  </div>

  <div className="flex flex-col lg:flex-row gap-6">
    /* Таблица расстояний */
    <div className="w-full lg:w-1/2 bg-rose-950/10">
      <div>
        <h4 className="text-lg font-bold text-rose-300">Таблица расстояний
        <div className="grid grid-cols-4 gap-2 text-slate-300">
          {nodes.map(n => {
            const d = currentStep.distances[n.id];
            const isTgt = currentStep.checkingEdge === n
            return (
              <div key={n.id} className={`p-2 rounded
              : 'border-slate-700/60 bg-slate-800/40'`} >
                <div className="text-xs text-slate-400">{n.v}</div>
                <div className={`font-mono font-bold text-rose-300`} >{d}</div>
              </div>
            )
          })}
        </div>
      </div>
    </div>
  </div>

```

```
{ id: 6, label: '6', x: 525, y: 250 },
];

const EDGES = [
  { from: 0, to: 1 }, { from: 0, to: 2 },
  { from: 1, to: 3 }, { from: 1, to: 4 },
  { from: 2, to: 5 }, { from: 2, to: 6 }
];

const ADJ: { [key: number]: number[] } = {
  0: [1, 2], 1: [3, 4], 2: [5, 6], 3: [], 4: [], 5: [],
};

interface Step {
  activeLine: number;
  queue: number[];
  visited: number[];
  currentNode: number | null;
  logs: string;
}

const generateBfsSteps = (): Step[] => {
  const steps: Step[] = [];
  const queue: number[] = [];
  const visited = new Set<number>();

  // Init
  steps.push({
    activeLine: 1,
    queue: [],
    visited: [],
    currentNode: null,
    logs: "Инициализация: начинаем с корня (0)."
  });

  queue.push(0);
  visited.add(0);
  steps.push({
```

```

        queue.push(n);

        steps.push({
            activeLine: 9,
            queue: [...queue],
            visited: Array.from(visited),
            currentNode: curr,
            logs: `Помечаем ${n} как посещенную и добавляем в очередь.`
        });
    }
}
} else {
    steps.push({
        activeLine: 6,
        queue: [...queue],
        visited: Array.from(visited),
        currentNode: curr,
        logs: `Соседей нет или все уже посещены.`
    });
}
}

steps.push({
    activeLine: 12,
    queue: [],
    visited: Array.from(visited),
    currentNode: null,
    logs: "Очередь пуста. Алгоритм завершен!"
});

return steps;
};

const STEPS = generateBfsSteps();

export default function BfsViz() {
    const [stepIdx, setStepIdx] = useState(0);
    const [isPlaying, setIsPlaying] = useState(false);

```



```

        />
        <text
          x={n.x} y={n.y}
          textAnchor="middle"
          dy=".3em"
          className="fill-white font-bold text-slate-400"
        >
          {n.label}
        </text>
      </g>
    );
  }}}
</svg>
</div>

```

```

{/* Код и Очередь */}
<div className="flex flex-col gap-4">
  <div className="bg-slate-900 border border-slate-800 p-4">
    <div className="bg-slate-800 text-slate-400 p-4 font-mono">
      <span>bfs(graph, start)</span>
      <span className="text-blue-400">⏮ar {step}</span>
    </div>
    <div className="p-4">
      {[
        { line: 1, code: 'queue = Queue()' },
        { line: 2, code: 'queue.push(start); visit(start);' },
        { line: 3, code: '}' },
        { line: 4, code: 'while not queue.isEmpty() {',
          activeLine: 4 },
        { line: 5, code: '  node = queue.pop()' },
        { line: 6, code: '  for neighbor in graph[node] {',
          activeLine: 6 },
        { line: 7, code: '    if neighbor not in visited {',
          activeLine: 7 },
        { line: 8, code: '      visited.add(neighbor);',
          activeLine: 8 },
        { line: 9, code: '      queue.push(neighbor);',
          activeLine: 9 },
        { line: 10, code: '    }',
          activeLine: 10 },
        { line: 11, code: '  }',
          activeLine: 11 },
        { line: 12, code: '}' },
        { line: 13, code: 'return visited' },
      ]}.map((cl) => {
        const isActive = step.activeLine === cl.line;
        return (

```

ФАЙЛ 29/78: src/components/ChapterImage.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import dijkstraImg from '../assets/dijkstra-kind.jpg';
import bellmanImg from '../assets/bellman-ford-truck.jpg';
import floydImg from '../assets/floyd-network.jpg';
```

```
const imageMap: Record<string, { src: string; alt: string;
  dijkstra: {
    src: dijkstraImg,
    alt: 'Добрый Дейкстра',
    caption: '    Добрый — только позитив!',
    borderColor: 'border-blue-500/30',
    captionColor: 'text-blue-400',
  },
  'bellman-ford': {
    src: bellmanImg,
    alt: 'Фура по болоту',
    caption: '    Тяжёлая, но ей пофиг на ямы!',
    borderColor: 'border-rose-500/30',
    captionColor: 'text-rose-400',
  },
  floyd: {
    src: floydImg,
    alt: 'Все ко Всем Флойд',
    caption: '    Все связаны со всеми!',
    borderColor: 'border-emerald-500/30',
    captionColor: 'text-emerald-400',
  },
};
```

```
export default function ChapterImage({ vizType }: { vizType: string }) {
  const info = imageMap[vizType];
  if (!info) return null;
```

```

        disabled={!prev}
        onClick={() => prev && onGo(prev.id)}
        title={prev ? prev.title : "Это первая тема"}
        className="flex items-center gap-1 px-2.5 py-1
          :bg-slate-700 enabled:hover:text-white disabled:opacity-50"
      >
        <ChevronLeft className="w-4 h-4" />
        <span className="hidden sm:inline">Назад</span>
      </button>
      <span className="text-slate-500 font-mono tabular-nums" />
      {index + 1} / {total}
    </span>
    <button
      disabled={!next}
      onClick={() => next && onGo(next.id)}
      title={next ? next.title : "Это последняя тема"}
      className="flex items-center gap-1 px-2.5 py-1
        :bg-slate-700 enabled:hover:text-white disabled:opacity-50"
    >
      <span className="hidden sm:inline">Вперёд</span>
      <ChevronRight className="w-4 h-4" />
    </button>
  </div>
);
}

return (
  <nav className="grid grid-cols-1 sm:grid-cols-2 gap-4" />
  {prev ? (
    <button
      onClick={() => onGo(prev.id)}
      className="group text-left p-4 rounded-xl bg-white
        transition-all"
    >
      <div className="flex items-center gap-1.5 text-slate-600" />
      <ChevronLeft className="w-3.5 h-3.5" /> Пре
    </div>
  ) : null}
  {next ? (
    <button
      onClick={() => onGo(next.id)}
      className="group text-right p-4 rounded-xl bg-white
        transition-all"
    >
      <div className="flex items-center gap-1.5 text-slate-600" />
      <ChevronRight className="w-3.5 h-3.5" /> След
    </div>
  ) : null}
);
}

```

```

    chapter: Chapter;
    prev?: Chapter;
    next?: Chapter;
    index: number;
    total: number;
    onGo: (id: string) => void;
    onOpenSimulator: (vizId: string) => void;
}

const isTouch = () => typeof window !== "undefined" &&

export const ChapterView: React.FC<Props> = ({ chapter,
  const contentRef = useRef<HTMLDivElement>(null);
  const [anchor, setAnchor] = useState<HintAnchor | null>(null);
  const hideTimer = useRef<number | null>(null);

  useTermHints(contentRef, [chapter.id]);

  const cancelHide = useCallback(() => {
    if (hideTimer.current) {
      window.clearTimeout(hideTimer.current);
      hideTimer.current = null;
    }
  }, []);

  const scheduleHide = useCallback(() => {
    cancelHide();
    hideTimer.current = window.setTimeout(() => setAnchor(anchor), [cancelHide]);

  useEffect(() => {
    setAnchor(null);
  }, [chapter.id]);

  const open = useCallback(
    (el: HTMLElement) => {
      const termId = el.dataset.termId;
      if (!termId) return;

```

```

        className="prose prose-invert max-w-none"
        onMouseOver={handlePointerOver}
        onMouseOut={handlePointerOut}
        onClick={handleClick}
        onFocus={handleFocus}
        dangerouslySetInnerHTML={{ __html: chapter.content }}
      />

    <p className="mt-6 flex items-start gap-2 text-
      ed-lg px-3 py-2">
      <MousePointerClick className="w-4 h-4 shrink-0" />
      <span>
        Подчёркнутые термины – интерактивные: наведи
        объяснение на пальцах, мини-анимацию и кноп
      </span>
    </p>

    {viz && (
      <section id="chapter-viz" className="mt-8 scr
        <div className="flex items-center justify-b
          <h3 className="flex items-center gap-2 te
            <Sparkles className="w-4 h-4 text-indigo
              Демонстрация: {viz.title}
            </h3>
            <button
              onClick={() => onOpenSimulator(chapter.
              className="flex items-center gap-1.5 te
            ate-300 hover:text-white hover:border-indigo
              >
                <Maximize2 className="w-3.5 h-3.5" /> P
              </button>
            </div>
            {viz.hint && <p className="text-xs text-sla
              <viz.Component />
            </section>
          )}

    <ChapterNav prev={prev} next={next} index={index}

```

```

{ line: "          if (visited[to]) {", highlight: "normal" },
{ line: "              low[v] = min(low[v], tin[to]);", highlight: "normal" },
{ line: "          } else {", highlight: "normal" },
{ line: "              dfs(to, v);", highlight: "normal" },
{ line: "              low[v] = min(low[v], low[to]);", highlight: "normal" },
{ line: "          }, highlight: "normal" },
{ line: "          if (low[to] > tin[v]) {", highlight: "normal" },
{ line: "              // ЭТО МОСТ!", highlight: "normal" },
{ line: "          }, highlight: "normal" },
{ line: "      }", highlight: "normal" },
{ line: "  }", highlight: "normal" },
{ line: "}", highlight: "normal" },
];

```

```

interface DfsStep {
  currentNode: number | null;
  visitedIndices: number[];
  tin: Record<number, number>;
  low: Record<number, number>;
  stack: number[];
  bridges: string[];
  log: string;
  activeEdge: [number, number] | null;
  backEdge: [number, number] | null;
  activeCodeLine: number[];
}

```

```

const GRAPH_NODES = [
  { id: 0, label: "A", x: 100, y: 120 },
  { id: 1, label: "B", x: 260, y: 120 },
  { id: 2, label: "C", x: 180, y: 200 },
  { id: 3, label: "D", x: 180, y: 300 },
  { id: 4, label: "E", x: 100, y: 370 },
  { id: 5, label: "F", x: 260, y: 370 },
  { id: 6, label: "G", x: 340, y: 200 }
];

```

```

const GRAPH_EDGES = [

```

```

    },
    {
        currentNode: 2, visitedIndices: [0,1,6,2],
        tin: {0:1,1:2,6:3,2:4}, low: {0:1,1:2,6:3,2:4}, sta
        activeEdge: [1,2], backEdge: null,
        log: "Из В идём в С. tin[C]=4, low[C]=4.",
        activeCodeLine: [4,5,7,10,11]
    },
    {
        currentNode: 2, visitedIndices: [0,1,6,2],
        tin: {0:1,1:2,6:3,2:4}, low: {0:1,1:2,6:3,2:1}, sta
        activeEdge: null, backEdge: [2,0],
        log: "Видим соседа А (уже посещён)! Обратное ребро
        activeCodeLine: [7,8,9]
    },
    {
        currentNode: 3, visitedIndices: [0,1,6,2,3],
        tin: {0:1,1:2,6:3,2:4,3:5}, low: {0:1,1:2,6:3,2:1,3
        activeEdge: [2,3], backEdge: null,
        log: "Идём С→D. tin[D]=5, low[D]=5.",
        activeCodeLine: [4,5,7,10,11]
    },
    {
        currentNode: 4, visitedIndices: [0,1,6,2,3,4],
        tin: {0:1,1:2,6:3,2:4,3:5,4:6}, low: {0:1,1:2,6:3,2
        activeEdge: [3,4], backEdge: null,
        log: "D→E. tin[E]=6, low[E]=6.",
        activeCodeLine: [4,5,7,10,11]
    },
    {
        currentNode: 5, visitedIndices: [0,1,6,2,3,4,5],
        tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6
        activeEdge: [4,5], backEdge: null,
        log: "E→F. tin[F]=7, low[F]=7.",
        activeCodeLine: [4,5,7,10,11]
    },
    {
        currentNode: 5, visitedIndices: [0,1,6,2,3,4,5],

```

```

    activeCodeLine: [11,13,16,17,18]
  },
  {
    currentNode: null, visitedIndices: [0,1,6,2,3,4,5],
    tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:1,6:1},
    activeEdge: null, backEdge: null,
    log: "Готово! Найдены мосты: (B,G) и (C,D). Алгоритм закончен.",
    activeCodeLine: []
  },
];

```

```

export function DfsBridgesSimulator() {
  const [dfsIdx, setDfsIdx] = useState(0);
  const [dfsAuto, setDfsAuto] = useState(false);
  const dfsTimer = useRef<NodeJS.Timeout | null>(null);

  useEffect(() => {
    if (dfsAuto) {
      dfsTimer.current = setInterval(() => {
        setDfsIdx(prev => {
          if (prev >= DFS_STEPS.length - 1) { setDfsAuto(false);
          return prev + 1;
        });
      }, 2500);
    }
    return () => { if (dfsTimer.current) clearInterval(dfsTimer.current);
  }, [dfsAuto]);

```

```

const step = DFS_STEPS[dfsIdx];

```

```

return (
  <div className="space-y-4">
    <div className="flex items-center justify-between">
      <h4 className="text-base font-bold text-white">Решение задачи
      <div className="flex items-center gap-1 bg-slate-800 p-1">
        <button onClick={() => { setDfsAuto(false); setDfsIdx(0);
          disabled={dfsIdx === 0}
          className="p-1.5 hover:bg-slate-700 rounded"

```

```

        let sc = "#475569", sw = 2, dash = "none"
        if (bridge) { sc = "#eab308"; sw = 4; }
        else if (activeTree) { sc = "#10b981"; sw = 2; }
        else if (activeBack) { sc = "#f43f5e"; sw = 2; }
        else if (step.visitedIndices.includes(e.u)) { sc = "#f43f5e"; sw = 2; }
        return <line key={e.key} x1={u.x} y1={u.y} x2={v.x} y2={v.y}
            stroke={sc} strokeWidth={sw} strokeDash={dash}
            className="transition-all duration-500" />
    }}}
    {step.currentNode !== null && step.currentNode.id === n.id ? (
        const n = GRAPH_NODES.find(x => x.id === n.id)
        if (!n) return null;
        return <circle cx={n.x} cy={n.y} r={18} fill={sc} />
    ) : null;
    })()}
    {GRAPH_NODES.map(n => {
        const cur = step.currentNode === n.id;
        const vis = step.visitedIndices.includes(n.id);
        const st = step.stack.includes(n.id);
        return <g key={n.id}>
            <circle cx={n.x} cy={n.y} r={12}
                fill={cur ? "#10b981" : st ? "#064e3b" : "#f43f5e"}
                stroke={cur ? "#fff" : st ? "#34d399" : "#f43f5e"}
                strokeWidth={2.5}
                className="transition-all duration-300" />
            <text x={n.x} y={n.y+3} textAnchor="middle">
                {n.label}</text>
            {vis && (
                <rect x={n.x-18} y={n.y-28} width={36} height={10}
                    fill="#f43f5e" className="transition-all duration-300" />
                <text x={n.x} y={n.y-20} textAnchor="middle">
                    {step.low[n.id] || "?"}</text>
                </>
            )}
        </g>
    )}
    </svg>

```

```

        {step.stack.length === 0
        ? <div className="text-[10px] text-slate-400"
        : step.stack.map((id, i) => {
            <div key={id} className={`px-2 py-0
            i === step.stack.length-1
            ? "bg-emerald-600/20 text-emerald-500"
            : "bg-slate-900 text-slate-400"
            }`} >Вершина {GRAPH_NODES.find(n=>n.id === id).name}
            <span className="text-slate-500 ml-2" >{step.stack[id]}
            </div>
        ))
    }
  </div>
</div>

{/* Log */}
<div className="bg-indigo-950/10 border border-indigo-500 p-4" >
  <p className="text-[11px] text-slate-300 leading-tight" >Шаг {step.id}
  <div className="mt-2 pt-2 border-t border-slate-700" >
    <span className="text-slate-400" >Мосты:
    <span className="font-mono font-bold text-slate-300" >{
      {step.bridges.length > 0 ? step.bridges.map(b => b.id).join(' ') : ''}
    }
    </span>
  </div>
</div>
</div>
</div>
</div>
);
}

```

ФАЙЛ 33/78: src/components/DijkstraViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import { useState, useEffect } from 'react';
```

```
const adjList: { [key: string]: { v: string; w: number } } =
  nodes.forEach(n => (adjList[n.id] = []));
edges.forEach(e => {
  adjList[e.u].push({ v: e.v, w: e.w });
});
```

```
const codeLines = [
  { line: 1, text: 'function dijkstra(graph, start):' },
  { line: 2, text: '    dist = {v: ∞}; dist[start] = 0' },
  { line: 3, text: '    pq = PriorityQueue(); pq.push(start, 0)' },
  { line: 4, text: '    while not pq.empty():' },
  { line: 5, text: '        d, u = pq.pop() // Извлекаем элемент с минимальным расстоянием' },
  { line: 6, text: '        if d > dist[u]: continue' },
  { line: 7, text: '        for v, weight in graph.neighbors(u):' },
  { line: 8, text: '            if dist[u] + weight < dist[v]:' },
  { line: 9, text: '                dist[v] = dist[u] + weight' },
  { line: 10, text: '                pq.push(v, dist[v])' },
  { line: 11, text: '    return dist // Все кратчайшие расстояния найдены' },
];
```

```
const [steps, setSteps] = useState<Step[]>([]);
const [currentStepIndex, setCurrentStepIndex] = useState(0);
const [isPlaying, setIsPlaying] = useState(false);
```

```
useEffect(() => {
  const simulationSteps: Step[] = [];
  const dist: Record<string, number> = { A: 0, B: 999 };
  const visited: Record<string, boolean> = { A: false, B: false };
  const prev: Record<string, string | null> = { A: null, B: null };

  simulationSteps.push({
    currentNode: null, checkingEdge: null,
    distances: { ...dist }, visited: { ...visited },
    log: ' Дейкстра готов к доставке. Начинаем из Пикет-Пост-Оффиса',
    activeCodeLine: 2,
  });
```

```
for (let iter = 0; iter < nodes.length; iter++) {
```

```

        log: `    Улучшение (Релаксация)! Новое кратчайшее
        activeCodeLine: 9,
    });
  }
}
}

simulationSteps.push({
  currentNode: null, checkingEdge: null,
  distances: { ...dist }, visited: { ...visited },
  log: `    Дейкстра всё доставил! Все возможные кратчайшие
  activeCodeLine: 11,
});

setSteps(simulationSteps);
}, []));

useEffect(() => {
  let timer: any = null;
  if (isPlaying && currentStepIndex < steps.length - 1) {
    timer = setTimeout(() => {
      setCurrentStepIndex((prev) => prev + 1);
    }, 1500);
  } else if (currentStepIndex >= steps.length - 1) {
    setIsPlaying(false);
  }
  return () => clearTimeout(timer);
}, [isPlaying, currentStepIndex, steps.length]);

const currentStep = steps[currentStepIndex] || null;

if (!currentStep) return <div className="text-white">

return (
  <div className="bg-slate-800/90 p-6 rounded-2xl border
    {/* Шапка симулятора */}
    <div className="flex flex-wrap items-center justify
      <div className="flex items-center gap-3">

```

```

<div className="w-full lg:w-2/3 bg-indigo-950/20
  justify-center min-h-[400px]">
  <div className="absolute top-3 left-3 bg-indigo-950/20
    font-bold shadow-sm">
    Шаг {currentStepIndex + 1} из {steps.length}
  </div>

  <svg className="w-full h-auto max-h-[500px]" >
    <defs>
      <marker id="arrow-dh-normal" markerWidth=
        <path d="M0,0 L6,3 L0,6 z" fill="#475568" />
      </marker>
      <marker id="arrow-dh-active" markerWidth=
        <path d="M0,0 L6,3 L0,6 z" fill="#f59e0b" />
      </marker>
      <marker id="arrow-dh-opt" markerWidth="6"
        <path d="M0,0 L6,3 L0,6 z" fill="#10b981" />
      </marker>
    </defs>
    {/* Рисуем */}
    {edges.map((edge, idx) => {
      const uNode = nodes.find((n) => n.id === edge.u)
      const vNode = nodes.find((n) => n.id === edge.v)
      const isChecking =
        currentStep.checkingEdge &&
        (currentStep.checkingEdge.u === edge.u &&
         currentStep.checkingEdge.v === edge.v)

      const isOptimal = currentStep.previous[edge.id]

      let marker = 'url(#arrow-dh-normal)';
      if (isChecking) marker = 'url(#arrow-dh-active)';
      else if (isOptimal) marker = 'url(#arrow-dh-opt)';

      return (
        <g key={idx}>
          <line
            x1={uNode.x} y1={uNode.y}
            x2={vNode.x} y2={vNode.y}
            stroke={isChecking ? 'red' : isOptimal ? 'green' : 'black'}
            strokeDash={isChecking ? [4, 4] : isOptimal ? [2, 2] : []}
            strokeWidth={isChecking ? 2 : isOptimal ? 2 : 1}
          />
          <marker
            d="arrow-dh-normal"
            d="arrow-dh-active"
            d="arrow-dh-opt"
            fill="#475568"
            fill="#f59e0b"
            fill="#10b981"
            fillOpacity={isChecking ? 0.5 : isOptimal ? 0.5 : 1}
            size={isChecking ? 10 : isOptimal ? 10 : 10}
            stroke="black"
            strokeDash={isChecking ? [4, 4] : isOptimal ? [2, 2] : []}
            strokeWidth={isChecking ? 2 : isOptimal ? 2 : 1}
            type="url"
            width={isChecking ? 10 : isOptimal ? 10 : 10}
            height={isChecking ? 10 : isOptimal ? 10 : 10}
            offset="0,0"
            orient="auto"
            refX={vNode.x}
            refY={vNode.y}
            style={{position: 'absolute', top: 0, left: 0}}
            stroke="black"
            strokeDash={isChecking ? [4, 4] : isOptimal ? [2, 2] : []}
            strokeWidth={isChecking ? 2 : isOptimal ? 2 : 1}
            type="url"
            width={isChecking ? 10 : isOptimal ? 10 : 10}
            height={isChecking ? 10 : isOptimal ? 10 : 10}
            offset="0,0"
            orient="auto"
            refX={vNode.x}
            refY={vNode.y}
            style={{position: 'absolute', top: 0, left: 0}}
          />
        </g>
      )
    })}
  </svg>

```

```

        <text x={node.x} y={node.y + 5} fill=
            {node.id}
        </text>
        <text x={node.x} y={node.y - 28} fill=
"middle" className="bg-indigo-950 px-1 py-0
            {dist === 999 ? '∞' : `d=${dist}`}
        </text>
        <text x={node.x} y={node.y + 35} fill=
            {node.name.split(' ')[0]}
        </text>
    </g>
  );
  }}}
</svg>
<div className="w-full bg-slate-950/80 p-4 rounded-2xl" >
  <p className="font-semibold text-amber-400" >
  <p className="min-h-[40px] flex items-center" >
</div>
</div>

{/* Список смежности */}
<div className="w-full lg:w-1/3 bg-emerald-950/80" >
  <h4 className="text-lg font-bold text-emerald-400" >
  <div className="space-y-2.5 flex-1 overflow-y-hidden" >
    {Object.keys(adjList).map((u) => {
      const isCurrentNode = currentStep.currentNode === u;
      return (
        <div key={u} className={`p-3 rounded-lg ${
          isCurrentNode ? 'bg-emerald-900/60 text-white' : 'text-slate-300'
        }`} >
          <div className="flex items-center gap-2" >
            <span className={`px-2 py-0.5 rounded` +
              (isCurrentNode ? 'bg-emerald-900/60 text-white' : 'text-slate-300')} >
              {u}
            </span>
            <span className="text-slate-400" >→</span>
          </div>
        </div>
      );
    })}
  </div>

```

```

const dist = currentStep.distances[n]
const prev = currentStep.previous[n]
const vis = currentStep.visited[n]
const isCurr = currentStep.currentN

return (
  <tr key={n.id} className={`border
    isCurr ? 'bg-indigo-950/60 for
  }`>
    <td className="py-2.5 px-3 flex
      <span className={`w-2 h-2 rou
        {n.name}
      </td>
      <td className="py-2.5 px-3 font
        {dist === 999 ? '∞' : dist}
      </td>
      <td className="py-2.5 px-3 text
        {prev || '-'}
      </td>
    </tr>
  );
  }}}
</tbody>
</table>
</div>
</div>
</div>

{/* Код алгоритма */}
<div className="w-full lg:w-1/2 bg-slate-900/60
  <div>
    <h4 className="text-lg font-bold text-slate
    <div className="bg-slate-950/80 rounded-lg p
      {codeLines.map((item) => {
        const isActive = currentStep.activeCode
        return (
          <div key={item.line} className={`py-1
            isActive ? 'bg-indigo-900/80 text

```

```

-----
  /* 4 */ "      memo[n] = fibMemo(n - 1, memo) + fibMemo
  /* 5 */ "      return memo[n];",
  /* 6 */ "}"
];

```

```

export function DPVisualizer() {
  const [targetN, setTargetN] = useState<number>(5);
  const [steps, setSteps] = useState<DPStep[]>([]);
  const [currentStepIndex, setCurrentStepIndex] = useSt
  const [isPlaying, setIsPlaying] = useState<boolean>(f
  const [speed] = useState<number>(800);
  const [activeTab, setActiveTab] = useState<'table' |

```

```

useEffect(() => {
  const generatedSteps: DPStep[] = [];
  let stepId = 0;
  const memoMap: { [key: number]: number } = {};

  function simulateFib(n: number): number {
    generatedSteps.push({
      id: stepId++,
      n,
      action: 'call',
      desc: `Вызов fibMemo(${n}). Проверяем наличие в
      codeLine: 2,
      memoState: { ...memoMap }
    });

```

```

    if (n in memoMap) {
      generatedSteps.push({
        id: stepId++,
        n,
        action: 'memo_hit',
        desc: `✂ Кэш-хит! fibMemo(${n}) уже посчитано
        codeLine: 2,
        memoState: { ...memoMap }
      });
      return memoMap[n];

```

```

    const finalRes = simulateFib(targetN);

    generatedSteps.push({
      id: stepId++,
      n: targetN,
      action: 'done',
      desc: `  Расчет завершен! fibMemo(${targetN}) = $
      codeLine: 6,
      memoState: { ...memoMap }
    });

    setSteps(generatedSteps);
    setCurrentStepIndex(0);
    setIsPlaying(false);
  }, [targetN]);

  useEffect(() => {
    let timer: any = null;
    if (isPlaying && currentStepIndex < steps.length -
      timer = setTimeout(() => {
        setCurrentStepIndex(prev => prev + 1);
      }, speed);
    } else if (currentStepIndex >= steps.length - 1) {
      setIsPlaying(false);
    }
    return () => clearTimeout(timer);
  }, [isPlaying, currentStepIndex, steps, speed]);

  const currentStep = steps[currentStepIndex] || null;
  const memoState = currentStep?.memoState || {};

  return (
    <div className="bg-slate-900 rounded-2xl border bor
      {/* Header & Controls */}
      <div className="flex flex-col lg:flex-row lg:item
        <div>
          <div className="flex items-center gap-3 mb-2">

```

```

        title="Шаг назад"
      >
        <ChevronLeft className="w-5 h-5" />
      </button>
      <button
        onClick={() => setIsPlaying(!isPlaying)}
        className={`flex items-center gap-1.5 px-2 py-2
          isPlaying
            ? 'bg-amber-500 text-slate-950 hover:bg-amber-600
            : 'bg-emerald-600 text-white hover:bg-emerald-700
          `}
      >
        {isPlaying ? <Pause className="w-4 h-4" /> : <Play className="w-4 h-4" />}
        {isPlaying ? 'Пауза' : 'Пуск'}
      </button>
      <button
        onClick={() => { setIsPlaying(false); setSteps([0]); }}
        disabled={currentStepIndex === steps.length - 1}
        className="p-2 text-slate-400 hover:text-slate-500"
        title="Шаг вперед"
      >
        <ChevronRight className="w-5 h-5" />
      </button>
    </div>
  </div>
</div>

```

```

{/* DP Table View */}
<div className="mb-8 bg-slate-950 p-4 md:p-6 rounded-lg" >
  <h4 className="text-xs font-bold uppercase tracking-wide" >Динамическое программирование
  <div className="flex flex-wrap items-center gap-2" >
    {Array.from({ length: targetN }, (_, i) => i + 1).map((num) => {
      const isCached = num in memoState || num <= 2;
      const val = num <= 2 ? 1 : memoState[num];
      const isCurrent = currentStep?.n === num;

      return (
        <div

```

```

        ? 'border-emerald-500 text-emerald-400 bg
        : 'border-transparent text-slate-400 hove
    }` }
  >
    <Eye className="w-4 h-4" />
    Стен-бай-стен состояние
  </button>
  <button
    onClick={() => setActiveTab('code')}
    className={`flex items-center gap-2 px-6 py-3
      activeTab === 'code'
        ? 'border-emerald-500 text-emerald-400 bg
        : 'border-transparent text-slate-400 hove
    }` }
  >
    <Code className="w-4 h-4" />
    Интерактивный код
  </button>
</div>

{/* Tab Content */}
<div className="min-h-[320px] flex flex-col justify
  {activeTab === 'table' && {
    <div className="bg-slate-950 p-6 rounded-2xl
      <h5 className="font-bold text-white text-sm
        <span className="text-emerald-400"> </span>
      </h5>
      <p className="text-sm text-slate-400">
        В классической рекурсии для  $N=\{targetN\}$  п
        вычисленное значение в таблице, мы момента
      </p>
      <div className="p-4 bg-slate-900 rounded-xl
        t-slate-300">
        <span>Всего шагов симуляции: {steps.length}
        <span className="text-emerald-400 font-bo
      </div>
    </div>
  }
})

```

```
        <span>Шаг {currentStepIndex + 1} из {steps.length}</span>
      </div>
      <div className="h-2 bg-slate-950 rounded-full overflow-hidden">
        <div
          className="h-full bg-gradient-to-r from-emerald-500 to-teal-500"
          style={{ width: `${(currentStepIndex + 1) * 100 / steps.length}%` }}
        ></div>
      </div>
    </div>
  </div>
);
}
```

ФАЙЛ 35/78: src/components/EulerSimulator.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 10

```
import { useState } from "react";
import { Undo } from "lucide-react";
```

```
const C1_NODES = [
  { id: 0, label: "A", x: 190, y: 60 },
  { id: 1, label: "B", x: 280, y: 140 },
  { id: 2, label: "C", x: 100, y: 140 },
  { id: 3, label: "D", x: 280, y: 250 },
  { id: 4, label: "E", x: 100, y: 250 }
];
```

```
const C1_EDGES = [
  { u: 0, v: 1 }, { u: 0, v: 2 },
  { u: 1, v: 3 }, { u: 2, v: 4 },
  { u: 1, v: 2 },
  { u: 3, v: 4 }
];
```

```
export function EulerSimulator() {
```

```

    } else {
      setC1Msg(` ЭЙЛЕРОВ ПУТЬ! Все рёбра пройдены! C
    }
    setC1Done(true);
  } else {
    setC1Msg(`Пройдено рёбер: ${nextEdges.length} / $
  }
};

```

```

return (
  <div className="space-y-4">
    <div className="flex items-center justify-between">
      <h4 className="text-base font-bold text-white">
        <button onClick={resetC1} className="flex items
          old border border-slate-700 transition-all">
            <Undo className="h-3.5 w-3.5" /> C6poc
          </button>
        </div>

    <div className="bg-slate-950 rounded-xl p-4 borde
      -hidden">
      <div className="absolute inset-0 bg-[radial-gra
      <svg className="w-full h-[260px] z-10 relative"
        {C1_EDGES.map((e, i) => {
          const u = C1_NODES.find(n => n.id === e.u)!
          const v = C1_NODES.find(n => n.id === e.v)!
          const key = `${Math.min(e.u,e.v)}-${Math.ma
          const used = c1VisitedEdges.includes(key);
          return <line key={i} x1={u.x} y1={u.y} x2={
            stroke={used ? "#6366f1" : "#475569"}
            strokeWidth={used ? 4 : 2}
            className="transition-all duration-300" />
          }}}
      {C1_NODES.map(n => {
        const isLast = c1Path[c1Path.length-1] === n
        const visited = c1Path.includes(n.id);
        return <g key={n.id} className="cursor-poin
          {isLast && <circle cx={n.x} cy={n.y} r={1

```

```

    id: number;
    question: string;
    options: string[];
    correctAnswer: number;
    explanation: string;
}

const quizQuestions: Question[] = [
  {
    id: 1,
    question: 'Что мы делаем в алгоритме Краскала, если  

      ребро (красный) цвет?',
    options: [
      'Мы радуемся и добавляем его в минимальное остовное дерево',
      'Мы перекрашиваем их в синий цвет и делим граф по компонентам',
      'Мы безжалостно ВЫБРАСЫВАЕМ это ребро, иначе получим цикл',
      'Мы вызываем алгоритм Дейкстры для поиска нового ребра'
    ],
    correctAnswer: 2,
    explanation: 'Верно! Если вершины уже в одном клане  

      это нарушит структуру дерева.'
  },
  {
    id: 2,
    question: 'Почему алгоритм Дейкстры впадает в ступор?',
    options: [
      'Потому что он написан только для положительных весов',
      'Он жадный и считает, что каждый следующий шаг дает минимальный прирост',
      'Отрицательные ребра создают гравитационный коллапс',
      'Дейкстра просто не любил отрицательные балансы на ребрах'
    ],
    correctAnswer: 1,
    explanation: 'Именно! Дейкстра уверен: если ты приедешь в пункт назначения  

      за минимальное время, то для чего нужен алгоритм Беллмана-Форда.'
  },
  {
    id: 3,

```

```
export const ExpressQuiz: React.FC = () => {
  const [currentQIndex, setCurrentQIndex] = useState<number>(0);
  const [selectedOption, setSelectedOption] = useState<string>('');
  const [isAnswered, setIsAnswered] = useState<boolean>(false);
  const [score, setScore] = useState<number>(0);
  const [showResults, setShowResults] = useState<boolean>(false);

  const currentQ = quizQuestions[currentQIndex];

  const handleSelectOption = (index: number) => {
    if (isAnswered) return;
    setSelectedOption(index);
    setIsAnswered(true);
    if (index === currentQ.correctAnswer) {
      setScore(prev => prev + 1);
    }
  };

  const handleNext = () => {
    if (currentQIndex + 1 < quizQuestions.length) {
      setCurrentQIndex(prev => prev + 1);
      setSelectedOption(null);
      setIsAnswered(false);
    } else {
      setShowResults(true);
    }
  };

  const handleRestart = () => {
    setCurrentQIndex(0);
    setSelectedOption(null);
    setIsAnswered(false);
    setScore(0);
    setShowResults(false);
  };

  if (showResults) {
    return (

```

```

    <span className="bg-purple-600 text-white px-4">
      Экспресс-тест
    </span>
    <h3 className="text-xl font-bold text-white">
  </div>
  <span className="text-sm font-mono text-slate-400">
    Bonpoc {currentQIndex + 1} / {quizQuestions.length}
  </span>
</div>

<div className="mb-8">
  <h4 className="text-xl md:text-2xl font-bold text-slate-800">
    {currentQ.question}
  </h4>

  <div className="space-y-4">
    {currentQ.options.map((option, idx) => {
      const isSelected = selectedOption === idx;
      const isCorrect = idx === currentQ.correctAnswer;

      let optionStyle = "bg-slate-800/80 hover:bg-slate-700/80";
      if (isAnswered) {
        if (isCorrect) {
          optionStyle = "bg-emerald-950/80 border border-emerald-500";
        } else if (isSelected) {
          optionStyle = "bg-rose-950/80 border-rose-500";
        } else {
          optionStyle = "bg-slate-800/40 border-slate-700";
        }
      }
    })}

    return (
      <div
        key={idx}
        onClick={() => handleSelectOption(idx)}
        className={"flex items-start gap-4 p-4"}
      >
        <div className="mt-0.5 shrink-0">

```

```
        (!isAnswered
          ? "bg-slate-800 text-slate-500 border border-slate-500"
          : "bg-indigo-600 hover:bg-indigo-500 active:bg-indigo-700")
      )
    >
      {currentQIndex + 1 < quizQuestions.length ? 'next' : 'finish'}
    </button>
  </div>
</div>
);
};
```

ФАЙЛ 37/78: src/components/FloydViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import { useState, useEffect } from 'react';

interface Step {
  k: number; i: number; j: number;
  matrix: number[][]; updated: boolean;
  log: string; activeCodeLine: number;
}

export default function FloydViz() {
  const [steps, setSteps] = useState<Step[]>([]);
  const [currentStepIndex, setCurrentStepIndex] = useState(0);
  const [isPlaying, setIsPlaying] = useState(false);

  const nodeNames = ['А 0', 'Б 1', 'В 2', 'Г 3', 'Д 4', 'Е 5'];
  const nodesSvg = [
    { id: 0, name: 'Аэропорт 0', x: 100, y: 60 },
    { id: 1, name: 'Аэропорт 1', x: 250, y: 60 },
    { id: 2, name: 'Аэропорт 2', x: 400, y: 60 },
    { id: 3, name: 'Аэропорт 3', x: 100, y: 180 },
    { id: 4, name: 'Аэропорт 4', x: 400, y: 180 },
  ];
```

```

simulationSteps.push({
  k: -1, i: -1, j: -1, matrix: dist.map(r => [...r]),
  log: '  Инициализация матрицы прямых путей,  $\infty$  (999)',
  activeCodeLine: 2,
});

const N = 7;
for (let k = 0; k < N; k++) {
  simulationSteps.push({
    k, i: -1, j: -1, matrix: dist.map(r => [...r]),
    log: `+ Внешний цикл. Разрешаем пути через пром. узлы`,
    activeCodeLine: 3,
  });

  for (let i = 0; i < N; i++) {
    for (let j = 0; j < N; j++) {
      if (i === j || i === k || j === k) continue;

      const oldVal = dist[i][j];
      const viaVal = dist[i][k] + dist[k][j];
      if (dist[i][k] !== 999 && dist[k][j] !== 999 && viaVal < oldVal) {
        dist[i][j] = viaVal;
        simulationSteps.push({
          k, i, j, matrix: dist.map(r => [...r]),
          log: `  Улучшение пути [${i}]→[${j}] через пром. узел [${k}].`,
          activeCodeLine: 7,
        });
      }
    }
  }
}

simulationSteps.push({
  k: 7, i: -1, j: -1, matrix: dist.map(r => [...r]),
  log: '  Все пары отработаны. Алгоритм Флойда завершён.',
  activeCodeLine: 8,
});

```

```

        <button onClick={() => { setIsPlaying(false);
            ; }} className="px-3 py-2 bg-emerald-600 te
    </div>
</div>

<div className="flex flex-col lg:flex-row gap-6 ml
    {/* Граф */}
    <div className="w-full lg:w-2/3 bg-indigo-950/2
        stify-center min-h-[400px]">
        <div className="absolute top-3 left-3 bg-indi
            ont-bold shadow-sm">
            {currentStep.k >= 0 && currentStep.k < 7 ?
        </div>

    <svg className="w-full h-auto max-h-[500px]" >
        <defs>
            <marker id="arrow-fw-normal" markerWidth=
            <path d="M0,0 L6,3 L0,6 z" fill="#475569" />
            <marker id="arrow-fw-active" markerWidth=
            <path d="M0,0 L6,3 L0,6 z" fill="#10b981" />
            <marker id="arrow-fw-via" markerWidth="6"
            th d="M0,0 L6,3 L0,6 z" fill="#818cf8" /></
        </defs>
        {Object.keys(adjList).flatMap((uStr) => {
            const u = parseInt(uStr);
            return adjList[u].map((edge) => {
                const uNode = nodesSvg.find((n) => n.id
                const vNode = nodesSvg.find((n) => n.id
                const isTarget = currentStep.i === u &&
                const isVia = (currentStep.i === u && c
                let strokeColor = '#475569'; let marker
                if (isTarget) { strokeColor = '#10b981'
                else if (isVia) { strokeColor = '#818cf8
                return (
                    <g key={` ${u} - ${edge.v} `}>
                        <line x1={uNode.x} y1={uNode.y} x2=
                    markerEnd={marker} strokeDasharray={isTarg
                        <circle cx={({uNode.x + vNode.x) / 2

```

```

const u = parseInt(uStr);
const isI = currentStep.i === u;
const isK = currentStep.k === u;
return (
  <div key={u} className={`flex flex-wrap
    isI ? 'bg-amber-900/40 border-amber-
    isK ? 'bg-indigo-900/40 border-ind
  }`>
    <span className="font-bold text-white
e-400">→</span>
    {adjList[u].map((edge, idx) => {
      const isUpd = currentStep.i === u
      const isVia = currentStep.k !== -
rentStep.j === edge.v);
      return (
        <span key={idx} className={`px-1
dow' : isVia ? 'bg-indigo-500 text-white bo
        {edge.v} ({edge.w})
      </span>
      )
    })}
  </div>
);
}}}
</div>
</div>
</div>

<div className="flex flex-col lg:flex-row gap-6">
  /* Матрица расстояний */
  <div className="w-full lg:w-1/2 bg-rose-950/10">
    <div className="flex justify-between items-center">
      <h4 className="text-lg font-bold text-rose-100">
      </div>
    <div className="overflow-x-auto">
      <table className="w-full text-center border-collapse">
        <thead><tr className="bg-rose-950/40 text-rose-100">
          <th className="p-2 border-r border-rose-100">

```

```
        </div>
      )})
    </div>
  </div>
</div>
</div>
</div>
);
}
```

ФАЙЛ 38/78: src/components/GraphTraversalViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import { useState, useEffect, useMemo } from 'react';
import { Play, Pause, SkipForward, Undo, RefreshCw } from 'lucide-react';
```

```
type Node = { id: string; label: string; x: number; y: number };
type Edge = { u: string; v: string };
```

```
const nodes: Node[] = [
  { id: 'A', label: 'A', x: 200, y: 40 },
  { id: 'B', label: 'B', x: 100, y: 120 },
  { id: 'C', label: 'C', x: 300, y: 120 },
  { id: 'D', label: 'D', x: 50, y: 200 },
  { id: 'E', label: 'E', x: 150, y: 200 },
  { id: 'F', label: 'F', x: 250, y: 200 },
  { id: 'G', label: 'G', x: 350, y: 200 },
];
```

```
const edges: Edge[] = [
  { u: 'A', v: 'B' }, { u: 'A', v: 'C' },
  { u: 'B', v: 'D' }, { u: 'B', v: 'E' },
  { u: 'C', v: 'F' }, { u: 'C', v: 'G' },
];
```

```

        steps.push({ type: 'backtrack', node: v, desc:
            ueOrStack: [...stack], visitedNodes: Array.from(
        }

    dfs(start);
    return steps;
}

function generateBFSSteps(start: string) {
    const steps: Action[] = [];
    const vis = new Set<string>();
    const q: string[] = [];

    q.push(start);
    vis.add(start);
    steps.push({ type: 'visit', node: start, desc: `Нача
        ray.from(vis)] }));

    while (q.length > 0) {
        const v = q[0];
        steps.push({ type: 'process', node: v, desc: `Д
            q], visitedNodes: [...Array.from(vis)] }));
        q.shift();

        for (const u of adj[v]) {
            if (!vis.has(u)) {
                vis.add(u);
                q.push(u);
                steps.push({ type: 'visit', node: u, de
                    tack: [...q], visitedNodes: [...Array.from(
                }
            }
        }

        steps.push({ type: 'backtrack', node: '', desc: `Оч
            om(vis)] }));

    return steps;
}

```

```

        <button onClick={() => setMode("dfs")}
            className={`px-4 py-2 text-xs font-
text-slate-400 hover:text-slate-300`} `>
            DFS (В глубину)
        </button>
        <button onClick={() => setMode("bfs")}
            className={`px-4 py-2 text-xs font-
"text-slate-400 hover:text-slate-300`} `>
            BFS (В ширину / Волна)
        </button>
    </div>

    <div className="flex flex-col md:flex-row g
        {/* SVG Graph View */}
        <div className="bg-slate-900 border bor
ive min-h-[300px]">
            <svg className="w-full h-full max-
                {/* Edges */}
                {edges.map((e, i) => {
                    const u = nodes.find(n => n
                    const v = nodes.find(n => n
                    // Check if edge is part of
                    const edgeTraversed = isVis

                    return (
                        <line key={i} x1={u.x}
                            className={`stroke
500' : 'stroke-emerald-500') : 'stroke-slate
                        });
                    )))

                {/* Nodes */}
                {nodes.map(n => {
                    const visited = isVisited(n
                    const current = isCurrent(n

                    let fillColor = "fill-slate
                    let strokeColor = "stroke-s

```



```

    emoji: string;
    symptom: string;
    priority: number; // 1 to 99 (Severity)
    animState: 'in' | 'idle' | 'winner' | 'out';
}

```

// Max-heap helpers for Patients

```

function siftUp(arr: Patient[]): Patient[] {
    const a = arr.map(x => ({ ...x }));
    let idx = a.length - 1;
    while (idx > 0) {
        const parent = Math.floor((idx - 1) / 2);
        if (a[parent].priority < a[idx].priority) {
            [a[parent], a[idx]] = [a[idx], a[parent]];
            idx = parent;
        } else break;
    }
    return a;
}

```

```

function siftDown(arr: Patient[]): Patient[] {
    const a = arr.map(x => ({ ...x }));
    const n = a.length;
    let idx = 0;
    while (true) {
        let largest = idx;
        const l = 2 * idx + 1;
        const r = 2 * idx + 2;
        if (l < n && a[l].priority > a[largest].priority) largest = l;
        if (r < n && a[r].priority > a[largest].priority) largest = r;
        if (largest !== idx) {
            [a[largest], a[idx]] = [a[idx], a[largest]];
            idx = largest;
        } else break;
    }
    return a;
}

```

```
})();
```

```
let uid = 300;
```

```
export const HeapViz: React.FC = () => {
  const [heap, setHeap] = useState<Patient[]>([]);
  const [customName, setCustomName] = useState('');
  const [customPriority, setCustomPriority] = useState(0);
  const [log, setLog] = useState<string[]>([]);
  const [shakeEmpty, setShake] = useState(false);
  const [busy, setBusy] = useState(false);
  const [healingPatient, setHealing] = useState<Patient>({});

  const addLog = (msg: string) => setLog(prev => [...prev, msg]);

  // --- INSERT: Новый пациент поступает в приемный покой
  const insertPatient = useCallback((name: string, priority: number) => {
    if (busy || heap.length >= 15) {
      addLog('⚠ Все палаты переполнены! Дождитесь выписки');
      setShake(true);
      setTimeout(() => setShake(false), 400);
      return;
    }
    setBusy(true);

    const preset = PATIENT_PRESETS[Math.floor(Math.random() * PATIENT_PRESETS.length)];
    const finalName = name || preset.name.split(' ')[0];
    const finalEmoji = preset.emoji;
    const finalSymptom = preset.symptom;

    const newPatient: Patient = {
      id: `p${uid++}`,
      name: finalName,
      emoji: finalEmoji,
      symptom: finalSymptom,
      priority,
      animState: 'in',
    };
  }, [uid, busy, heap, setShake, setLog]);
```

```

const topPatient = heap[0];
setHealing(topPatient);

addLog(` СРОЧНО! Забираем самого тяжелого: ${topPatient.name}`);

// Step 1: Root lights up as winner
setHeap(prev => prev.map((x, i) => i === 0 ? { ...x, animStat: 'winning' } : x));

setTimeout(() => {
  // Step 2: Root exits the tree, tree reorganizes
  setHeap(prev => prev.map((x, i) => i === 0 ? { ...x, animStat: 'leaving' } : x));

  setTimeout(() => {
    // Run binary heap extraction
    setHeap(prev => {
      if (prev.length === 0) return [];
      const a = prev.map(x => ({ ...x }));
      a[0] = { ...a[a.length - 1] };
      a.pop();
      return siftDown(a).map(x => ({ ...x, animStat: 'sifting' }));
    });

    // Patient gets healed in the bed
    setTimeout(() => {
      setHealing(prev => prev ? { ...prev, animStat: 'healing' } : null);
      addLog(` Успех! Пациент ${topPatient.name} излечен!`);
      setTimeout(() => {
        setHealing(null);
        setBusy(false);
      }, 600);
    }, 800);

    }, 350);
  }, 650);
}, [busy, heap]);

const reset = () => {
  setHeap(INITIAL PATIENTS.map(x => ({ ...x, animStat: 'idle' })));
}

```

```

<div className="text-3xl"> </div>
<div>
  <h3 className="font-black text-emerald-400 text-center">
    Priority Queue</h3>
  <p className="text-xs text-slate-300 mt-1 leading-relaxed">
    В реанимации не важно, кто пришел раньше, а кто в
    остояние</b> (максимальный приоритет).
    Куча (Heap) автоматически перестраивает пац
    оказывался на самом верху дерева (в корне).
  </p>
</div>
</div>

{/* УПРАВЛЕНИЕ */}
<div className="flex flex-col lg:flex-row items-stretch gap-4">

  {/* Добавить случайного */}
  <button
    onClick={handleRandomInsert}
    disabled={busy || heap.length >= 15}
    className="flex-1 min-w-[150px] bg-gradient-to-r from-slate-400 to-slate-500
      opacity-40 disabled:cursor-not-allowed text-white font-medium
      rounded-lg shadow-sm transition-all cursor-pointer focus:outline-none"
  >
    <span> Привезти по Скорой (INSERT)</span>
  </button>

  {/* Добавить кастомного */}
  <form onSubmit={handleCustomInsert} className="flex-1">
    <input
      type="text"
      required
      value={customName}
      onChange={e => setCustomName(e.target.value)}
      placeholder="Имя пациента"
      className="flex-1 bg-slate-800 border border-emerald-500 transition-colors placeholder:text-slate-400"
    />
  </form>
</div>

```

```

    </button>
  </div>

  {/* ИНТЕРАКТИВНОЕ ОТДЕЛЕНИЕ */}
  <div className="grid grid-cols-1 lg:grid-cols-12" >

    {/* ЛЕВАЯ КОЛОНКА: ДЕРЕВО ПАЦИЕНТОВ */}
    <div className={`lg:col-span-8 bg-slate-900 rounded-[380px] overflow-hidden ${shakeEmpty ? 'animate-shake' : ''}` >
      <div className="absolute top-4 left-4 text-[18px] font-medium" >
        Сортировка по тяжести (Двоичная куча / Max-Heap)
      </div>

      <div className="relative w-full h-full min-h-[400px]" >
        <svg className="absolute inset-0 w-full h-full" >
          {edges}
        </svg>

        {heap.map(({patient, idx}) => {
          const pos = nodePos(idx);
          const isRoot = idx === 0;
          return (
            <div
              key={patient.id}
              className={`
                absolute flex flex-col items-center
                -translate-x-1/2 -translate-y-1/2 transition-all
                ${patient.animState === 'in' ? 'animate-in' : ''}
                ${patient.animState === 'winner' ? 'animate-winner' : ''}
                ${patient.animState === 'out' ? 'animate-out' : ''}
                ${isRoot ? 'bg-rose-950/80 border-rose-500' : ''}
              `
              style={{
                left: `${pos.x}%`,
                top: `${pos.y}%`,
                width: isRoot ? 60 : 52,
                height: isRoot ? 60 : 52,
              }}
            </div>
          )
        })}
      </div>
    </div>
  </div>

```

```

        <span>Пациент: {healingPatient.name}
      </h4>
      <p className="text-emerald-400 text-xl">
        Состояние: Стабилизация ({healingPatient.status})
      </p>
      <div className="flex justify-center gap-4">
        <span className="w-2 h-2 rounded-full bg-emerald-400">
          <Heart className="w-4 h-4 text-rose-500" />
        </span>
      </div>
    </div>
  </div>
) : (
  <div className="text-center text-slate-600">
    <span className="text-5xl mb-2 block">Ждём вас</span>
    <p className="text-xs font-bold font-mono">Ваш врач</p>
    <p className="text-[10px] mt-1">Ждём вас</p>
  </div>
)}
</div>

<div className="w-full h-3 bg-gradient-to-r from-emerald-400 to-emerald-500">
</div>

</div>

{/* ЛОГ ДЕЙСТВИЙ */}
<div className="bg-slate-900 border border-slate-800 p-4">
  <div className="text-[10px] font-mono text-slate-400">
    {log.map((entry, idx) => (
      <div
        key={idx}
        className={`text-xs font-mono flex items-center justify-between p-2`}
      >
        {idx === 0 ? <span className="text-emerald-400 font-weight-bold">Действие</span> : null}
        <span>{entry}</span>
      </div>
    ))}
  </div>
</div>

```

```

export const Svg: React.FC<{ children: React.ReactNode;
  <div className="w-full">
    <svg viewBox={`0 0 ${W} ${H}`} className="w-full h-
      {children}
    </svg>
    {caption && <div className="text-[10px] text-slate-
  </div>
};

```

```

export const Node: React.FC<{
  x: number;
  y: number;
  r?: number;
  fill: string;
  label?: string | number;
  stroke?: string;
}> = ({ x, y, r = 13, fill, label, stroke }) => (
  <g style={{ transition: "all .35s ease" }}>
    <circle cx={x} cy={y} r={r} fill={fill} stroke={stroke}
      {label !== undefined && (
        <text x={x} y={y} textAnchor="middle" dominantBaseline="middle">
          {label}
        </text>
      )}
    </g>
);

```

```

export const Edge: React.FC<{
  a: [number, number];
  b: [number, number];
  color?: string;
  width?: number;
  dashed?: boolean;
}> = ({ a, b, color = C.edge, width = 2, dashed }) => (
  <line
    x1={a[0]}
    y1={a[1]}
    x2={b[0]}
    y2={b[1]}
    stroke={color}
    stroke-width={width}
    stroke-dasharray={dashed ? [4, 4]} : []
  >
);

```

```

    <text x={130} y={112} textAnchor="middle" fontSize=
      9
    </text>
    <text x={72} y={54} textAnchor="middle" fontSize=
      3
    </text>
    <text x={188} y={54} textAnchor="middle" fontSize=
      4
    </text>

    <Node x={pos.i[0]} y={pos.i[1]} label="i" fill={C
    <Node x={pos.k[0]} y={pos.k[1]} label="k" fill={v
    <Node x={pos.j[0]} y={pos.j[1]} label="j" fill={r
  </Svg>
);
};

```

/** Компоненты связности: несколько «островов», которые

```

export const ComponentsDemo: React.FC = () => {
  const comps: [string, string][][] = [
    [ "A", "B"], [ "B", "C"],
    [ "D", "E"],
    [ "F", "G"], [ "G", "H"], [ "F", "H"],
  ];
  const pos: Record<string, [number, number]> = {
    A: [26, 40], B: [62, 92], C: [26, 92],
    D: [120, 40], E: [120, 92],
    F: [190, 34], G: [232, 78], H: [178, 96],
  };
  const colors = [C.active, C.warn, C.done];
  const step = useLoop(comps.length + 1, 900);
  const compOf = (v: string) => comps.findIndex((e) =>

```

```

  return (
    <Svg caption={step === 0 ? "Граф распался на острова"
      {comps.flatMap((edges, ci) =>
        edges.map(([u, v], i) => {
          <Edge key={` ${ci} - ${i}`} a={pos[u]} b={pos[v]}

```

```

        fill={C.warn}
      >
        {w}
      </text>
    )}
  </g>
);
}}
{Object.entries(pos).map(([k, [x, y]]) => {
  <Node key={k} x={x} y={y} label={k} fill={C.idle}
  >>
})}
</Svg>
);
};

```

/** Сжатие координат: разрежённые числа → плотные индексы

```

export const CoordCompressDemo: React.FC = () => {
  const raw = [1000, 5, 74000, 5, 300];
  const sorted = [5, 300, 1000, 74000];
  const step = useLoop(3, 1200);
  const boxW = 46;
  const startX = (260 - raw.length * (boxW + 4)) / 2;

  return (
    <Svg
      caption={
        step === 0
          ? "Исходные координаты – огромные и с дырами"
          : step === 1
            ? "Сортируем уникальные значения"
            : "Каждое число заменяем его номером → плотные индексы"
      }
    >
      {raw.map((v, i) => {
        <g key={i}>
          <rect x={startX + i * (boxW + 4)} y={20} width={boxW} height={20} />
          <text x={startX + i * (boxW + 4) + boxW / 2} y={30}>
            {v}
          </text>
        </g>
      })}
    </Svg>
  );
};

```

```

const color = (id: string) =>
  id === "x" ? "#6366f1" : id === "p" ? "#0ea5e9" : id

return (
  <Svg caption={captions[step]}>
    {f.edges.map(([a, b], i) => (
      <line
        key={i}
        x1={f.pos[a][0]}
        y1={f.pos[a][1]}
        x2={f.pos[b][0]}
        y2={f.pos[b][1]}
        stroke={C.edge}
        strokeWidth={1.8}
        style={{ transition: "all .45s ease" }}
      />
    ))}
    {Object.entries(f.pos).map(([id, [x, y]]) => (
      <g key={id} style={{ transform: `translate(${x}, ${y})` }}>
        <circle r={13} fill={color(id)} stroke={C.idl} />
        <text textAnchor="middle" dominantBaseline="bottom">
          {id}
        </text>
      </g>
    ))}
  </Svg>
);
};

```

```

/** Zig – один поворот, когда родитель уже корень. */
export const ZigDemo: React.FC = () => (
  <SplayFrames
    captions={["p – корень, x под ним", "Один поворот: "]}
    frames={[
      { pos: { p: [130, 32], x: [78, 92] }, edges: [{"p": "x"}] },
      { pos: { x: [130, 32], p: [182, 92] }, edges: [{"x": "p"}] }
    ]}
  />

```

```

const GRAPH_EDGES: [string, string][] = [
  ["A", "B"], ["A", "C"], ["B", "D"], ["C", "D"], ["D", "E"],
];

export const BfsDemo: React.FC = () => {
  const order = ["A", "B", "C", "D", "E", "F"];
  const step = useLoop(order.length + 1, 850);
  const visited = new Set(order.slice(0, step).flat());
  const wave = step > 0 && step <= order.length ? order[step - 1] : null;
  return (
    <Svg caption={`Уровень ${Math.max(0, Math.min(step, order.length))}`}
      {GRAPH_EDGES.map(([u, v], i) => {
        <Edge key={i} a={GRAPH_POS[u]} b={GRAPH_POS[v]}
        />
      })}
      {Object.entries(GRAPH_POS).map(([k, [x, y]]) => {
        <Node key={k} x={x} y={y} label={k}
          fill={wave?.includes(k) ? C.active : visited.has(k) ? C.visited : C.inactive}
          stroke={wave?.includes(k) ? "#a5b4fc" : undefined}
        />
      })}
    />
  );
};

```

```

export const DfsDemo: React.FC = () => {
  const path = ["A", "B", "D", "E", "F", "C"];
  const step = useLoop(path.length + 1, 750);
  const visited = path.slice(0, step);
  const head = visited[visited.length - 1];
  return (
    <Svg caption={head ? `Идём вглубь: ${visited.join(" ")}` : ""}
      {GRAPH_EDGES.map(([u, v], i) => {
        const iu = visited.indexOf(u);
        const iv = visited.indexOf(v);
        const onPath = iu >= 0 && iv >= 0 && Math.abs(iu - iv) <= 1;
        return <Edge key={i} a={GRAPH_POS[u]} b={GRAPH_POS[v]}
          fill={onPath ? C.onPath : visited.has(u) || visited.has(v) ? C.visited : C.inactive}
        />
      })}
      {Object.entries(GRAPH_POS).map(([k, [x, y]]) => {
        <Node key={k} x={x} y={y} label={k} fill={k === head ? C.head : visited.has(k) ? C.visited : C.inactive}
        />
      })}
    />
  );
};

```

```

    });
};

export const RelaxDemo: React.FC = () => {
  const step = useLoop(3, 1100);
  const dV = [12, 12, 7][step];
  const improved = step === 2;
  return (
    <Svg caption={improved ? "12 > 3 + 4 → пишем 7. Путь" : ""}>
      <Edge a={[60, 70]} b={[200, 70]} color={improved ? "red" : "black"} />
      <text x={130} y={54} textAnchor="middle" fontSize=14 />
      <Node x={60} y={70} r={20} fill={C.active} label="3" />
      <Node x={200} y={70} r={20} fill={improved ? C.done : C.active} label="4" />
      <text x={60} y={106} textAnchor="middle" fontSize=14 />
      <text x={200} y={106} textAnchor="middle" fontSize=14 />
      <text x={130} y={22} textAnchor="middle" fontSize=14 />
    </Svg>
  );
};

export const BridgeDemo: React.FC = () => {
  const step = useLoop(2, 1300);
  const pos: Record<string, [number, number]> = {
    A: [40, 40], B: [40, 95], C: [95, 68], D: [170, 68]
  };
  const edges: [string, string][] = [
    ["A", "B"], ["A", "C"], ["B", "C"], ["C", "D"], ["D", "A"]
  ];
  const cut = step === 1;
  return (
    <Svg caption={cut ? "Убрали C-D → граф развалился на 2 компонента" : ""}>
      {edges.map(([u, v], i) => {
        const isBridge = u === "C" && v === "D";
        if (isBridge && cut) return null;
        return <Edge key={i} a={pos[u]} b={pos[v]} color={isBridge && cut ? "red" : "black"} />
      })}
      {Object.entries(pos).map(([k, [x, y]]) => (
        <Node key={k} x={x} y={y} label={k} fill={cut ? C.done : C.active} />
      ))}
    </Svg>
  );
};

```

```

return (
  <Svg caption={`Жадно берём самые лёгкие рёбра без ц
    {edges.map((e, i) => {
      const on = taken.has(`${e.u}-${e.v}`);
      return (
        <g key={i}>
          <Edge a={pos[e.u]} b={pos[e.v]} color={on ?
          <text x={({pos[e.u][0] + pos[e.v][0]} / 2} y=
            textAnchor="middle" fontSize={9} fontWeigh
          </g>
        );
      });
    })}
    {Object.entries(pos).map(([k, [x, y]]) => (<Node
  </Svg>
);
};

export const SccDemo: React.FC = () => {
  const pos: Record<string, [number, number]> = {
    A: [45, 35], B: [110, 35], C: [77, 95], D: [185, 45]
  };
  const edges: [string, string][] = [["A", "B"], ["B",
  const step = useLoop(2, 1400);
  const scc = ["A", "B", "C"];
  const inScc = (u: string, v: string) => step === 1 &&
  return (
    <Svg caption={step === 1 ? "{A,B,C} – цикл: из кажд
      {edges.map(([u, v], i) => {
        <Edge key={i} a={pos[u]} b={pos[v]} color={inSc
      })}
      {Object.entries(pos).map(([k, [x, y]]) => {
        <Node key={k} x={x} y={y} label={k} fill={step
      })}
    </Svg>
  );
};

```

```

        </g>
      )})
      <text x={130} y={16} textAnchor="middle" fontSize=
    </Svg>
  );
};

export const QueueDemo: React.FC = () => {
  const states = [["A", "B"], ["A", "B", "C"], ["B", "C"], ["C"]];
  const step = useLoop(states.length, 800);
  const items = states[step];
  return (
    <Svg caption="pop_front слева, push_back справа – к
      <defs>
        <marker id="mdArrow" markerWidth="6" markerHeight="6"
          <path d="M0,0 L6,3 L0,6 Z" fill={C.done} />
        </marker>
      </defs>
      <text x={12} y={44} fontSize={9} fill={C.dim}>ВЫХОД</text>
      <text x={206} y={44} fontSize={9} fill={C.dim}>ВХОД</text>
      {items.map((v, i) => (
        <g key={v} style={{ transition: "all .3s" }}>
          <rect x={30 + i * 54} y={56} width={46} height={20} fill={C.dim} />
          <text x={53 + i * 54} y={71} textAnchor="middle" />
        </g>
      ))}
      <line x1={24} y1={71} x2={10} y2={71} stroke={C.dim} />
    </Svg>
  );
};
};

export const DsuDemo: React.FC = () => {
  const parents = [[0, 1, 2, 3], [0, 0, 2, 3], [0, 0, 2, 3], [0, 0, 2, 3]];
  const step = useLoop(parents.length, 950);
  const p = parents[step];
  const pos: [number, number][] = [[50, 95], [105, 95], [160, 95], [215, 95]];
  const rootColor = ["#6366f1", "#10b981", "#f59e0b", "#f59e0b"];

```



```

    {a.map((v, i) => (
      <g key={i}>
        <rect x={20 + i * 39} y={24} width={34} height={12}
          fill={show && i >= l && i <= r ? C.active : C.dim} />
        <text x={37 + i * 39} y={36} textAnchor="middle">{v}</text>
      </g>
    ))}
    <text x={4} y={72} fontSize={8} fill={C.dim}>P</text>
    {pref.map((v, i) => (
      <g key={i}>
        <rect x={20 + i * 33} y={78} width={29} height={12}
          fill={show && i === l ? C.bad : show && i === r ? C.active : C.dim}
          stroke={C.idleStroke} style={{ transition: "stroke 0.2s" }} />
        <text x={34 + i * 33} y={90} textAnchor="middle">{v}</text>
      </g>
    ))}
  </Svg>
);
};

```

```

export const SparseTableDemo: React.FC = () => {
  const step = useLoop(3, 1000);
  const len = [1, 2, 4][step];
  const count = 8 - len + 1;
  return (
    <Svg caption={`Уровень j=${step}: готовые ответы для j`}>
      {Array.from({ length: 8 }).map((_, i) => (
        <g key={i}>
          <rect x={12 + i * 30} y={18} width={26} height={12}
            fill={C.dim} />
          <text x={25 + i * 30} y={29} textAnchor="middle">{i}</text>
        </g>
      ))}
      {Array.from({ length: count }).map((_, i) => (
        <rect key={i} x={12 + i * 30} y={52 + (i % 4) * 12}
          fill={C.active} opacity={i === 0 ? 1 : 0.4} stroke={C.dim} />
      ))}
    </Svg>
  );
};

```

```

const s = "abacaba";
const pi = [0, 0, 1, 0, 1, 2, 3];
const step = useLoop(s.length + 1, 700);
const i = Math.max(0, step - 1);
const len = step > 0 ? pi[i] : 0;
return (
  <Svg caption={step > 0 ? `π[${i}] = ${len}: префикс`
    {s.split("").map((ch, idx) => (
      <g key={idx}>
        <rect x={18 + idx * 33} y={26} width={28} height={20}
          fill={step > 0 && (idx < len || (idx > i - 1)) ? C.fill : C.idle}
          style={{ transition: "fill .25s" }} />
        <text x={32 + idx * 33} y={40} textAnchor="middle">{ch}</text>
        <text x={32 + idx * 33} y={70} textAnchor="middle">{idx}</text>
        <text x={32 + idx * 33} y={70} textAnchor="middle">{idx}</text>
        fill={idx <= i && step > 0 ? C.warn : C.idle} />
      </g>
    )}}
    <text x={4} y={40} fontSize={8} fill={C.dim}>s</text>
    <text x={4} y={70} fontSize={8} fill={C.dim}>π</text>
    <text x={130} y={106} textAnchor="middle" fontSize={12}>
      π[i] — длина самого длинного «префикс = суффикс»
    </text>
  </Svg>
);
};

```

```

export const DpDemo: React.FC = () => {
  const rows = 3, cols = 5;
  const step = useLoop(rows * cols + 1, 320);
  return (
    <Svg caption={`Заполняем таблицу подзадач: готово ${step}`}
      {Array.from({ length: rows }).map((_, r) =>
        Array.from({ length: cols }).map((_, c) => {
          const idx = r * cols + c;
          const filled = idx < step;
          return (
            <g key={idx}>

```

```
| "prefix-function" | "dp" | "floyd" | "components" |  
| "zig" | "zig-zig" | "zig-zag";
```

```
const REGISTRY: Record<DemoKind, React.FC> = {  
  bfs: BfsDemo,  
  dfs: DfsDemo,  
  heap: HeapDemo,  
  stack: StackDemo,  
  queue: QueueDemo,  
  dsu: DsuDemo,  
  trie: TrieDemo,  
  "trie-bfs": TrieBfsDemo,  
  "suffix-link": SuffixLinkDemo,  
  toposort: ToposortDemo,  
  relax: RelaxDemo,  
  "prefix-sum": PrefixSumDemo,  
  "sparse-table": SparseTableDemo,  
  "segment-tree": SegmentTreeDemo,  
  bridge: BridgeDemo,  
  articulation: ArticulationDemo,  
  mst: MstDemo,  
  amortized: AmortizedDemo,  
  np: NpDemo,  
  scc: SccDemo,  
  "prefix-function": PrefixFunctionDemo,  
  dp: DpDemo,  
  floyd: FloydDemo,  
  components: ComponentsDemo,  
  graph: GraphBasicsDemo,  
  "coord-compress": CoordCompressDemo,  
  zig: ZigDemo,  
  "zig-zig": ZigZigDemo,  
  "zig-zag": ZigZagDemo,  
};
```

```
export const MiniDemo: React.FC<{ kind: DemoKind }> = (  
  const Cmp = REGISTRY[kind];  
  if (!Cmp) return null;
```

```

const viz = getViz(vizId ?? undefined);
if (!vizId || !viz) return null;
const Component = viz.Component;

return createPortal(
  <div className="fixed inset-0 z-[110] flex items-end" >
    <div className="absolute inset-0 bg-slate-950/80" >
      <div className="relative w-full sm:max-w-5xl max-h-5xl shadow-2xl flex flex-col" >
        <div className="flex items-center gap-3 px-4 py-4" >
          <div className="min-w-0 flex-1" >
            <h3 className="font-bold text-white text-sm" >{viz.hint}
            <p className="text-[11px] text-white" >{viz.description}
          </div>
          <button
            onClick={onClose}
            className="p-2 rounded-lg bg-slate-800 hover:bg-slate-700"
            aria-label="Заккрыть симулятор"
          >
            <X className="w-5 h-5" />
          </button>
        </div>
        <div className="overflow-y-auto overscroll-contain" >
          <Component />
        </div>
      </div>
    </div>,
    document.body
  );
};

```

ФАЙЛ 46/78: src/components/hints/TermPopover.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 10

```

import React, { useEffect, useLayoutEffect, useRef, useImperativeHandle,

```

```

    const below = anchor.rect.top < height + GAP + 8;
    const top = below ? anchor.rect.bottom + GAP : anchor.rect.top - height - GAP;

    setPos({ top: Math.max(8, Math.min(top, vh - height)), left: anchor.left });
  }, [anchor]);

useEffect(() => {
  if (!anchor) return;
  const onKey = (e: KeyboardEvent) => {
    if (e.key === "Escape") onClose();
  };
  window.addEventListener("keydown", onKey);
  window.addEventListener("scroll", onClose, true);
  return () => {
    window.removeEventListener("keydown", onKey);
    window.removeEventListener("scroll", onClose, true);
  };
}, [anchor, onClose]);

if (!anchor) return null;
const entry = GLOSSARY_BY_ID.get(anchor.termId);
if (!entry) return null;
const viz = getViz(entry.simulator);
const simulatorId = entry.simulator;

return createPortal(
  <div
    ref={cardRef}
    onMouseEnter={onCardEnter}
    onMouseLeave={onCardLeave}
    className="fixed z-[100] term-popover"
    style={{
      top: pos ? pos.top : -9999,
      left: pos ? pos.left : -9999,
      width: Math.min(CARD_W, typeof window !== "undefined" ? window.innerWidth : 1000),
      visibility: pos ? "visible" : "hidden",
    }}
  >
    {viz}
  </div>
, document.body);

```

```
        </button>
      })
    </div>
  </div>
</div>,
document.body
);
};
```

ФАЙЛ 47/78: src/components/JohnsonViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import { useState } from 'react';
import { RotateCcw, SkipForward, Undo } from 'lucide-react';

export function JohnsonViz() {
  const [step, setStep] = useState(0);
  const MAX_STEP = 7;

  const nodes = [
    { id: 'S', x: 200, y: 150, label: 'S' },
    { id: 'A', x: 200, y: 50, label: 'A' },
    { id: 'B', x: 320, y: 220, label: 'B' },
    { id: 'C', x: 80, y: 220, label: 'C' },
  ];

  const edges = [
    { id: 'SA', u: 'S', v: 'A', weight: 0, labelDx: 0, labelDy: 0 },
    { id: 'SB', u: 'S', v: 'B', weight: 0, labelDx: 0, labelDy: 0 },
    { id: 'SC', u: 'S', v: 'C', weight: 0, labelDx: 0, labelDy: 0 },
    { id: 'AB', u: 'A', v: 'B', weight: -2, newWeight: 1, labelDx: 0, labelDy: 0 },
    { id: 'BC', u: 'B', v: 'C', weight: 1, newWeight: 2, labelDx: 0, labelDy: 0 },
    { id: 'CA', u: 'C', v: 'A', weight: 2, newWeight: 3, labelDx: 0, labelDy: 0 },
  ];
```

```

    };

    const getMarkerUrl = (e: any) => {
      if (e.u === 'S') {
        if (step === 0 || step === 7) return "";
        if (step === 1) return "url(#arrow-amber)";
        if (step === 2) return "url(#arrow-pink)";
        return "url(#arrow-slate-dim)";
      }
      if (e.id === 'AB' && step === 4) return "url(#a";
      if (e.id === 'AB' && step > 4) return "url(#arr";
      if (e.id === 'BC' && step === 5) return "url(#a";
      if (e.id === 'BC' && step > 5) return "url(#arr";
      if (e.id === 'CA' && step === 6) return "url(#a";
      if (e.id === 'CA' && step > 6) return "url(#arr";

      if (step >= 4) return "url(#arrow-slate-dim)";
      return "url(#arrow-slate)";
    };

    const getEdgeContent = (e: any) => {
      if (e.u === 'S') return e.weight;

      if (step < 4) return e.weight;

      if (e.id === 'AB') {
        if (step === 4) return `${e.weight} + (${0}`;
        return e.newWeight;
      }
      if (e.id === 'BC') {
        if (step < 5) return e.weight;
        if (step === 5) return `${e.weight} + (${-2}`;
        return e.newWeight;
      }
      if (e.id === 'CA') {
        if (step < 6) return e.weight;
        if (step === 6) return `${e.weight} + (${-1}`;
        return e.newWeight;
      }
    };
  
```

ных вершин:

```

        <span className="text-pink-300 ml-1" style="font-size: 1.2em;">
    </div>
);
case 3: return (
    <div>
        <b>Шаг 3: Магическая формула.</b><br>
        Всем ребрам графа назначаем новый вес.<br>
        <div className="text-center font-monospace" style="border: 1px solid black; padding: 5px; margin: 10px 0;">
            Это повышает или понижает вес ребра?
        </div>
    </div>
);
case 4: return (
    <div>
        <b>Шаг 3.1: Пересчет ребра A → B.</b><br>
        Старый вес: <span className="text-green-300" style="font-size: 1.2em;">0</span>
        Вычисляем:  $-2 + 0 - (-2) =$  <span className="text-green-300" style="font-size: 1.2em;">-2</span>
    </div>
);
case 5: return (
    <div>
        <b>Шаг 3.2: Пересчет ребра B → C.</b><br>
        Старый вес: 1. <br/>
        Вычисляем:  $1 + (-2) - (-1) =$  <span className="text-green-300" style="font-size: 1.2em;">-1</span>
    </div>
);
case 6: return (
    <div>
        <b>Шаг 3.3: Пересчет ребра C → A.</b><br>
        Старый вес: 2. <br/>
        Вычисляем:  $2 + (-1) - 0 =$  <span className="text-green-300" style="font-size: 1.2em;">1</span>
    </div>
);
case 7: return (
    <div className="text-emerald-300">
        <b>Готово!</b> Фиктивную вершину S мы добавили в граф.
        Все новые веса ребер в графе стали отрицательными.
        При этом старые кратчайшие пути остались прежними.
    </div>
);

```

```

        </marker>
    </defs>

    { /* Edges */
    {edges.map((e, i) => {
        const u = nodes.find(n => n.id === e.u);
        const v = nodes.find(n => n.id === e.v);

        const isS = e.u === 'S';
        if (isS && (step === 0 || step === 1)) return;

        const midX = (u.x + v.x) / 2;
        const midY = (u.y + v.y) / 2;

        const edgeColorClass = getEdgeColorClass(e);
        const textHigh = isEdgeTextHigh(e);
        const textEmer = isEdgeTextEmer(e);

        return (
            <g key={i}>
                <line
                    x1={u.x} y1={u.y}
                    className={`stroke-${edgeColorClass}`}
                    markerEnd={getMarkerEnd(e)}
                />

                <text
                    x={midX} y={midY}
                    textAnchor="middle"
                    dominantBaseline="middle"
                    className={`text-${edgeColorClass}`}
                    style={{
                        textHigh: textHigh ? 'normal' : 'none',
                        textEmer: textEmer ? 'normal' : 'none',
                        isS: isS ? 'fill:#f00' : ''
                    }}
                />
                {getEdgeContent(e)}
            </text>
        </g>
    )
    }
    }
    </svg>
    </script>

```

```

        <div className="w-full md:w-[320px] flex
          {/* Log Box */}
          <div className="bg-[#0f172a] p-5 rounded
l justify-center leading-relaxed text-sm text-white
            {getDesc()}
          </div>

          {/* Controls */}
          <div className="flex bg-slate-900 border
            <button onClick={() => setStep(1)}
              className="p-3 bg-slate-800 border
-slate-400 transition-colors" title="Сброси
                <RotateCcw strokeWidth={3}
              </button>
            <button onClick={() => setStep(1)}
              className="p-3 bg-slate-800 border
-slate-400 transition-colors" title="Вар на
                <Undo strokeWidth={3} size=
              </button>
            <button onClick={() => setStep(
              className={`flex-1 font-bold
                ${step === MAX_STEP
                  ? 'bg-emerald-600/50'
                  : 'bg-indigo-600 hover:
                {step === MAX_STEP ? "Готово"
                {step !== MAX_STEP && <Skip
              </button>
            </div>

          {/* Progress Dots */}
          <div className="flex justify-center
            {Array.from({length: MAX_STEP +
              <div key={i} className={`w-12 h-12
125' : i < step ? 'bg-indigo-900' : 'bg-slate-800'
              )}}
            </div>
          </div>
        </div>

```

```

const initialEdges: GEdge[] = [
  { u: 0, v: 1 },
  { u: 1, v: 2 },
  { u: 2, v: 0 },
  { u: 2, v: 3 }, // Connection edge
  { u: 3, v: 4 },
  { u: 4, v: 3 },
];

interface AlgoStep {
  phase: "init" | "dfs1" | "transpose" | "dfs2" | "fini";
  desc: string;
  visited: Set<number>;
  currentNode: number | null;
  order: number[];
  transposed: boolean;
  sccMap: Record<number, number>; // node -> sccId
  currentSccId: number;
  activeEdges: GEdge[];
}

export const KosarajuViz: React.FC = () => {
  const [steps, setSteps] = useState<AlgoStep[]>([]);
  const [currentStepIdx, setCurrentStepIdx] = useState(0);
  const [isPlaying, setIsPlaying] = useState(false);

  useEffect(() => {
    // Generate step-by-step Kosaraju steps
    const generatedSteps: AlgoStep[] = [];
    const visited = new Set<number>();
    const order: number[] = [];
    const sccMap: Record<number, number> = {};

    const recordStep = (
      phase: AlgoStep["phase"],
      desc: string,
      currentNode: number | null,
    ) => {
      const step: AlgoStep = {
        phase,
        desc,
        visited,
        currentNode,
        order,
        transposed: false,
        sccMap,
        currentSccId: 0,
        activeEdges: initialEdges,
      };
      generatedSteps.push(step);
      setSteps(generatedSteps);
    };

    // Step 1: Initial state
    recordStep("init", "Initial state", null);

    // Step 2: DFS 1 (Post-order traversal)
    recordStep("dfs1", "DFS 1 (Post-order traversal)", null);

    // Step 3: Transpose the graph
    recordStep("transpose", "Transpose the graph", null);

    // Step 4: DFS 2 (Strongly Connected Components)
    recordStep("dfs2", "DFS 2 (Strongly Connected Components)", null);

    // Step 5: Final result
    recordStep("fini", "Final result", null);
  }, []);
};

```

```

        -1,
    );

    for (const to of adj[u]) {
        if (!visited.has(to)) {
            dfs1(to);
        }
    }

    order.push(u);
    recordStep(
        "dfs1",
        `DFS 1: вершина ${u} полностью обработана. Запи-
        у,
        false,
        -1,
    );
};

// Run DFS1 on all unvisited
for (let i = 0; i < 5; i++) {
    if (!visited.has(i)) {
        dfs1(i);
    }
}

const topoOrder = [...order].reverse();
recordStep(
    "transpose",
    `Первый проход завершен! Стек выхода в топологиче-
    ебра.`,
    null,
    true,
    -1,
);

// Transpose adj
const adjT: number[][] = Array.from({ length: 5 },
```

```

        recordStep(
            "dfs2",
            `DFS 2: вершина ${u} из стека уже покрашена в
            u,
            true,
            sccId,
        );
    }
}

recordStep(
    "finished",
    `Алгоритм успешно завершен! Найдено ${sccId} компонент,
    null,
    true,
    sccId,
);

setSteps(generatedSteps);
setCurrentStepIdx(0);
}, []));

const step = steps[currentStepIdx] || {
    phase: "init",
    desc: "Загрузка...",
    visited: new Set(),
    currentNode: null,
    order: [],
    transposed: false,
    sccMap: {},
    currentSccId: -1,
    activeEdges: initialEdges,
};

useEffect(() => {
    let timer: NodeJS.Timeout;
    if (isPlaying && currentStepIdx < steps.length - 1)
        timer = setInterval(() => {

```

```

    })
    {isPlaying ? "Пауза " : "Старт "}
</button>

<button
  onClick={reset}
  className="flex items-center justify-center
  ors"
>
  <RotateCcw className="w-4 h-4" />
</button>
</div>
</div>

<div className="grid grid-cols-1 lg:grid-cols-12" >
  {/* Playfield SVG */}
  <div className="lg:col-span-8 bg-[#0B1120] relative" >
    <svg className="w-full h-full max-w-[650px]" >
      {/* Markers for directed arrows */}
      <defs>
        <marker
          id="arrow"
          viewBox="0 0 10 10"
          refX="22"
          refY="5"
          markerWidth="6"
          markerHeight="6"
          orient="auto-start-reverse"
        >
          <path d="M 0 1 L 10 5 L 0 9 z" fill="#6" />
        </marker>
        <marker
          id="arrow-active"
          viewBox="0 0 10 10"
          refX="22"
          refY="5"
          markerWidth="6"
          markerHeight="6"

```

```

    } else {
      if {
        step.currentNode === edge.u ||
        step.currentNode === edge.v
      } {
        stroke = "#10b981";
        markerId = "arrow-active";
      }
    }

    // Adjust slightly for bidirectional link
    const isBidi =
      (edge.u === 3 && edge.v === 4) ||
      (edge.u === 4 && edge.v === 3);
    const dy = isBidi ? (edge.u === 3 ? -10 :

    return (
      <line
        key={`edge-${idx}`}
        x1={x1}
        y1={y1 + dy}
        x2={x2}
        y2={y2 + dy}
        stroke={stroke}
        strokeWidth="2.5"
        markerEnd={`url(#${markerId})`}
        className="transition-all duration-300"
      />
    );
  }}}

  {/* Nodes */}
  {initialNodes.map((node) => {
    const isCurrent = step.currentNode === node.id;
    const isVisited =
      step.visited.has(node.id) ||
      (step.phase === "dfs1" && step.currentNode === node.id);
    const sccId = step.sccMap[node.id];

```

```

        fontSize="10px"
        fontWeight="bold"
      >
        SCC #{sccId}
      </text>
    })
  </g>
);
}}
</svg>

```

```

{step.transposed && (
  <div className="absolute top-4 left-4 bg-in
  1 rounded">
    Граф транспонирован (Ребра обратные)
  </div>
)}
</div>

```

```

{/* Logs & Stacks */}
<div className="lg:col-span-4 bg-slate-950 p-5
  00 overflow-y-auto">
  <h4 className="text-xs font-bold text-slate-4
    Порядок обхода и стек
  </h4>

```

```

<div className="bg-slate-900 border border-sl
  <p className="text-xs text-white leading-re
    {step.desc}
  </p>
</div>

```

```

<div className="flex-1 space-y-4 overflow-y-a
  {/* Top-Sort output / Order stack represent
  <div>
    <span className="text-[10px] text-slate-5
      СТЕК ВЫХОДА (DFS 1 ORDER):
    </span>

```

```

        <div className="mt-4 pt-3 border-t border-slate-200">
          <span>
            Шаг {currentStepIdx + 1} из {steps.length}
          </span>
          <div className="flex gap-1.5">
            <button
              disabled={currentStepIdx === 0}
              onClick={() => setCurrentStepIdx((prev) => prev - 1)}
              className="bg-slate-900 border border-slate-700 px-2 py-1 rounded">
              <span>Назад</span>
            </button>
            <button
              disabled={currentStepIdx >= steps.length - 1}
              onClick={() => setCurrentStepIdx((prev) => prev + 1)}
              className="bg-slate-900 border border-slate-700 px-2 py-1 rounded">
              <span>Вперед</span>
            </button>
          </div>
        </div>
      </div>
    </div>
  </div>
);
};

```

ФАЙЛ 49/78: src/components/KruskalSimulator.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 100

```

import React, { useState } from 'react';
import { Play, RotateCcw, SkipForward, HelpCircle, Check } from 'lucide-react';

```

```
-----
  { id: '0-1', u: 0, v: 1, weight: 2, status: 'pending' }
  { id: '3-4', u: 3, v: 4, weight: 3, status: 'pending' }
  { id: '1-2', u: 1, v: 2, weight: 4, status: 'pending' }
  { id: '0-3', u: 0, v: 3, weight: 5, status: 'pending' }
  { id: '1-4', u: 1, v: 4, weight: 6, status: 'pending' }
  { id: '4-5', u: 4, v: 5, weight: 7, status: 'pending' }
  { id: '3-6', u: 3, v: 6, weight: 8, status: 'pending' }
  { id: '2-5', u: 2, v: 5, weight: 9, status: 'pending' }
  { id: '6-7', u: 6, v: 7, weight: 10, status: 'pending' }
  { id: '4-7', u: 4, v: 7, weight: 11, status: 'pending' }
  { id: '7-8', u: 7, v: 8, weight: 12, status: 'pending' }
  { id: '5-8', u: 5, v: 8, weight: 13, status: 'pending' }
];
```

```
export const KruskalSimulator: React.FC = () => {
  const [activeAlgo, setActiveAlgo] = useState<'kruskal'>('kruskal')
  const [vertices, setVertices] = useState<Vertex[]>(initialVertices)
  const [edges, setEdges] = useState<Edge[]>(initialEdges)
  const [stepIndex, setStepIndex] = useState<number>(0)
  const [heapQueue, setHeapQueue] = useState<string[]>([])
  const [activeCheatTab, setActiveCheatTab] = useState<'none'>('none')

  const [logs, setLogs] = useState<StepLog[]>([
    {
      title: 'Введение в раскраску (Краскал)',
      description: '«Краскал – это раскраска?». Представим, что мы хотим начать!',
      type: 'info',
    },
  ])
};
```

```
// --- KRUSKAL STEPS ---
```

```
const kruskalSteps = [
  // Step 1: Inspect A-B (2)
  () => {
    setEdges(prev => prev.map(e => e.id === '0-1' ? {
      id: '0-1', u: 0, v: 1, weight: 2, status: 'pending'
    } : e))
    setLogs(prev => [
      {
        title: 'Шаг 1: Берем самое дешевое ребро A-B (в
```

```

        description: 'Оно соединяет красную вершину В и
        type: 'info'
    }, ...prev]);
},
// Step 6: Include B-C
() => {
    setVertices(prev => prev.map(v => v.id === 2 ? {
    setEdges(prev => prev.map(e => e.id === '1-2' ? {
    setLogs(prev => [{
        title: 'Успех: Ребро В-С в осто́ве',
        description: 'Вершина С перекрашена в красный ц
        type: 'success'
    }, ...prev]));
},
// Step 7: Inspect A-D (5)
() => {
    setEdges(prev => prev.map(e => e.id === '0-3' ? {
    setLogs(prev => [{
        title: 'Шаг 4: Берем ребро А-Д (вес 5)',
        description: 'ВНИМАНИЕ! Ребро соединяет КРАСНУЮ
        ю – мы перекрашиваем всё в один цвет!"',
        type: 'warning'
    }, ...prev]));
},
// Step 8: Include A-D (Merge to purple)
() => {
    setVertices(prev => prev.map(v => v.color === 'bl
    setEdges(prev => prev.map(e => e.status === 'incl
    setLogs(prev => [{
        title: 'Слияние кланов: Ребро А-Д добавлено',
        description: 'Мы объединили красный и синий клан
        type: 'success'
    }, ...prev]));
},
// Step 9: Inspect B-E (6) - Cycle!
() => {
    setEdges(prev => prev.map(e => e.id === '1-4' ? {
    setLogs(prev => [{

```

```

        description: 'Оно соединяет фиолетовую вершину I
        type: 'info'
    }, ...prev]));
},
// Step 14: Include D-G
() => {
    setVertices(prev => prev.map(v => v.id === 6 ? {
    setEdges(prev => prev.map(e => e.id === '3-6' ? {
    setLogs(prev => [{
        title: 'Успех: Ребро D-G в осто́ве',
        description: 'Вершина G окрашена в фиолетовый.',
        type: 'success'
    }, ...prev]));
},
// Step 15: Inspect C-F (9) - Cycle!
() => {
    setEdges(prev => prev.map(e => e.id === '2-5' ? {
    setLogs(prev => [{
        title: 'Шаг 8: Берем ребро C-F (вес 9)',
        description: 'Оно соединяет вершины C и F, кото
        type: 'warning'
    }, ...prev]));
},
// Step 16: Reject C-F
() => {
    setEdges(prev => prev.map(e => e.id === '2-5' ? {
    setLogs(prev => [{
        title: 'Отказ: Цикл обнаружен!',
        description: 'Выбрасываем ребро C-F.',
        type: 'error'
    }, ...prev]));
},
// Step 17: Inspect G-H (10)
() => {
    setEdges(prev => prev.map(e => e.id === '6-7' ? {
    setLogs(prev => [{
        title: 'Шаг 9: Берем ребро G-H (вес 10)',
        description: 'Соединяет фиолетовую G и бесцветн

```

```

    }, ...prev]));
  },
  // Step 22: Include H-I
  () => {
    setVertices(prev => prev.map(v => v.id === 8 ? {
    setEdges(prev => prev.map(e => e.id === '7-8' ? {
    setLogs(prev => [{
      title: ' Успех: MST построено Краскалом!',
      description: 'Все 9 вершин окрашены в единый фи
        + 4 + 5 + 7 + 8 + 10 + 12 = 51.',
      type: 'success'
    }, ...prev]));
  },
];

// --- PRIM STEPS ---
const primSteps = [
  // Step 1: Start at E
  () => {
    setVertices(prev => prev.map(v => v.id === 4 ? {
    setHeapQueue(['D-E (3)', 'B-E (6)', 'E-F (7)', 'E
    setEdges(prev => prev.map(e => ['3-4', '1-4', '4-5
    setLogs(prev => [{
      title: 'Шаг 1: Плесень зарождается в Токио (Вер
      description: 'Мы выбрали вершину E как стартовую
        ритетную очередь (Heap): D-E(3), B-E(6), E-F(
      type: 'info'
    }, ...prev]));
  },
  // Step 2: Pop D-E (3)
  () => {
    setEdges(prev => prev.map(e => e.id === '3-4' ? {
    setLogs(prev => [{
      title: 'Шаг 2: Куча выдает самое дешевое ребро
      description: 'Плесень тянется по ребру D-E к вер
      type: 'info'
    }, ...prev]));
  },
];

```

```

        description: 'Плесень стремительно бежит к вершине',
        type: 'info'
    }, ...prev]);
},
// Step 7: Include A-B, add B's edges
() => {
    setVertices(prev => prev.map(v => v.id === 1 ? {
    setEdges(prev => prev.map(e => e.id === '0-1' ? {
        eap' } : e));
    setHeapQueue(['B-C (4)', 'B-E (6 - цикл)', 'E-F (7)']);
    setLogs(prev => [{
        title: 'Успех: Вершина B захвачена плесенью',
        description: 'В кучу добавлено B-C(4). Ребро B-E(6) добавлено в кучу.',
        type: 'success'
    }, ...prev]);
},
// Step 8: Pop B-C (4)
() => {
    setEdges(prev => prev.map(e => e.id === '1-2' ? {
    setLogs(prev => [{
        title: 'Шаг 5: Куча выдает ребро B-C (вес 4)',
        description: 'Плесень тянется к вершине C.',
        type: 'info'
    }, ...prev]);
},
// Step 9: Include B-C, add C's edges
() => {
    setVertices(prev => prev.map(v => v.id === 2 ? {
    setEdges(prev => prev.map(e => e.id === '1-2' ? {
        eap' } : e));
    setHeapQueue(['B-E (6 - цикл)', 'E-F (7)', 'D-G (9)']);
    setLogs(prev => [{
        title: 'Успех: Вершина C захвачена',
        description: 'Ребро C-F(9) добавлено в кучу.',
        type: 'success'
    }, ...prev]);
},
// Step 10: Pop B-E (6) - Cycle!
```

```

    },
    // Step 14: Pop D-G (8)
    () => {
        setEdges(prev => prev.map(e => e.id === '3-6' ? {
            setLogs(prev => [{
                title: 'Шаг 8: Куча выдает ребро D-G (вес 8)',
                description: 'Плесень расширяется вниз к G.',
                type: 'info'
            }, ...prev]));
    },
    // Step 15: Include D-G, add G's edges
    () => {
        setVertices(prev => prev.map(v => v.id === 6 ? {
            setEdges(prev => prev.map(e => e.id === '3-6' ? {
                eap' } : e));
        setHeapQueue(['C-F (9 - цикл)', 'G-H (10)', 'E-H (11)']);
        setLogs(prev => [{
            title: 'Успех: Вершина G захвачена',
            description: 'В кучу добавлено G-H(10).',
            type: 'success'
        }, ...prev]));
    },
    // Step 16: Pop C-F (9) - Cycle!
    () => {
        setEdges(prev => prev.map(e => e.id === '2-5' ? {
            setLogs(prev => [{
                title: 'Шаг 9: Куча выдает ребро C-F (вес 9)',
                description: 'Вершины C и F уже заражены плесенью',
                type: 'warning'
            }, ...prev]));
    },
    // Step 17: Reject C-F
    () => {
        setEdges(prev => prev.map(e => e.id === '2-5' ? {
            setHeapQueue(['G-H (10)', 'E-H (11)', 'F-I (13)']);
            setLogs(prev => [{
                title: 'Отказ: Игнорируем ребро C-F',
                description: 'Выбрасываем из кучи.',
            }]);
        }
    }
}

```

```

        title: 'Отказ: Игнорируем E-H',
        description: 'Выбрасываем из кучи.',
        type: 'error'
    }, ...prev]));
},
// Step 22: Pop H-I (12)
() => {
    setEdges(prev => prev.map(e => e.id === '7-8' ? {
    setLogs(prev => [{
        title: 'Шаг 12: Куча выдает H-I (вес 12)',
        description: 'Плесень тянется к последней чистой',
        type: 'info'
    }, ...prev]));
},
// Step 23: Include H-I
() => {
    setVertices(prev => prev.map(v => v.id === 8 ? {
    setEdges(prev => prev.map(e => e.id === '7-8' ? {
    setHeapQueue(['F-I (13 - цикл)']);
    setLogs(prev => [{
        title: '    Успех: Плесень захватила весь граф',
        description: 'Все 9 вершин поглощены. Итоговый граф',
        type: 'success'
    }, ...prev]));
},
];

// --- BORUVKA STEPS ---
const boruvkaSteps = [
    // Step 1: Five-Year Plan 1 (Inspecting outgoing)
    () => {
        // Highlight the cheapest outgoing edges for EACH
        // Node 0(A) -> A-B(2)
        // Node 1(B) -> A-B(2)
        // Node 2(C) -> B-C(4)
        // Node 3(D) -> D-E(3)
        // Node 4(E) -> D-E(3)
    }
];

```

```

    },
    // Step 3: Five-Year Plan 2 (Inspecting bridge between two kolkhozes)
    () => {
        // Now both kolkhozes look at all outgoing roads
        // Outgoing roads between {A, B, C} and {D, E, F}
        // A-D (5), B-E (6), C-F (9).
        // Cheapest is A-D (5).
        setEdges(prev => prev.map(e => e.id === '0-3' ? {
            id: '0-3',
            status: 'included',
            weight: 5,
            color: 'blue',
            type: 'edge'
        }, ...prev));
        setLogs(prev => [{
            title: 'Вторая пятилетка: Колхозы выбирают мост',
            description: 'Теперь Красный и Синий колхозы одружились по мосту A-D с весом 5. Остальные мосты B-E(6) и C-F(9) пока не рассмотрены.',
            type: 'warning'
        }, ...prev]);
    },
    // Step 4: Concurrently merge into single Kolkhoz
    () => {
        setVertices(prev => prev.map(v => ({ ...v, color: 'blue' })));
        setEdges(prev => prev.map(e => ({
            id: e.id,
            status: e.status === 'included' || e.id === '0-3' ? 'included' : 'pending',
            weight: e.weight,
            color: e.color,
            type: 'edge'
        })));
        setLogs(prev => [{
            title: 'Объединение завершено: Один гигантский колхоз',
            description: 'Оба колхоза объединились по мосту A-D с весом 5. Остальные мосты B-E(6) и C-F(9) пока не рассмотрены.',
            type: 'success'
        }, ...prev]);
    },
    // Step 5: Check remaining edges & mark as rejected
    () => {
        setEdges(prev => prev.map(e => e.status === 'pending' ? {
            id: e.id,
            status: 'rejected',
            weight: e.weight,
            color: e.color,
            type: 'edge'
        }, ...prev));
        setLogs(prev => [{
            title: 'Успех: MST построен Борувкой за 2 шага',
            description: 'Все лишние ребра (B-E, E-H, C-F, H-I) отброшены. Вес: 51. Благодаря параллельности, мы потратили 2 шага.',
            type: 'success'
        }, ...prev]);
    }
};

```

```

        {
          title: 'Введение в Коллективизацию (Борувка)',
          description: 'Логика «Борувки»: Каждая из 9 д
            чтобы объединиться в колхоз.',
          type: 'info',
        },
      ]));
    }
  };

const getColorClass = (color: string, id: number) =>
  switch (color) {
    case 'red': return 'bg-red-500 border-red-300 tex
    case 'blue': return 'bg-blue-500 border-blue-300
    case 'purple': return 'bg-purple-600 border-purple
    case 'mold': return 'bg-emerald-500 border-emerald
    default:
      if (activeAlgo === 'prim' && id === 4 && stepIn
        return 'bg-slate-700 border-emerald-500 text-
      }
      return 'bg-slate-700 border-slate-500 text-slate
    }
  };

const getEdgeStroke = (status: string, color: string)
  if (status === 'inspecting') return '#f59e0b';
  if (status === 'rejected') return '#ef4444';
  if (status === 'in_heap') return '#0284c7';
  if (status === 'included') {
    if (color === 'red') return '#ef4444';
    if (color === 'blue') return '#3b82f6';
    if (color === 'purple') return '#a855f7';
    if (color === 'mold') return '#10b981';
  }
  return '#475569';
};

return (
```

```

+
    (activeAlgo === 'prim'
      ? 'bg-emerald-600 text-white shadow-md rounded-xl border-2'
      : 'text-slate-400 hover:text-slate-300')
  }
>
  <Layers className="w-3.5 h-3.5" />
  <span>Прима (Билет 19)</span>
</button>
<button
  onClick={() => handleReset('boruvka')}
  className={
    "flex items-center gap-1.5 px-3 py-2
+
    (activeAlgo === 'boruvka'
      ? 'bg-rose-600 text-white shadow-md rounded-xl border-2'
      : 'text-slate-400 hover:text-slate-300')
    }
  >
    <Users className="w-3.5 h-3.5" />
    <span>Борувка (Билет 20)</span>
  </button>
</div>

<button
  onClick={() => handleReset(activeAlgo)}
  className="flex items-center justify-center gap-2 font-semibold text-xs rounded-xl border-2"
  >
    <RotateCcw className="w-3.5 h-3.5" />
    <span>Сбросить</span>
  </button>

<button
  onClick={handleNextStep}
  disabled={stepIndex >= currentSteps.length}
  className={
    "flex items-center justify-center gap-2

```

```

        </>
      ) : activeAlgo === 'prim' ? (
        <>
          <div className="flex items-center gap-4">
            <div className="w-3 h-1 bg-sky-600 rounded-full"></div>
          </div>
          <div className="flex items-center gap-4">
            <div className="w-3 h-3 rounded-full bg-slate-300"></div>
          </div>
        </>
      ) : (
        <>
          <div className="flex items-center gap-4">
            <div className="w-3 h-3 rounded-full bg-slate-300"></div>
          </div>
          <div className="flex items-center gap-4">
            <div className="w-3 h-3 rounded-full bg-slate-300"></div>
          </div>
          <div className="flex items-center gap-4">
            <div className="w-3 h-3 rounded-full bg-slate-300"></div>
          </div>
        </>
      )
    })
  </div>

  <div className="absolute top-4 right-4 bg-slate-300 p-4 shadow-sm">
    {Math.floor((stepIndex + 1) / currentSteps.length) * 100}%
  </div>

  {/* SVG Edges */}
  <svg className="absolute inset-0 w-full h-full pointer-events-none">
    {edges.map(edge => {
      const u = vertices[edge.u];
      const v = vertices[edge.v];
      const strokeColor = getEdgeStroke(edge);
      const isInspecting = edge.status === 'inspecting' ? 'stroke:#000,stroke-width:2px' : 'stroke:#ccc,stroke-width:1px';
    })}
  </svg>

```

```

        <div
            key={vertex.id}
            style={{ top: vertex.y + "%", left: v
            className={"absolute -translate-x-1/2
font-bold text-base border-2 transition-all
id)}}
        >
            <span>{vertex.label}</span>
            {vertex.id === 4 && activeAlgo === 'p
                <span className="text-[8px] block -l
            }}
        </div>
    )))
</div>

{stepIndex >= currentSteps.length && (
    <div className="absolute inset-x-6 bottom
shadow-xl z-20">
        <p className={"font-bold text-base " +
ld-300' : 'text-rose-300'}}>
            MST построено с помощью {activeAlgo
        </p>
        <p className="text-slate-300 text-xs mt
            Общий вес минимального остова: 51. Вс
        </p>
    </div>
    })
</div>

{/* Logs / Heap */}
<div className="bg-slate-950 rounded-2xl bord
    <div className="p-4 bg-slate-900 border-b b
        <h4 className="font-bold text-slate-200 t
            Ход выполнения
        </h4>
    </div>

    {activeAlgo === 'prim' && (

```

```

        {log.type === 'info' && <Play class=
        <h5 className="font-bold text-sm le
        </div>
        <p className="text-xs text-slate-300
        </div>
        )})
        </div>
        </div>
        </div>
    </div>

```

```

    /* Cheat Sheet: Complexity & Code for all 3 algo
    <div className="bg-slate-900/80 border border-sla
        <div className="flex flex-col sm:flex-row items
            <div className="flex items-center gap-2.5">
                <BookOpen className="w-5 h-5 text-indigo-400
                <h4 className="text-xl font-bold text-white
            </div>

```

```

    <div className="flex bg-slate-950 p-1 rounded
        <button
            onClick={() => setActiveCheatTab('kruskal
            className={"px-3 py-1.5 rounded text-xs f
            == 'kruskal' ? "bg-indigo-600 text-white" :
        >

```

Краскал (Билет 18)

</button>

<button

```

            onClick={() => setActiveCheatTab('prim')}
            className={"px-3 py-1.5 rounded text-xs f
            == 'prim' ? "bg-emerald-600 text-white" : "
        >

```

Прима (Билет 19)

</button>

<button

```

            onClick={() => setActiveCheatTab('boruvka
            className={"px-3 py-1.5 rounded text-xs f
            == 'boruvka' ? "bg-rose-600 text-white" : "

```

```
def find(i):
    if parent[i] == i: return i
    parent[i] = find(parent[i])
    return parent[i]
```

```
def union(i, j):
    r_i, r_j = find(i), find(j)
    if r_i != r_j:
        parent[r_i] = r_j
        return True
    return False
```

2. Жадный выбор ребер

edges.sort() # $O(E \log E)$

mst = []

```
for w, u, v in edges:
    if union(u, v):
        mst.append((u, v, w))`}
```

```
</pre>
</div>
</div>
</div>
{activeCheatTab === 'prim' && {
  <div className="grid grid-cols-1 lg:grid-cols-2">
    <div className="lg:col-span-1 space-y-4">
      <div className="bg-slate-950 p-4 rounded-3xl">
        <p className="text-slate-400 text-xs font-mono">
          <Clock className="w-3.5 h-3.5 text-emerald-500"/>
        </p>
        <p className="text-2xl font-black text-white">Prim's Algorithm</p>
        <p className="text-slate-400 text-xs font-mono">
            mt
          </p>
        </div>
      <div className="bg-slate-950 p-4 rounded-3xl">
        <p className="text-slate-400 text-xs font-mono">
          for
            <Award className="w-3.5 h-3.5 text-emerald-500"/>
          </p>
        </div>
      </div>
    </div>
  </div>
}
```

```

    })

    {activeCheatTab === 'boruvka' && (
      <div className="grid grid-cols-1 lg:grid-cols-2" >
        <div className="lg:col-span-1 space-y-4">
          <div className="bg-slate-950 p-4 rounded-3xl">
            <p className="text-slate-400 text-xs font-medium">
              <Clock className="w-3.5 h-3.5 text-rose-500" />
            </p>
            <p className="text-2xl font-black text-white">Время
            <p className="text-slate-400 text-xs mt-1">
              p>
            </div>
            <div className="bg-slate-950 p-4 rounded-3xl">
              <p className="text-slate-400 text-xs font-medium">
                <Award className="w-3.5 h-3.5 text-rose-500" />
              </p>
              <p className="text-2xl font-black text-white">Награда
              <p className="text-slate-400 text-xs mt-1">
            </div>
            <div className="bg-slate-950 p-4 rounded-3xl">
              <p className="text-slate-400 text-xs font-medium">
                ении:</p>
              <p className="text-xs text-slate-300 leading-relaxed">
                , организовав массовое распараллеленное слияние
              </div>
            </div>
            <div className="lg:col-span-2">
              <div className="bg-slate-950 p-4 rounded-3xl">
                <p className="text-slate-400 text-xs font-medium">
                  <Code className="w-3.5 h-3.5 text-rose-500" />
                </p>
                <pre className="text-xs text-emerald-400 font-mono">
                  80 flex-1">
                {`def boruvka(n, edges):
                  parent = list(range(n))
                  mst = []
                  while len(mst) < n - 1:

```

```

    highlight: 'положительными',
    highlightColor: 'text-emerald-400',
    rest: 'весами. Подсунь отрицательное ребро – сломает',
    complexity: 'O((V+E) log V)',
    border: 'border-blue-500/30 hover:border-blue-400/60',
    titleColor: 'text-blue-400',
    badge: 'text-blue-400/70 bg-blue-950/40',
  },
  {
    img: bellmanImg,
    alt: 'Фура по болоту Форд-Беллман',
    title: 'Фура по Болоту (Ф-Б)',
    desc: 'Медленный, тяжёлый – зато тащит',
    highlight: 'отрицательные',
    highlightColor: 'text-rose-400',
    rest: 'веса и детектит отрицательные циклы.',
    complexity: 'O(V · E)',
    border: 'border-rose-500/30 hover:border-rose-400/60',
    titleColor: 'text-rose-400',
    badge: 'text-rose-400/70 bg-rose-950/40',
  },
  {
    img: floydImg,
    alt: 'Все-ко-Все Флойд',
    title: 'Все ко Все (Флойд)',
    desc: 'Матрица + три цикла {',
    highlight: 'k, i, j',
    highlightColor: 'text-emerald-400',
    rest: '}'. Ищет пути от всех ко всем.',
    complexity: 'O(V³)',
    border: 'border-emerald-500/30 hover:border-emerald-400/60',
    titleColor: 'text-emerald-400',
    badge: 'text-emerald-400/70 bg-emerald-950/40',
  },
];

export default function MnemonicCards() {
  return (

```

```

/**
 * Мобильное содержание: липкая панель снизу-сверху + в
 * На десктопе не рендерится (там обычный сайдбар).
 */
export const MobileToc: React.FC<Props> = ({ selectedCh
  const [open, setOpen] = useState(false);

  useEffect(() => {
    document.body.style.overflow = open ? "hidden" : ""
    return () => {
      document.body.style.overflow = "";
    };
  }, [open]);

  useEffect(() => {
    const onKey = (e: KeyboardEvent) => e.key === "Escap
    window.addEventListener("keydown", onKey);
    return () => window.removeEventListener("keydown",
  }, []);

  return (
    <>
      <div className="lg:hidden sticky top-[57px] z-40
        <button
          onClick={() => setOpen(true)}
          className="w-full flex items-center gap-2.5 p
          aria-label="Открыть содержание"
        >
          <span className="p-1.5 rounded-lg bg-indigo-6
            <List className="w-4 h-4" />
          </span>
          <span className="min-w-0 flex-1">
            <span className="block text-[10px] uppercase
              Содержание • {index + 1} из {total}
            </span>
            <span className="block text-[13px] font-bol
          </span>

```

```
import React from 'react';
import { BookOpen, Cpu, Sparkles, FileDown, FileText }

interface NavbarProps {
  activeTab: 'guide' | 'simulator';
  setActiveTab: (tab: 'guide' | 'simulator') => void;
}

export const Navbar: React.FC<NavbarProps> = ({ activeTab }) => {
  return (
    <header className="bg-slate-900 border-b border-slate-800">
      <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8">
        <div className="flex items-center gap-3 w-full">
          <div className="bg-gradient-to-tr from-indigo-500 to-indigo-600 w-6 h-6">
            <Sparkles className="w-6 h-6" />
          </div>
          <div className="min-w-0 flex-1">
            <h1 className="text-xl font-extrabold text-white">
              Универсальное пособие
            </h1>
            <p className="text-xs text-indigo-400 font-medium">
              Подготовка к экзаменам: Алгоритмы, Структуры
            </p>
          </div>
        </div>
      </div>

      <div className="flex items-center w-full lg:w-auto">
        <button
          id="tab-guide-btn"
          onClick={() => setActiveTab('guide')}
          className={`flex-1 lg:flex-none flex items-center justify-center text-white font-medium duration-200 whitespace-nowrap ${
            activeTab === 'guide'
              ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500'
              : 'text-slate-400 hover:text-white'
          }`>
          Гид
        </button>
      </div>
    </header>
  );
}
```

```
        <FileText className="w-4 h-4" /> TXT
      </a>
    </div>
  </div>
</header>
);
};
```

ФАЙЛ 53/78: src/components/PlanarityDemo.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
export function PlanarityDemo() {
  return (
    <div className="space-y-4">
      <h4 className="text-base font-bold text-white">K5
      <p className="text-xs text-slate-400">Просто смотри
      <div className="grid grid-cols-1 md:grid-cols-2 gap-4">
        { /* K5 */ }
        <div className="bg-slate-950 rounded-xl p-4 border border-slate-800">
          <div className="absolute top-2 left-2 bg-slate-900/30 w-auto z-20">K5 – НЕПЛАНАРЕН</div>
          <svg className="w-full h-full absolute inset-0">
            {(() => {
              const pts = [
                {id:0,x:160,y:40},
                {id:1,x:270,y:100},
                {id:2,x:230,y:230},
                {id:3,x:90,y:230},
                {id:4,x:50,y:100}
              ];
              const edges = [
                [0,1],[0,2],[0,3],[0,4],
                [1,2],[1,3],[1,4],
                [2,3],[2,4],
```

```

<div className="absolute bottom-2 left-2 right-2
  z-20">
  V=5, E=10 {'>'} 3V-6 = 9 {'-->'} {'\u274C'}
</div>
</div>

{/* K3,3 */}
<div className="bg-slate-950 rounded-xl p-4 border
  <div className="absolute top-2 left-2 bg-slate-
    00/30 w-auto z-20">K3,3 – НЕПЛАНАРЕН</div>
  <svg className="w-full h-full absolute inset-0
    {(() => {
      const pts = [
        {id:0,x:60,y:60},{id:1,x:160,y:60},{id:2,x:260,y:60},
        {id:3,x:60,y:220},{id:4,x:160,y:220},{id:5,x:260,y:220},
      ];
      const edges = [
        [0,3],[0,4],[0,5],
        [1,3],[1,4],[1,5],
        [2,3],[2,4],[2,5]
      ];
      function findPt(id:number) { return pts.find(p => p.id === id); }
      function intersect(a:number,b:number,c:number,d:number) {
        if (a===c||a===d||b===c||b===d) return true;
        const p1=findPt(a),p2=findPt(b),p3=findPt(c),p4=findPt(d);
        if (!p1||!p2||!p3||!p4) return false;
        function ccw(ax:number,ay:number,bx:number,by:number,cx:number,cy:number) {
          return (cy-ay)*(bx-ax)>(by-ay)*(cx-ax);
        }
        return ccw(p1.x,p1.y,p3.x,p3.y,p4.x,p4.y)<
          && ccw(p1.x,p1.y,p2.x,p2.y,p3.x,p3.y);
      }
      const crosses:number[]=[];
      for(let i=0;i<edges.length;i++) for(let j=i+1;j<edges.length;j++)
        if (intersect(edges[i][0],edges[i][1],edges[j][0],edges[j][1]))
          if (!crosses.includes(i)) crosses.push(i);
          if (!crosses.includes(j)) crosses.push(j);
    })}
  </svg>
</div>

```

```
import React, { useState, useCallback } from 'react';
import { Sparkles } from 'lucide-react';
```

```
interface Customer {
  id: string;
  name: string;
  emoji: string;
  color: string;
  bubbleText: string;
  animState: 'in' | 'idle' | 'eating' | 'out';
}
```

```
const PRESETS = [
  { name: 'Студент Вася', emoji: '👨🎓', color: 'from-blue',
    bubbleText: 'матана!' },
  { name: 'Кодер Семён', emoji: '👨💻', color: 'from-violet',
    bubbleText: 'порцию борща!' },
  { name: 'Кот Борис', emoji: '🐈', color: 'from-amber',
    bubbleText: 'миска, пожалуйста.' },
  { name: 'Бабушка Люба', emoji: '👵', color: 'from-rose',
    bubbleText: 'супчик!' },
  { name: 'Инопланетянин', emoji: '👽', color: 'from-emerald',
    bubbleText: 'салатик и фруктовое пюре для нашего НЛО!' },
  { name: 'Бизнесмен', emoji: '👔', color: 'from-slate',
    bubbleText: 'бургер с тартаром.' },
];
```

```
let counter = 3;
```

```
export const QueueViz: React.FC = () => {
  const [queue, setQueue] = useState<Customer[]>([
    { id: 'q1', name: 'Студент Вася', emoji: '👨🎓', color: 'from-blue',
      bubbleText: 'матана!', animState: 'idle' },
    { id: 'q2', name: 'Кодер Семён', emoji: '👨💻', color: 'from-violet',
      bubbleText: 'порцию борща!', animState: 'idle' },
    { id: 'q3', name: 'Кот Борис', emoji: '🐈', color: 'from-amber',
      bubbleText: 'миска, пожалуйста.', animState: 'idle' },
  ])
```

```

    if (isServing) return;
    if (queue.length === 0) {
      setShake(true);
      addLog('⚠ DEQUEUE: В очереди никого нет! Повар ск
      setTimeout(() => setShake(false), 400);
      return;
    }

    setIsServing(true);
    const headGuest = queue[0];

    addLog(`  DEQUEUE: Повар наливает суп первому – ${h

// Step 1: Head guest gets eating animation
setQueue(prev => prev.map((c, idx) => idx === 0 ? {

setTimeout(() => {
  // Step 2: Guest leaves with happy face, queue sh
  setQueue(prev => prev.map((c, idx) => idx === 0 ?

  setServedHistory(prev => [`${headGuest.emoji} ${h

  setTimeout(() => {
    setQueue(prev => prev.slice(1));
    setIsServing(false);
    addLog(`  ${headGuest.name} поел и ушел сытым.
  }, 400);
}, 900);
}, [queue, isServing]));

const reset = () => {
  counter = 3;
  setQueue([
    { id: 'r1', name: 'Студент Вася', emoji: ' ', c
      т голода после матана!', animState: 'in' },
    { id: 'r2', name: 'Кодер Семён', emoji: ' ', c
      апишу ИИ за порцию борща!', animState: 'in' },
    { id: 'r3', name: 'Кот Борис', emoji: ' ', col

```

```

        onClick={dequeue}
        disabled={isServing || queue.length === 0}
        className="flex-1 min-w-[150px] bg-gradient-to-r
          opacity-40 disabled:cursor-not-allowed text
          :scale-95 transition-all cursor-pointer fle
      >
      <span> Налить суп первому (DEQUEUE)</span>
    </button>

    <button
      onClick={reset}
      className="px-5 py-3.5 rounded-2xl bg-slate-800
        r-pointer border border-slate-700"
    >
      Сброс
    </button>
  </div>

  { /* ИНТЕРАКТИВНАЯ КУХНЯ */ }
  <div className="grid grid-cols-1 md:grid-cols-12" >

    { /* ЛЕВАЯ КОЛОНКА: ОЧЕРЕДЬ ЗА СУПОМ */ }
    <div className={`md:col-span-9 bg-slate-900 rounded
      -[300px] overflow-hidden ${shakeEmpty ? 'anim
    <div className="absolute top-4 left-4 text-[1.2em]
      Очередь голодных гостей (Буфер FIFO / BFS)
    </div>

    <div className="flex-1 flex flex-col justify-between
      <div className="flex items-end justify-between

      { /* ПОВАРИХА И КОТЕЛ С СУПОМ */ }
      <div className="flex flex-col items-center
        <div className="relative" >
          <div className="absolute -top-10 left
            <div className="w-1.5 h-6 bg-slate-400
              <div className="w-2 h-4 bg-slate-400
            </div>
          </div>
        </div>
      </div>
    </div>
  </div>

```

```

        >
        { /* Облачко мыслей */ }
        <div className={`
            absolute -top-16 left-1/2 -tr
8 text-center text-slate-200 leading-tight
        ${isHead ? 'opacity-100 border
        `}>
            <div className="font-bold tex
            {customer.bubbleText}
            <div className="absolute -bot
it rotate-45"></div>
        </div>

        { /* Персонаж */ }
        <div className={`
            w-14 h-14 rounded-2xl border-:
relative
            ${customer.color}
            ${isHead ? 'ring-4 ring-amber
        `}>
            <span className="text-3xl">{c

        { /* Индексы HEAD/TAIL */ }
        {(isHead || isTail) && (
            <span className={`
                absolute -bottom-2 text-[
                ${isHead ? 'bg-amber-500
            `}>
                {isHead && isTail ? 'h+t'
            </span>
        )}
        </div>
    </div>
    );
    })
    })
</div>

```

```
      }}
    </div>

    {/* Пол */}
    <div className="w-full h-3 bg-gradient-to-r from-indigo-500 to-purple-500" >
    </div>
  </div>

  {/* ЛОГ ДЕЙСТВИЙ */}
  <div className="bg-slate-900 border border-slate-800 p-4" >
    <div className="text-[10px] font-mono text-slate-400" >
      {log.map(({entry, idx} => {
        <div
          key={idx}
          className={`text-xs font-mono flex items-center justify-between p-2`} >
          {idx === 0 ? <span className="text-indigo-500 font-weight-bold">История действий</span> : null}
          <span>{entry}</span>
        </div>
      )})}
    </div>
  </div>
);
};
```

ФАЙЛ 55/78: src/components/SalmonAutomatonWidget.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 10

```
import React, { useState, useEffect } from 'react';
import { Play, RotateCcw, Plus, Volume2, Info } from 'lucide-react';
```

```
interface TrieNode {
  id: number;
  char: string;
  parent: number;
```

```
    }, [speechText]));
```

```
// Построение начального бора (Trie) без fail-ссылок
const buildInitialTrie = (wordList: string[]) => {
  const newNodes: { [key: number]: TrieNode } = {
    0: { id: 0, char: 'ROOT', parent: 0, children: {} }
  };
  let count = 0;
```

```
  wordList.forEach((word) => {
    let curr = 0;
    for (let i = 0; i < word.length; i++) {
      const c = word[i].toUpperCase();
      if (newNodes[curr].children[c] === undefined) {
        count++;
        newNodes[curr].children[c] = count;
        newNodes[count] = {
          id: count,
          char: c,
          parent: curr,
          children: {},
          fail: 0,
          term_link: 0,
          is_terminal: false,
          words: [],
          depth: newNodes[curr].depth + 1
        };
      }
      curr = newNodes[curr].children[c];
    }
    if (!newNodes[curr].words.includes(word)) {
      newNodes[curr].words.push(word);
      newNodes[curr].is_terminal = true;
    }
  });
});
```

```
let currentX = 0;
const paddingX = 70;
```

```
const handleAddWord = (e: React.FormEvent) => {
  e.preventDefault();
  const cleanWord = newWord.trim().toUpperCase().replace(' ', '');
  if (!cleanWord) return;
  if (words.includes(cleanWord)) {
    setSpeechText(`Слово «${cleanWord}» уже есть в слове`);
    return;
  }
  const updated = [...words, cleanWord];
  setWords(updated);
  setNewWord('');
  buildInitialTrie(updated);
};
```

```
const removeWord = (wordToRemove: string) => {
  if (words.length <= 1) {
    setSpeechText('Оставь хотя бы одно слово, иначе нечего удалять');
    return;
  }
  const updated = words.filter((w) => w !== wordToRemove);
  setWords(updated);
  buildInitialTrie(updated);
};
```

// Запуск BFS

```
const startBFS = () => {
  const rootChildren = Object.values(nodes[0].children);
  const queue = [...rootChildren];
  const updatedNodes = { ...nodes };
  rootChildren.forEach((childId) => {
    updatedNodes[childId].fail = 0;
  });

  setNodes(updatedNodes);
  setBfsQueue(queue);
  setSimulationStep(1);
  setCurrentNode(null);
  setSpeechText('');
};
```

```

        updatedNodes[u].term_link = updatedNodes[failTarget]
    }

    logs += `Для #${u} ('${char}') fail = #${failTarget}
}

if (childrenIds.length === 0) {
    logs += 'У этой вершины нет детей, просто переходим к следующему символу'
}

setNodes(updatedNodes);
setBfsQueue(newQueue);
setCurrentNode(r);
setSpeechText(logs);
};

return (
    <div className="bg-slate-800/90 rounded-2xl p-6 border-1 border-slate-700" >
        {/* Шапка виджета */}
        <div className="flex flex-wrap items-center justify-between" >
            <div className="flex items-center space-x-3" >
                <span className="text-3xl" > </span>
                <div>
                    <h4 className="text-2xl font-bold text-white" >
                        Интерактивный конструктор: Говорящий Эхо-автомат
                    </h4>
                    <p className="text-sm text-slate-400" >
                        Построение дерева и расчет суффиксных ссылок
                    </p>
                </div>
            </div>
            <div className="flex items-center gap-2" >
                <button
                    onClick={() => buildInitialTrie(words)}
                    className="flex items-center gap-1 bg-slate-700 text-white rounded-md px-3 py-2 transition-colors"
                    title="Сбросить автомат"
                >

```

```

        { /* Глаз (с морганием) */ }
        <circle cx="145" cy="85" r="12" fill="#ff
        {isBlinking ? (
            <line x1="133" y1="85" x2="157" y2="85"
        ) : (
            <
            <circle cx="148" cy="85" r="6" fill="
            <circle cx="151" cy="82" r="2" fill="
            </>
        )}

        { /* Рот (анимация разговора) */ }
        {isTalking ? (
            <path d="М 165 105 Q 155 120 165 125 Q
        ) : (
            <path d="М 165 110 Q 155 115 168 118" s
        )}

        { /* Улыбка / жабры */ }
        <path d="М 130 80 Q 125 100 130 120" stro
        <path d="М 120 85 Q 115 100 120 115" stro
    </svg>

    { /* Декоративные пузыри */ }
    <div className="absolute -top-1 right-4 tex
    <div className="absolute top-8 right-1 text
    </div>
    <span className="mt-3 bg-rose-600/30 border b
        Эхо-Лосось Сеня
    </span>
    </div>

    { /* Пузырь с текстом (Баббл) */ }
    <div className="md:col-span-2 bg-slate-800 bord
        tween min-h-[140px]">
        { /* Треугольник баббла */ }
        <div className="absolute -left-3 top-1/2 -tra
            slate-800 border-b-[12px] border-b-transpar

```

```

        <button
          onClick={() => buildInitialTrie(words)}
          className="bg-indigo-600 hover:bg-indigo-500
            transition-transform active:scale-95"
        >
          <RotateCcw className="w-4 h-4" /> Перезагрузить
        </button>
      )}
    </div>
  </div>
</div>

{/* SVG Canvas for Automaton Tree */}
<div className="mb-8 bg-slate-900 border border-slate-800
  <h5 className="text-white font-bold mb-3 flex items-center
    <span> </span> Дерево и Суффиксные (fail) ссылки
  </h5>

  {(() => {
    let maxX = 0;
    let maxY = 0;
    Object.values(nodes).forEach(n => {
      if (n.x && n.x > maxX) maxX = n.x;
      if (n.y && n.y > maxY) maxY = n.y;
    });
    const svgWidth = Math.max(maxX + 80, 400);
    const svgHeight = Math.max(maxY + 60, 200);

    return (
      <svg viewBox={`0 0 ${svgWidth} ${svgHeight}`}
        <defs>
          <marker id="arrowhead" markerWidth="6" markerHeight="6"
            <polygon points="0 0, 6 3, 0 6" fill="white" />
          </marker>
          <marker id="fail-arrow" markerWidth="6" markerHeight="6"
            <polygon points="0 0, 6 3, 0 6" fill="white" />
          </marker>

```

```

        // Wait, all nodes get fail link. Let's
        tep >= 1 and computed it
        // To avoid too many lines, let's draw
        const target = nodes[node.fail];
        if (!target || !node.x || !node.y || !t

        // Simple bezier curve for fail link
        const isCurrent = currentNode === node.

        return (
            <path
                key={`fail-${node.id}`}
                d={`M ${node.x} ${node.y - 10} Q ${
- 16}`}
                fill="none"
                stroke={isCurrent ? '#f43f5e' : 'rgb
                strokeWidth={isCurrent ? 2 : 1.5}
                strokeDasharray="4 4"
                markerEnd="url(#fail-arrow)"
            />
        );
    }}}

```

```

{ /* Draw Nodes */
Object.values(nodes).map(node => {
    if (!node.x || !node.y) return null;
    const isRoot = node.id === 0;
    const isCurrent = currentNode === node.
    const isInQueue = bfsQueue.includes(node

    let fill = '#1e293b';
    let stroke = '#475569';

    if (isCurrent) {
        fill = '#2563eb';
        stroke = '#60a5fa';
    } else if (isInQueue) {
        fill = '#b45309';

```

```

        }}}
      </svg>
    );
  }}()}}
</div>

{/* Управление словарем */}
<div className="grid grid-cols-1 lg:grid-cols-3 gap-4">
  <div className="bg-slate-900/50 p-5 rounded-xl">
    <div>
      <h5 className="text-white font-bold mb-3 flex items-center">
        <span> </span> Словарь (Паттерны)
      </h5>
      <div className="flex flex-wrap gap-2 mb-4">
        {words.map((w) => {
          <span
            key={w}
            className="bg-slate-800 border border-slate-700 padding-2 10px"
            style={{borderRadius: 10px}}
          >
            {w}
            <button
              onClick={() => removeWord(w)}
              className="text-slate-500 hover:text-white"
              title="Удалить"
            >
              ✕
            </button>
          </span>
        )}}
      </div>
    </div>
  </div>

  <form onSubmit={handleAddWord} className="flex items-center gap-2">
    <input
      type="text"
      value={newWord}
      onChange={(e) => setNewWord(e.target.value)}
    />
  </form>
</div>

```

```

</thead>
<tbody className="divide-y divide-slate-800">
  {Object.values(nodes).map((node) => {
    const isCurrent = currentNode === node
    const isInQueue = bfsQueue.includes(node)

    return (
      <tr
        key={node.id}
        className={`transition-colors ${
          isCurrent
            ? 'bg-blue-900/40 font-bold text-white'
            : isInQueue
            ? 'bg-slate-800/80'
            : 'hover:bg-slate-800/40'
        }`}
      >
        <td className="py-2.5 px-3 flex items-center">
          <span
            className={`w-2 h-2 rounded-full ${
              node.id === 0
                ? 'bg-purple-500'
                : isCurrent
                ? 'bg-blue-400 animate-pulse'
                : isInQueue
                ? 'bg-amber-400'
                : 'bg-slate-600'
            }`}
          ></span>
          <span>#{node.id}</span>
          {node.id === 0 && <span className="font-size-0.8em">root</span>}
        </td>
        <td className="py-2.5 px-3">
          <span className="bg-slate-800 block w-10 h-10 rounded flex items-center justify-center text-white font-size-0.8em">
            {node.char}
          </span>
        </td>
        <td className="py-2.5 px-3 text-slate-400 font-size-0.8em">
          {node.value}
        </td>
      >
    )
  })}

```

```

        });
      }
    }
  </tbody>
</table>
</div>
</div>
</div>
</div>
</div>
  );
};

```

ФАЙЛ 56/78: src/components/SegmentTreeVisualizer.tsx
 КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useState, useEffect, useCallback, useMemo } from 'react';
import { Play, Pause, RotateCcw, ChevronLeft, ChevronRight } from 'lucide-react';

```

```

// ===== TYPES =====
interface Step {
  id: number;
  desc: string;
  codeLine: number;
  treeState: Record<number, number>; // node index to value
  highlightNodes: number[]; // current highlighted nodes
  mergeChildren?: [number, number]; // pair being merged
  arrayHighlight?: [number, number]; // [L, R] range in array
  result?: number; // partial result
}

```

```

// ===== PURE LOGIC: build tree =====
function buildTreeFull(arr: number[]): Record<number, number> {
  const n = arr.length;
  const tree: Record<number, number> = {};
  function go(v: number, l: number, r: number) {
    if (l === r) { tree[v] = arr[l]; return; }
    const mid = (l + r) >> 1;
    go(2 * v, l, mid);
    go(2 * v + 1, mid, r);
    tree[v] = tree[2 * v] + tree[2 * v + 1];
  }
  go(1, 0, n - 1);
  return tree;
}

```

```

        return;
    }
    const m = (l + r) >> 1;
    steps.push({ id: id++, desc: `build(${v}, ${l}, ${r})`,
        state: { ...tree }, highlightNodes: [v], arrayHighlight: [l, r] });
    go(2 * v, l, m);
    steps.push({ id: id++, desc: `Вернулись в build(${v}, ${l}, ${r})`,
        highlightNodes: [v], arrayHighlight: [l, r] });
    go(2 * v + 1, m + 1, r);
    tree[v] = (tree[2 * v] ?? 0) + (tree[2 * v + 1] ?? 0);
    steps.push({ id: id++, desc: `Объединяем: tree[${v}]`,
        state: { ...tree }, highlightNodes: [v], mergeChildren: true });
}
go(1, 0, n - 1);
steps.push({ id: id++, desc: `Дерево построено! Корень: ${tree[1]}`,
    Highlight: [0, n - 1] });
return steps;
}

```

// 2. Top-Down Query steps

```

function genQueryTopDownSteps(arr: number[], ql: number, qr: number) {
    const n = arr.length;
    const tree = buildTreeFull(arr);
    const steps: Step[] = [];
    let id = 0;

    function go(v: number, l: number, r: number, ql: number, qr: number) {
        if (ql > r || qr < l) {
            steps.push({ id: id++, desc: `query(${v}, [${l}..${r}]`,
                codeLine: 2, treeState: tree, highlightNodes: [v] });
            return 0;
        }
        if (ql <= l && r <= qr) {
            steps.push({ id: id++, desc: `query(${v}, [${l}..${r}]`,
                value: tree[v], codeLine: 4, treeState: tree, highlightNodes: [v] });
            return tree[v];
        }
        const m = (l + r) >> 1;
    }
}

```

```

        highlights.push(l);
        l++;
    }
    if (r & 1) {
        r--;
        res += tree[r] ?? 0;
        steps.push({ id: id++, desc: `r=${r + 1} нечётный`, treeState: tree, highlightNodes: [r], arrayHighlight: [qL, qR], result: res });
        highlights.push(r);
    }
    l >>= 1;
    r >>= 1;
    if (l < r) {
        steps.push({ id: id++, desc: `Поднимаемся: l = ${l}`, treeState: tree, highlightNodes: [l], arrayHighlight: [qL, qR], result: res });
    }
}
steps.push({ id: id++, desc: `Запрос sum([${qL}..${qR}])`, treeState: tree, highlightNodes: [1], arrayHighlight: [qL, qR], result: res });
return steps;
}

```

// ===== TREE RENDERING HELPER =====

```
interface VNode { v: number; l: number; r: number; val: number; left: VNode; right: VNode; }
```

```
function buildVisualTree(treeState: Record<number, number>) {
    if (l === r) return { v, l, r, val: treeState[v] ?? null, left: null, right: null };
    const m = (l + r) >> 1;
    return { v, l, r, val: treeState[v] ?? null, left: buildVisualTree(treeState, l, m), right: buildVisualTree(treeState, m + 1, r) };
}

```

// ===== CODE SNIPPETS =====

```
const BUILD_CODE = [
    "build(v, l, r) {",
    "    if (l === r) { tree[v] = A[l]; return; }",
    "    // -----",
    "    // -----",

```

```

useEffect(() => {
  if (!playing || idx >= steps.length - 1) { if (idx :
  const t = setTimeout(() => setIdx(i => i + 1), speed
  return () => clearTimeout(t);
}, [playing, idx, steps, speed]);

const step = steps[idx] || steps[0];
if (!step) return null;

const clr = { indigo: { btn: 'bg-indigo-600 hover:bg-
border-indigo-500 bg-indigo-600', childRing: 'ring-
-900' }, emerald: { btn: 'bg-emerald-600 hover:bg-e
30 border-emerald-500 bg-emerald-600', childRing: '
-slate-900' }, amber: { btn: 'bg-amber-600 hover:bg
der-amber-500 bg-amber-600', childRing: 'ring-rose-
[color];

return { step, idx, setIdx, playing, setPlaying, speed
}

// ===== SIMULATION PANEL =====
function SimPanel({ title, icon, steps, code, color, arr
| 'emerald' | 'amber'; arr: number[] }) {
  const p = Player({ steps, color });
  if (!p) return null;
  const { step, idx, setIdx, playing, setPlaying, speed
  const n = arr.length;

  const vTree = useMemo(() => buildVisualTree(step.tree

  const renderNode = useCallback((nd: VNode): React.Re
    const isHigh = step.highlightNodes.includes(nd.v);
    const isChild = step.mergeChildren?.includes(nd.v);
    const has = nd.val !== null;

    let cls = 'bg-slate-800 border-slate-700 text-slate
    if (isHigh) cls = `${clr.ring} text-white ring-2 sc
    else if (isChild) cls = `${clr.childRing} text-ambe

```

```

    <div key={i} className={`flex flex-col item
go' ? 'bg-indigo-950 border-indigo-500' : co
er-500'} -translate-y-0.5` : 'bg-slate-900 l
    <span className="text-[7px] text-slate-500
    <span className="text-[11px] font-bold font
    </div>
  );
  }}}
</div>

{/* Tree */}
<div className="flex-1 p-2 overflow-x-auto bg-sla
  <div className="scale-[0.85] origin-top min-w-m
</div>

{/* Code */}
<pre className="px-3 py-2 text-[9px] md:text-[10px]
  x">
  {code.map((line, i) => {
    const ln = i + 1;
    const active = step.codeLine === ln;
    return (
      <div key={ln} className={`px-2 py-0.5 rounded
2 border-indigo-500 text-indigo-200' : colo
      ' : 'bg-amber-600/20 border-l-2 border-ambe
      <span className="inline-block w-4 text-sl
      </div>
    );
  })}
</pre>

{/* Description + result */}
<div className="px-3 py-2.5 bg-slate-900/60 borde
  px]">
  <span className={`mt-0.5 inline-block w-2 h-2 r
  <span className="leading-relaxed">{step.desc}</
  {step.result !== undefined && (
    <span className={`ml-auto shrink-0 font-mono

```



```

        <input
          type="number"
          min={0}
          max={arr.length - 1}
          value={qL}
          onChange={e => setQL(Math.min(Number(e.target.value),
            className="w-14 bg-slate-900 border border-indigo-500
            focus:border-indigo-500"
        />
      </div>
      <div className="flex items-center gap-1">
        <span className="text-xs text-slate-400">
          <input
            type="number"
            min={0}
            max={arr.length - 1}
            value={qR}
            onChange={e => setQR(Math.min(Number(e.target.value),
              className="w-14 bg-slate-900 border border-indigo-500
              focus:border-indigo-500"
            />
          </div>
        </div>
      </div>
      <div className="text-[10px] text-slate-500">
        Запрос:  $\text{sum}(A[\{qL\}..\{qR\}]) = \{arr.slice(qL, qR+1).reduce((sum, val) => sum + val, 0)}\}$ 
      </div>
    </div>
  </div>
</div>

{/* ---- TABS ---- */}
<div className="flex flex-wrap items-center gap-1">
  <button onClick={() => setView('build')} className={
    `text-slate-400 ${view === 'build' ? 'border-indigo-500 border' : ''}
    -colors ${view === 'build' ? 'border-indigo-500 border' : ''}
    -slate-200'`}>
    <Hammer className="w-3.5 h-3.5" /> Построить
  </button>
  <button onClick={() => setView('queries')} className={

```

```

        icon="↑ "
        steps={queryBUSteps}
        code={QUERY_BU_CODE}
        color="amber"
        arr={arr}
      />
    </div>
  )}

{view === 'theory' && (
  <div className="space-y-5">
    {/* Why it splits this way */}
    <div className="bg-slate-950 p-5 rounded-xl b
      <h4 className="text-base font-bold text-whi
        во разбилось именно так для N={arr.length}?
        <p className="text-sm text-slate-300 leadin
          Каждый узел берёт <code className="bg-sla
          к получает <code className="bg-slate-900 px
          -slate-900 px-1 rounded font-mono text-emerg
          евая часть забирает на 1 больше (округление
          </p>
          <div className="bg-slate-900 p-4 rounded-lg
            <div className="text-slate-400">Корень: L
            <div className="text-slate-300">mid = |(0
            <div className="text-indigo-300">→ Левый:
            <div className="text-emerald-300">→ Правы
            h - 1) >> 1)} эл.)</div>
            <div className="text-slate-500 mt-2">Всег
          } внутренних.</div>
        </div>
      </div>
    </div>

    {/* Comparison cards */}
    <div className="grid grid-cols-1 xl:grid-cols
      <div className="bg-slate-950 p-5 rounded-xl
        <div className="absolute -top-3 left-4 bg
          e border border-emerald-500 shadow-md">
            ↑ Запрос Сверху-вниз (Рекурсия)

```

```
        <strong className="text-white">Итеративны  
        кальности. Но если нужны <strong className=  
        ame="text-white">рекурсивный (сверху-вниз)<  
        </p>  
    </div>  
</div>  
    })  
</div>  
};  
}
```

ФАЙЛ 57/78: src/components/Sidebar.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { useMemo, useState } from "react";  
import { ChevronRight, Compass, Search, Sparkles, X } from "lucide-react";  
import { chapters } from "../data/content";  
import { VIZ_REGISTRY } from "../vizRegistry";
```

```
interface SidebarProps {  
  selectedChapterId: string;  
  setSelectedChapterId: (id: string) => void;  
  /** Мобильный режим: список живёт в выдвижной панели */  
  onNavigate?: () => void;  
}
```

```
export const Sidebar: React.FC<SidebarProps> = ({ selectedChapterId, setSelectedChapterId, onNavigate }) => {  
  const [query, setQuery] = useState("");
```

```
  const groups = useMemo(() => {  
    const q = query.trim().toLowerCase();  
    const filtered = q ? chapters.filter((c) => c.title.toLowerCase().includes(q)) : chapters;  
    const map = new Map<string, typeof chapters>();  
    for (const c of filtered) {  
      const key = c.category?.trim() || "Прочие темы";
```

```

</div>
<div className="text-[10px] font-bold upper"
<div className="space-y-1">
  {items.map((chapter) => {
    const isSelected = chapter.id === selectedChapterId
    const hasViz = Boolean(VIZ_REGISTRY[chapter.id])
    return (
      <button
        key={chapter.id}
        onClick={() => {
          setSelectedChapterId(chapter.id);
          onNavigate?.();
          window.scrollTo({ top: 0, behavior: 'smooth' });
        }}
        className={`w-full text-left px-3 py-2 ${
          isSelected
            ? "bg-indigo-600/15 border-indigo-600/20"
            : "bg-slate-800/40 border-transparent"
        }`}
      >
        {hasViz && (
          <span
            className="w-1.5 h-1.5 rounded-full"
            title="Есть интерактивная визуализация"
          />
        )}
        <span className="font-semibold text-left">
          <ChevronRight
            className={`w-4 h-4 shrink-0 mt-0.5`}
            isSelected ? "text-indigo-400" : "text-slate-400"
          />
        </span>
      </button>
    );
  })}
</div>
</div>
))}
</div>

```

```

    id: string;
    text: string;
    probability: number;
}

export const Simulator: React.FC = () => {
  const [activeTab, setActiveTab] = useState<'stack' |

  // Stack state
  const [stack, setStack] = useState<Plate[]>([
    { id: '1', name: 'Тарелка из-под пасты', icon: ' ' },
    { id: '2', name: 'Тарелка из-под пиццы', icon: ' ' },
    { id: '3', name: 'Блюдец от торта', icon: ' ' },
  ]));
  const [popMessage, setPopMessage] = useState<string |

  // Queue state
  const [queue, setQueue] = useState<Person[]>([
    { id: '1', name: 'Студент Вася', icon: ' ' },
    { id: '2', name: 'Голодный кодер', icon: ' ' },
    { id: '3', name: 'Кот Борис', icon: ' ' },
  ]));
  const [dequeueMessage, setDequeueMessage] = useState<

  // Heap state
  const [heap, setHeap] = useState<Hypothesis[]>([
    { id: '1', text: 'Кандидат: "Привет, я искусственный"', },
    { id: '2', text: 'Кандидат: "Привет, я испек вкусный"', },
    { id: '3', text: 'Кандидат: "Привет, я испанский ин"', },
  ]));
  const [newCustomText, setNewCustomText] = useState('');
  const [newCustomProb, setNewCustomProb] = useState(0.);
  const [heapMessage, setHeapMessage] = useState<string

  // Stack handlers
  const addPlate = () => {
    const presets = [
      { name: 'Сковорода от омлета', icon: ' ' },

```

```

    { name: 'Курьер доставки', icon: ' ' },
    { name: 'Усталый админ', icon: ' ' },
    { name: 'Турист с рюкзаком', icon: ' ' },
  ];
  const randomPreset = presets[Math.floor(Math.random() * presets.length)];
  const newPerson = {
    id: Date.now().toString(),
    name: randomPreset.name,
    icon: randomPreset.icon
  };
  setQueue(prev => [...prev, newPerson]);
  setDequeueMessage(` Встал в конец очереди: ${newPerson.name} `);
};

const servePerson = () => {
  if (queue.length === 0) {
    setDequeueMessage("⚠ Очередь пуста! Суп ждет нового посетителя");
    return;
  }
  const firstPerson = queue[0];
  setQueue(prev => prev.slice(1));
  setDequeueMessage(` Выдан суп первому в очереди (Фамилия: ${firstPerson.name}) `);
};

const resetQueue = () => {
  setQueue([
    { id: '1', name: 'Студент Вася', icon: ' ' },
    { id: '2', name: 'Голодный кодер', icon: ' ' },
    { id: '3', name: 'Кот Борис', icon: ' ' },
  ]);
  setDequeueMessage(" Очередь восстановлена.");
};

// Heap handlers
const addHypothesis = (e: React.FormEvent) => {
  e.preventDefault();
  if (!newCustomText.trim()) return;
  const item: Hypothesis = {

```

```

return (
  <div className="bg-slate-900/90 border border-slate
    <div className="border-b border-slate-800 pb-6 mb
      <h2 className="text-2xl font-bold text-transpar
        p-3">
        <span> Интерактивный симулятор структур данных
      </h2>
      <p className="text-slate-400 text-sm mt-2">
        Проверь работу принципов LIFO, FIFO и Приоритет
      </p>
    </div>

    { /* Tabs */ }
    <div className="grid grid-cols-1 md:grid-cols-3 g
      <button
        onClick={() => setActiveTab('stack')}
        className={`flex items-center justify-between
          activeTab === 'stack'
            ? 'bg-blue-600/20 border-blue-500 text-bl
            : 'bg-slate-800/60 border-slate-700/60 te
          }`}
      >
      <div className="flex items-center gap-3">
        <Layers className={`w-5 h-5 ${activeTab ===
          <div className="text-left">
            <div className="font-bold text-sm text-wh
              <div className="text-xs opacity-80">LIFO
            </div>
          </div>
        </div>
        <span className="text-xs bg-blue-500/20 text-l
          DFS
        </span>
      </button>

      <button
        onClick={() => setActiveTab('queue')}
        className={`flex items-center justify-between
          activeTab === 'queue'

```

```

{activeTab === 'stack' && {
  <div className="space-y-6">
    <div className="bg-slate-800/80 p-5 rounded-x
      tify-between gap-4">
      <div>
        <h3 className="text-lg font-bold text-whi
          <span> Симуляция стопки тарелок</span>
          <span className="text-xs font-mono bg-b
        </h3>
        <p className="text-slate-400 text-xs mt-1
          Новые тарелки ставятся наверх. Мыть мож
        </p>
      </div>
      <div className="flex flex-wrap items-center
        <button
          onClick={addPlate}
          className="flex-1 md:flex-none flex ite
          rounded-xl text-sm font-bold shadow-lg tra
        >
          <Plus className="w-4 h-4" /> Поставить
        </button>
        <button
          onClick={popPlate}
          className="flex-1 md:flex-none flex ite
          border-blue-500/40 px-4 py-2.5 rounded-xl
        >
          Помыть верхнюю
        </button>
        <button
          onClick={resetStack}
          title="Сбросить"
          className="p-2.5 bg-slate-800 hover:bg-
          tion-colors cursor-pointer"
        >
          <RotateCcw className="w-5 h-5" />
        </button>
      </div>
    </div>
  </div>
}

```

```

        >
        <div className="flex items-center"
          <span className="text-2xl">{plate.name}</span>
          <span className={`font-bold text-${plate.name}
            {plate.name}</span>
          </div>
          <span className={`text-xs font-mono
            isTop ? 'bg-blue-500 text-white' : ''}>
            {isTop ? 'Готово к помывке' : ''}
          </span>
        </div>
      );
    }
  </div>
)
<div className="mt-6 text-xs text-slate-500"
  ПОЛОЖИЛ ПОСЛЕДНЕЙ → ПОМЫЛ ПЕРВОЙ (LIFO) •
</div>
</div>
</div>
)}

```

```

{/* QUEUE TAB */}
{activeTab === 'queue' && {
  <div className="space-y-6">
    <div className="bg-slate-800/80 p-5 rounded-x
      tify-between gap-4">
      <div>
        <h3 className="text-lg font-bold text-white">
          <span> Симуляция очереди за супом</span>
          <span className="text-xs font-mono bg-in
        /span>
        </h3>
        <p className="text-slate-400 text-xs mt-1">
          Кто первым пришел, тот первым получил суп
        </p>
      </div>
    </div>
  </div>
}
}

```

```

{queue.length === 0 ? (
  <div className="text-center py-12 text-slate-500">
    <div className="text-5xl mb-3"> </div>
    <p className="text-base font-bold">Очередь пуста
    <p className="text-xs mt-1">Добавьте кого-нибудь
  </div>
) : (
  <div className="w-full flex items-center justify-between">
    <div className="flex flex-col items-center justify-center gap-2">
      300">
        <span className="text-3xl"> </span>
        <span className="text-xs font-bold font-sans">
          Очередь из {queue.length} человек
        </span>
      </div>

      <div className="text-indigo-500 flex items-center gap-2">
        <ArrowRight className="w-6 h-6 animate-pulse">
        </div>

      {queue.map((person, index) => {
        const isFirst = index === 0;
        return (
          <div
            key={person.id}
            className={`flex flex-col items-center justify-center gap-2
              ${isFirst
                ? 'bg-gradient-to-b from-indigo-500 to-white
                  relative'
                : 'bg-slate-900/90 border-slate-500'}
              `}
            >
            {isFirst && (
              <span className="absolute -top-10 left-1/2
                -translate-x-1/2 text-indigo-500 font-sans">
                Первый (Head)
              </span>
            )}
            <span className="text-4xl mb-2">{person.name}</span>
            <span className={`font-bold text-slate-500`}

```

```

        </p>
    </div>
    <div className="flex items-center gap-3 w-full">
        <button
            onClick={popHeap}
            className="flex-1 md:flex-none flex items-center justify-center py-2.5 rounded-xl text-sm font-bold shadow-sm"
        >
            Взять кандидата с макс. вероятностью
        </button>
        <button
            onClick={resetHeap}
            title="Сбросить"
            className="p-2.5 bg-slate-800 hover:bg-slate-700 transition-colors cursor-pointer"
        >
            <RotateCcw className="w-5 h-5" />
        </button>
    </div>
</div>

{/* Add Form */}
<form onSubmit={addHypothesis} className="bg-slate-800 p-12 gap-4 items-end">
    <div className="md:col-span-6">
        <label className="block text-xs font-bold">Название гипотезы:</label>
        <input
            type="text"
            value={newCustomText}
            onChange={e => setNewCustomText(e.target.value)}
            placeholder="например: 'Привет, я лучший!'"
            className="w-full bg-slate-900 border border-emerald-500 transition-colors"
        />
    </div>
    <div className="md:col-span-3">
        <label className="block text-xs font-bold">Вероятность:</label>
        <span className="text-emerald-500">{probability}</span>
    </div>
</form>

```

```

    <p className="text-base font-bold">Куча
    <p className="text-xs mt-1">Добавьте ги
  </div>
) : (
  <div className="w-full max-w-2xl space-y-1"
    <div className="flex items-center justify-between"
      <span>КАНДИДАТ / ГИПОТЕЗА</span>
      <span>ВЕРОЯТНОСТЬ (ПРИОРИТЕТ)</span>
    </div>

    {sortedHeap.map(({item, index}) => {
      const isTop = index === 0;
      const percent = (item.probability * 100).toFixed(
      return (
        <div
          key={item.id}
          className={`flex items-center justify-between
            ${isTop
              ? 'bg-gradient-to-r from-emerald-500 to-emerald-
00/10 scale-[1.01]'}
            : 'bg-slate-900/80 border-slate-800'}
          `}
        >
          <div className="flex items-center justify-between"
            <span className={`flex items-center justify-between
              ${isTop ? 'bg-emerald-500 text-white'}
            `}>
              #{index + 1}
            </span>
            <div>
              <div className={`font-bold text-base
                {item.text}
              </div>
              {isTop && (
                <span className="text-[10px] font-medium
-0.5 rounded inline-block mt-1">
                  Вершина Кучи (Root) • Буд
                </span>

```

```
import React, { useMemo, useState } from "react";
import { BookOpen, Play, Search, X } from "lucide-react";
import { VIZ_REGISTRY } from "../vizRegistry";
import { chapters } from "../data/content";

interface Props {
  /** Открыть тренажёр в модальном окне. */
  onOpen: (vizId: string) => void;
  /** Перейти к теме, к которой привязан тренажёр. */
  onGoChapter: (chapterId: string) => void;
}

/**
 * Каталог всех интерактивных тренажёров.
 *
 * Раньше вкладка «Симулятор» показывала один случайный
 * теперь это витрина: видно всё, что вообще можно поты
 */
export const SimulatorHub: React.FC<Props> = ({ onOpen,
  const [query, setQuery] = useState("");

  const items = useMemo(() => {
    const titleById = new Map(chapters.map((c) => [c.id, c.title]));
    const all = Object.entries(VIZ_REGISTRY).map(([id, entry], i) => ({
      id,
      entry,
      chapterTitle: titleById.get(id),
    }));
    const q = query.trim().toLowerCase();
    if (!q) return all;
    return all.filter(
      (i) =>
        i.entry.title.toLowerCase().includes(q) ||
        (i.entry.hint ?? "").toLowerCase().includes(q) ||
        (i.chapterTitle ?? "").toLowerCase().includes(q)
    );
  }, [query]);
```

```

    >
    <h3 className="font-bold text-white text-sm" >
    {entry.hint && <p className="text-[12px] text-white" >
    {!entry.hint && <div className="flex-1" />}
    </h3>

    <div className="flex items-center gap-2 mt-2" >
      <button
        onClick={() => onOpen(id)}
        className="flex items-center gap-1.5 text-white
        transition-colors"
        >
        <Play className="w-3.5 h-3.5" /> Запустить
      </button>
      {chapterTitle && (
        <button
          onClick={() => onGoChapter(id)}
          title={chapterTitle}
          className="flex items-center gap-1.5 text-white
          0 hover:text-white transition-colors min-w-[100px]"
          >
            <BookOpen className="w-3.5 h-3.5 shrink-0" />
            <span className="truncate max-w-[9rem]" >{chapterTitle}
          </button>
        )}
      </div>
    </div>
  )})}
</div>
</div>
);
};

```

```

frames: [
  {
    title: "До: x – ребёнок корня",
    text: "p стоит в корне, x висит слева. Дедушки не",
    pos: { p: [180, 40], x: [95, 110], C: [265, 110],
    edges: [{"p", "x"}, {"p", "C"}, {"x", "A"}, {"x",
  },
  {
    title: "После: x – корень",
    text: "Правый поворот вокруг p: x поднимается, p
    pos: { x: [160, 40], A: [80, 110], p: [245, 110],
    edges: [{"x", "A"}, {"x", "p"}, {"p", "B"}, {"p",
  },
],
};

/* ----- Zig-Zig -----
const ZIGZIG: Case = {
  key: "zig-zig",
  label: "Zig-Zig",
  when: "x и p – дети с ОДНОЙ стороны (оба слева или об
  rotations: "2 поворота, сначала верхний (g, p)",
  frames: [
    {
      title: "До: прямая линия g → p → x",
      text: "Три узла выстроились в «бамбук». Именно эт
      pos: { g: [230, 35], D: [310, 105], p: [140, 105]
      edges: [{"g", "p"}, {"g", "D"}, {"p", "x"}, {"p",
    },
    {
      title: "Шаг 1: поворот ВЕРХНЕЙ пары (g, p)",
      text: "Сначала крутим деда с родителем – это ключ
      pos: { p: [150, 35], x: [70, 105], C: [195, 105],
      edges: [{"p", "x"}, {"p", "g"}, {"x", "A"}, {"x",
    },
    {
      title: "Шаг 2: поворот пары (p, x) – x в корне",
      text: "Бамбук превратился в куст: глубина всех уз

```

```

-----
  p: { fill: "#0ea5e9", stroke: "#7dd3fc" },
  g: { fill: "#f59e0b", stroke: "#fcd34d" },
  sub: { fill: "#1e293b", stroke: "#475569" },
};

const isSubtree = (id: NodeId) => id === "A" || id ===

const Tree: React.FC<{ frame: Frame }> = ({ frame }) =>
  const { pos, edges } = frame;
  return (
    <svg viewBox={`0 0 ${VB.w} ${VB.h}`} className="w-f
      {edges.map(([a, b], i) => {
        const pa = pos[a];
        const pb = pos[b];
        if (!pa || !pb) return null;
        return (
          <line
            key={`-${a}-${b}-${i}`}
            x1={pa[0]}
            y1={pa[1]}
            x2={pb[0]}
            y2={pb[1]}
            stroke="#475569"
            strokeWidth={2}
            style={{ transition: "all .5s cubic-bezier(
          />
        );
      }}}

    {(Object.keys(pos) as NodeId[]).map((id) => {
      const p = pos[id];
      if (!p) return null;
      const c = COLOR[isSubtree(id) ? "sub" : id];
      const r = isSubtree(id) ? 15 : 19;
      return (
        <g
          key={id}
          style={{ transform: `translate(${p[0]}px, $

```

```

return (
  <div className="w-full bg-slate-950 rounded-2xl border-2 border-indigo-300">
    <div className="flex gap-2 mb-4 overflow-x-auto no-scrollbar">
      {CASES.map((c, i) => (
        <button
          key={c.key}
          onClick={() => setCaseIdx(i)}
          className={`px-3 sm:px-4 py-2 rounded-lg text-white
            ${i === caseIdx ? "bg-indigo-600 text-white" : "bg-slate-900"}
          `}
        >
          {c.label}
        </button>
      ))}
    </div>

    <div className="mb-3 text-xs sm:text-[13px] text-white">
      <span className="font-bold text-indigo-300">Корпус</span>
      <span className="text-slate-500">({active.rotation}°)</span>
    </div>

    <div className="bg-slate-900 rounded-xl border border-indigo-300 p-4">
      <Tree frame={current} />
    </div>

    <div className="mt-3 min-h-[64px]">
      <div className="text-sm font-bold text-white mb-2">
        {frame + 1}/{total}. {current.title}
      </div>
      <p className="text-[13px] text-slate-400 leading-5">
        {current.description}
      </p>
    </div>

    <div className="flex flex-wrap items-center gap-2">
      <button
        onClick={() => setPlaying((p) => !p)}
        className="flex items-center gap-1.5 px-3 py-2 text-white border border-indigo-300 rounded"
      >
        {playing ? "Пауза" : "Воспроизвести"}
      </button>
    </div>
  </div>
)

```

```
        <span className="flex items-center gap-1.5">
          <span className="w-3 h-3 rounded-full" style={
            style
          } />
        </span>
        <span className="flex items-center gap-1.5">
          <span className="w-3 h-3 rounded-full" style={
            style
          } />
        </span>
      </div>
    </div>
  </div>
);
};
```

ФАЙЛ 61/78: src/components/SplayTreeViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { useState } from "react";
```

```
import {
  RotateCcw,
  HelpCircle,
  BookOpen,
} from "lucide-react";
```

```
interface SplayNode {
  key: number;
  left: SplayNode | null;
  right: SplayNode | null;
  x?: number;
  y?: number;
}
```

```
// Simple BST insertion to build initial tree
```

```
const insertBST = (root: SplayNode | null, key: number) => {
  if (!root) return { key, left: null, right: null };
  if (key < root.key) {
    root.left = insertBST(root.left, key);
  }
};
```

```
// Values: 10, 20, 30, 40, 50, 60
const createInitialTree = (): SplayNode => {
  let r: SplayNode | null = null;
  const initialKeys = [40, 20, 50, 10, 30, 60];
  initialKeys.forEach((key) => {
    r = insertBST(r, key);
  });
  return r!;
};

const [tree, setTree] = useState<SplayNode>(createInitialTree);
const [inputValue, setInputValue] = useState<string>('');
const [log, setLog] = useState<string[]>([
  "Дерево инициализировано. Попробуйте кликнуть на узел."
]);
const [highlightedNode, setHighlightedNode] = useState<SplayNode | null>(null);

// Splay operation that tracks steps!
const splayStepByStep = (
  root: SplayNode | null,
  key: number,
): { newRoot: SplayNode | null; steps: string[] } => {
  const steps: string[] = [];
  if (!root) return { newRoot: null, steps: ["Дерево пустое."] };

  // We implement the classic top-down or bottom-up splay.
  // For visualization logs, we do a bottom-up explanation.
  let path: { node: SplayNode; dir: "left" | "right" | null }[] = [];
  let current: SplayNode | null = root;

  // Find the element or the element where search failed.
  let lastNode: SplayNode = root;
  while (current) {
    lastNode = current;
    if (key === current.key) {
      path.push({ node: current, dir: null });
      break;
    } else if (key < current.key) {
      path.push({ node: current, dir: "left" });
      current = current.left;
    } else {
      path.push({ node: current, dir: "right" });
      current = current.right;
    }
  }

  // Now we perform the splay operation using the path.
  // This is a simplified version of the splay operation.
  // In a real implementation, you would need to handle rotations.
  // For visualization, we just move the node to the root.
  if (path.length > 0) {
    const [lastNode, ...path] = path;
    // Move the node to the root.
    // This is a simplified version of the splay operation.
    // In a real implementation, you would need to handle rotations.
    // For visualization, we just move the node to the root.
    // This is a simplified version of the splay operation.
    // In a real implementation, you would need to handle rotations.
    // For visualization, we just move the node to the root.
  }

  return { newRoot: lastNode, steps: steps };
};
```

```

    });
} else if (k > node.left.key) {
    // Zig-Zag (Left-Right)
    node.left.right = splayAction(node.left.right);
    if (node.left.right) {
        node.left = leftRotate(node.left);
        steps.push(`Компонент Zig-Zag: левый поворот`);
    }
}

if (!node.left) return node;
steps.push(
    `Финальный поворот Zig (Правое вращение корня)`);
return rightRotate(node);
} else {
    if (!node.right) return node;

    if (k < node.right.key) {
        // Zag-Zig (Right-Left)
        node.right.left = splayAction(node.right.left);
        if (node.right.left) {
            node.right = rightRotate(node.right);
            steps.push(`Компонент Zig-Zag: правый поворот`);
        }
    } else if (k > node.right.key) {
        // Zag-Zag (Right-Right)
        node.right.right = splayAction(node.right.right);
        node = leftRotate(node);
        steps.push(
            `Вращение Zag-Zag (Левый поворот родителя д...`);
    }
}

if (!node.right) return node;
steps.push(
    `Финальный поворот Zag (Левое вращение корня)`);
};

```

```

    if (isNaN(val)) return;
    handleSplay(val);
    setInputValue("");
};

const resetTree = () => {
    setTree(createInitialTree());
    setHighlightedNode(null);
    setLog(["Дерево возвращено в исходное состояние."]);
};

// Assign coordinate positions using a simple recursion
const computeCoordinates = (
    node: SplayNode | null,
    x: number,
    y: number,
    levelWidth: number,
): void => {
    if (!node) return;
    node.x = x;
    node.y = y;
    if (node.left) {
        computeCoordinates(node.left, x - levelWidth, y + 1);
    }
    if (node.right) {
        computeCoordinates(node.right, x + levelWidth, y + 1);
    }
};

const drawTree = cloneTree(tree);
computeCoordinates(drawTree, 300, 40, 140);

// Render lines and nodes
const renderLines = (node: SplayNode | null): React.ReactNode[] => {
    if (!node) return [];
    let lines: React.ReactNode[] = [];
    if (node.left && node.left.x && node.left.y) {
        lines.push(

```

```

        onClick={() => handleSplay(node.key)}
        data-title={`Кликните, чтобы выполнить Splay(${node.key})`}
      >
        <circle
          cx={node.x}
          cy={node.y}
          r="18"
          fill={isHighlighted ? "#4f46e5" : "#1e293b"}
          stroke={isHighlighted ? "#818cf8" : "#64748b"}
          strokeWidth={isHighlighted ? "3" : "2"}
          className="transition-all duration-300 group-hover:stroke-indigo-400"
        />
        <text
          x={node.x}
          y={node.y}
          dy=".3em"
          textAnchor="middle"
          fill="white"
          fontSize="12px"
          fontWeight="bold"
          className="select-none pointer-events-none"
        >
          {node.key}
        </text>
      </g>,
    );
    circles = circles.concat(renderNodes(node.left));
    circles = circles.concat(renderNodes(node.right));
    return circles;
  };

  return (
    <div className="bg-slate-900 rounded-xl border border-slate-800" >
      {/* Header */}
      <div className="bg-slate-800 p-4 border-b border-slate-700" >
        <h3 className="font-bold text-white flex items-center" >
          <BookOpen className="w-5 h-5 text-indigo-400 mr-2" />
          Интерактивное Splay-дерево
        </h3>
      </div>
    </div>
  );
}

```

```

        onClick={() => {
          e.preventDefault();
          const val = parseInt(inputValue);
          if (!isNaN(val)) {
            let newTree = cloneTree(tree);
            newTree = insertBST(newTree, val);
            const { newRoot, steps } = splayStep(newTree, val);
            if (newRoot) setTree(newRoot);
            setLog([`Вставили [${val}].`, ...steps]);
            setInputValue("");
            setHighlightedNode(val);
          }
        }}
        className="bg-indigo-950 hover:bg-slate-900"
      >
        Вставить
      </button>
    </form>

    <button
      onClick={resetTree}
      className="flex items-center gap-1.5 px-3"
    >
      <RotateCcw className="w-3.5 h-3.5" /> Сбросить
    </button>
  </div>
</div>

{/* Logs */}
<div className="lg:col-span-5 bg-slate-950 border-1
  to overflow-y-auto custom-scrollbar">
  <h4 className="text-xs font-bold text-slate-400">
    Ход вращения (Splay-логи)
  </h4>
  <div className="flex-1 space-y-2 mb-4 overflow-hidden">
    {log.map((step, idx) => (
      <div

```

```

<div>
  <p className="font-bold text-slate-300 mb-1">
    1. Отсутствующий элемент
  </p>
  <p className="text-slate-400 leading-relaxed">
    Допустим, мы ищем{" "}
    <code className="text-indigo-400 font-sans">его нет. Алгоритм спустится к листу, на
    неудачей (например,{" "})
    <code className="text-indigo-400">[20]</code>
    <code className="text-indigo-400">[30]</code>
    поднимет в корень{" "}
    <strong className="text-white">
      последний успешно просмотренный узел
    </strong>
    , на котором он споткнулся! Это оставяет
    сверху.
  </p>
</div>
<div className="mt-3 text-[10px] text-indigo-400">
  Поиск настраивает локальный кэш под реали
</div>
</div>

<div className="bg-slate-900/60 p-4 rounded-l" style="background-color: #000000; color: #ffffff; padding: 10px; border-radius: 10px 10px 0 0;">
  <div>
    <p className="font-bold text-slate-300 mb-1">
      2. Эффект качелей (Swing)
    </p>
    <p className="text-slate-400 leading-relaxed">
      Если агент запрашивает по очереди{" "}
      <code className="text-rose-400">[10]</code>
      <code className="text-rose-400">[60]</code>
      углы), дерево будет превращаться в палку
      сторону. Первая операция Splay(60) стои
      <strong className="text-white">0(n)</strong>
      глубину n. Вторая Splay(10) заставляет
      Постоянный свинг убивает производительн
    
```

```
import React, { useState, useCallback } from 'react';
import { ArrowUp, Sparkles } from 'lucide-react';

interface Plate {
  id: string;
  name: string;
  emoji: string;
  color: string;
  isDirty: boolean;
  animState: 'in' | 'idle' | 'washing' | 'clean';
}

const PLATE_PRESETS = [
  { name: 'Тарелка из-под спагетти', emoji: '🍝', color: 'from-pasta' },
  { name: 'Тарелка из-под пиццы', emoji: '🍕', color: 'from-pizza' },
  { name: 'Блюдец от торта', emoji: '🍰', color: 'from-pie' },
  { name: 'Тарелка от тако', emoji: '🌮', color: 'from-yummy' },
  { name: 'Тарелка из-под суши', emoji: '🍣', color: 'from-sushi' },
  { name: 'Сковорода от глазуньи', emoji: '🍳', color: 'from-pan-fry' },
];

let counter = 0;

export const StackViz: React.FC = () => {
  const [dirtyStack, setDirtyStack] = useState<Plate[]>([
    { id: 'p1', name: 'Тарелка из-под спагетти', emoji: '🍝', state: 'idle' },
    { id: 'p2', name: 'Тарелка из-под пиццы', emoji: '🍕', state: 'idle' },
    { id: 'p3', name: 'Блюдец от торта', emoji: '🍰', state: 'idle' },
  ]));

  const [cleanRack, setCleanRack] = useState<Plate[]>([
    { id: 'c1', name: 'Тарелка из-под спагетти', emoji: '🍝', state: 'idle' },
    { id: 'c2', name: 'Тарелка из-под пиццы', emoji: '🍕', state: 'idle' },
    { id: 'c3', name: 'Блюдец от торта', emoji: '🍰', state: 'idle' },
  ]));

  const [isWashing, setIsWashing] = useState(false);
  const [showSoap, setShowSoap] = useState(false);
```

```

    addLog('△ POP: Нечего мыть! Все тарелки чистые.')
    setTimeout(() => setShake(false), 400);
    return;
  }

  setIsWashing(true);
  const topPlate = dirtyStack[dirtyStack.length - 1];

  addLog(`  POP: Начинаем мыть верхнюю тарелку с ${topPlate.name}`);

  // Step 1: Start washing animation (sponge and soap)
  setDirtyStack(prev => prev.map(p => p.id === topPlate.id ? {
    ...p,
    isDirty: false,
    animState: 'clean',
  }) : p);

  setCleanRack(prev => [cleanPlate, ...prev].slice(0, cleanRack.length));
  setDirtyStack(prev => prev.slice(0, prev.length - 1));
  setShowSoap(false);
  setIsWashing(false);
  addLog(`  Готово! Чистая тарелка ${topPlate.emoji}`);
}, 850);
}, [dirtyStack, isWashing]));

const reset = () => {
  counter = 0;
  setDirtyStack([
    { id: 'r1', name: 'Тарелка из-под спагетти', emoji: '🍝', animState: 'in' },
    { id: 'r2', name: 'Тарелка из-под пиццы', emoji: '🍕', animState: 'in' },
    { id: 'r3', name: 'Блюдец от торта', emoji: '🍰', animState: 'in' },
  ]);
  setCleanRack([]);
  setShake(true);
  setDirtyStack([]);
  setShowSoap(true);
  setIsWashing(true);
  addLog('△ POP: Начали мыть!');
};

```

```

        onClick={popPlate}
        disabled={isWashing || dirtyStack.length === 0}
        className="flex-1 min-w-[150px] bg-gradient-to-r from-slate-800 to-slate-900
        opacity-40 disabled:cursor-not-allowed text-white font-mono text-sm
        border border-slate-700 border-radius-10px padding-4 transition-all cursor-pointer"
      >
        <span> Помыть верхнюю (POP)</span>
      </button>

    <button
      onClick={reset}
      className="px-5 py-3.5 rounded-2xl bg-slate-800 text-white font-mono text-sm
      cursor-pointer border border-slate-700"
    >
      Сброс
    </button>
  </div>

  { /* ИНТЕРАКТИВНАЯ КУХНЯ */ }
  <div className="grid grid-cols-1 md:grid-cols-12 gap-4" >

    { /* ЛЕВАЯ КОЛОНКА: СТОПКА ГРЯЗНЫХ ТАРЕЛОК */ }
    <div className="md:col-span-7 bg-slate-900 rounded-2xl padding-4" >
      <div className="absolute top-4 left-4 text-white font-mono text-sm" >
        Грязная стопка (Стек вызовов / LIFO)
      </div>

      { /* Визуализация раковины */ }
      <div className="absolute top-4 right-4 flex items-center justify-center" >
        <div className="bg-slate-800/80 text-[10px] font-mono" >
          <span className="w-1.5 h-1.5 rounded-full border border-slate-700" >
            <span>РАКОВИНА </span>
          </span>
        </div>
      </div>

      { /* Мыльные пузыри при мытье */ }
      {showSoap && {
        <div className="absolute inset-0 pointer-events-none" >

```

```

        key={plate.id}
        className={`
          w-full max-w-[280px] h-10 rounded
          shadow-md select-none transition-all duration-200
          ${plate.color}
          ${plate.animState === 'in' ? 'animate-in' : ''}
          ${plate.animState === 'washing' ? 'animate-wash' : ''}
          ${isTop ? 'ring-4 ring-blue-500/50' : ''}
        `}
      >
        <span className="text-lg">{plate.name}</span>
        <span className="text-slate-800 text-sm">
          {isTop && (
            <span className="text-[9px] bg-blue-500/50 px-1">
              LIFO
            </span>
          )}
        </div>
      </div>
    );
  }
}
</div>
))

{/* Столешница раковины */}
<div className="w-full h-4 bg-gradient-to-r from-slate-200 to-slate-400">
  </div>

{/* ПРАВАЯ КОЛОНКА: СУШИЛКА ДЛЯ ЧИСТЫХ ТАРЕЛОК */}
<div className="md:col-span-5 bg-slate-900 rounded-2xl p-4">
  <div className="absolute top-4 left-4 text-[10px] font-mono">
    Сушилка для чистой посуды
  </div>

  <div className="absolute top-4 right-4 flex items-center justify-center gap-2">
    <div className="order-emerald-800/80 text-[9px] font-mono">
      <Sparkles className="w-3.5 h-3.5 animate-pulse">
        <span>ЧИСТО </span>
      </div>
    </div>
  </div>
</div>

```

```
        <span>{entry}</span>
      </div>
    )})
  </div>
</div>
);
};
```

ФАЙЛ 63/78: src/components/StringAlgorithmsViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import { useState, useEffect, useRef, useMemo } from "react";
import { Play, Pause, SkipForward, Undo, RefreshCw } from "lucide-react";

function generateKmpSteps(pattern: string, text: string): Step[] {
  if (!pattern) pattern = " ";
  const s = pattern + "#" + text;
  const n = s.length;
  const pi = new Array(n).fill(0);
  const steps: any[] = [];

  steps.push({ i: 0, j: 0, pi: [...pi], desc: `Начало` });

  for (let i = 1; i < n; i++) {
    let j = pi[i - 1];

    steps.push({ i, j, pi: [...pi], highlight: [i, j] });

    while (j > 0 && s[i] !== s[j]) {
      steps.push({ i, j, pi: [...pi], highlight: [i, j], desc: `откат` });
      j = pi[j - 1];
    }
  }
}
```

```

    }

    steps.push({ i, l, r, zTarget: z[i], zSource: i });
    while (i + z[i] < n && s[z[i]] === s[i + z[i]]) {
      steps.push({ i, l, r, zTarget: z[i], zSource: i, desc: `падают! Окно растёт.` });
      z[i]++;
    }

    if (i + z[i] < n) {
      steps.push({ i, l, r, zTarget: z[i], zSource: i, desc: `разные. Стоп.` });
    } else {
      steps.push({ i, l, r, z: [...z], desc: `Упёрся.` });
    }

    steps.push({ i, l, r, z: [...z], desc: `Фиксируем.` });

    if (i + z[i] - 1 > r) {
      l = i;
      r = i + z[i] - 1;
      steps.push({ i, l, r, z: [...z], desc: `Обновили.` });
    }

    if (z[i] === pattern.length) {
      steps.push({ i, l, r, z: [...z], found: true });
    }
  }
  return { s, steps, patternLength: pattern.length };
}

export function StringAlgorithmsViz({ defaultMode = "kmp" }) {
  const [mode, setMode] = useState<"kmp" | "z">(defaultMode);
  const [pattern, setPattern] = useState("aba");
  const [text, setText] = useState("abacaba");

  // Ensure inputs are valid dynamically
  const safePattern = pattern || " ";

```

```

<div className="space-y-4">
  <div className="flex items-center justify-b
    <div className="flex bg-slate-900 borde
      <button onClick={() => setMode("kmp
        className={`px-3 py-1.5 text-xs
e" : "text-slate-400 hover:text-slate-300"}
        КМП (π-ФУНКЦИЯ)
      </button>
      <button onClick={() => setMode("z")
        className={`px-3 py-1.5 text-xs
" : "text-slate-400 hover:text-slate-300"}`
        Z-ФУНКЦИЯ
      </button>
    </div>

    <div className="flex items-center gap-2
      <div className="bg-slate-800 p-1 fl
        <span className="text-[10px] te
        <input value={pattern} onChange
xt-white text-xs font-bold font-mono px-2 p
      </div>
      <div className="bg-slate-800 p-1 fl
        <span className="text-[10px] te
        <input value={text} onChange={e
te text-xs font-bold font-mono px-2 py-1 ou
      </div>
    </div>
  </div>

  <div className="bg-slate-950 rounded-xl bor
px] flex items-center justify-start">
    <div className="flex gap-1.5 px-4 h-ful
      {s.split("").map((char, index) => {
        const isPattern = index < patte
        const isSeparator = index === p

        let borderColor = "border-slate
        let bgColor = "bg-slate-900";

```

```

<div className={`w-8 h-
xtColor} font-mono text-lg transition-color
    {char}
</div>

    {/* Value array box */}
    <div className="text-[1
slate-700/50 font-mono font-bold">
        {mode === "kmp" ? s
    </div>

    {/* KMP fingers */}
    {mode === "kmp" && step
        <div className="abs
z-20">
            <span classNam
        </div>
    })
    {mode === "kmp" && step
        <div className="abs
            <span classNam
        </div>
    })
    {mode === "kmp" && step
        <div className="abs
20">
            <span classNam
        </div>
    })

    {/* Z fingers */}
    {mode === "z" && step.i
        <div className="abs
e z-20">
            <span classNam
        </div>
    })
    {mode === "z" && step.z

```

```

        <SkipForward strokeWidth={3} si
    </button>
    <button onClick={reset} className="
border-slate-800">
        <RefreshCw strokeWidth={3} size
    </button>
</div>

    {/* Description Log */}
    <div className="flex-1 bg-slate-900/50
erflow-hidden">
        <div className="absolute top-0 righ
            <span className="font-mono text
        </div>
        <div className="text-[10px] text-sl
{steps.length}</div>
        <div className="text-sm text-slate-:
            {step.desc}
        </div>
    </div>
</div>

    {/* Why this matters card */}
    <div className="bg-slate-800 p-4 rounded-xl
        <span className="text-2xl mt-1"> </span>
        <p className="text-xs text-slate-400 le
            Обрати внимание на хитрость с решёт
            Мы пишем <b>Шаблон + # + Текст</b>.
            Когда значени {mode === 'kmp' ? "пр
о было найдено!
            Пройди шаги вручную, посмотри, как
ля пропуска работы (в Z).
        </p>
    </div>

    {/* Code Sneak Peek */}
    <div className="bg-slate-900 border border-
        <div className="bg-slate-800 px-4 py-2 b

```

```

interface TNode {
    id: number;
    x: number;
    y: number;
    label: string;
    name: string;
}

interface TEdge {
    u: number;
    v: number;
}

const dagNodes: TNode[] = [
    { id: 0, x: 100, y: 150, label: "0", name: "Матан 1 (Математика)" },
    { id: 1, x: 280, y: 70, label: "1", name: "Дискретка" },
    { id: 2, x: 280, y: 230, label: "2", name: "Линейный алгебра" },
    { id: 3, x: 460, y: 150, label: "3", name: "Алгоритмы" },
    { id: 4, x: 640, y: 150, label: "4", name: "Машинное обучение" },
];

const dagEdges: TEdge[] = [
    { u: 0, v: 1 },
    { u: 0, v: 2 },
    { u: 1, v: 3 },
    { u: 2, v: 3 },
    { u: 3, v: 4 },
];

interface TStep {
    desc: string;
    visited: Set<number>;
    fullyProcessed: Set<number>;
    currentNode: number | null;
    topoList: number[];
    dfsStack: number[];
}

```

```

        u,
    );

    for (const to of adj[u]) {
        if (!visited.has(to)) {
            dfs(to);
        } else if (!fullyProcessed.has(to)) {
            // Explaining potential cycle detected (not in
            recordStep(
                `⚠ Внимание: Рёбра в серую вершину ${to} о
                u,
            );
        }
    }
}

fullyProcessed.add(u);
dfsStack.pop();
topoList.push(u); // prepend behavior: we write in
recordStep(
    `DFS: закончили обработку '${dagNodes[u].name}'
    ,
    u,
);
};

// Run DFS for all unvisited
for (let i = 0; i < 5; i++) {
    if (!visited.has(i)) {
        recordStep(
            `Запускаем новый обход DFS от нетронутой верши
            null,
        );
        dfs(i);
    }
}

recordStep(
    "Топологическая сортировка завершена! Полученный

```

```

    });

    return (
      <div className="bg-slate-900 rounded-xl border border-
        <div className="bg-slate-800 p-4 border-b border-
          <h3 className="font-bold text-white flex items-
            <Clock className="w-5 h-5 text-indigo-400" />
            Топологическая сортировка (Зависимости обучен
          </h3>

      <div className="flex items-center gap-2 bg-slate-
        <button
          onClick={togglePlay}
          className="flex items-center gap-2 px-3 py-
            500 text-white"
        >
          {isPlaying ? (
            <Pause className="w-4 h-4 ml-1" />
          ) : (
            <Play className="w-4 h-4 ml-1" />
          )}
          {isPlaying ? "Пауза " : "Старт "}
        </button>

        <button
          onClick={reset}
          className="flex items-center justify-center
            ors"
        >
          <RotateCcw className="w-4 h-4" />
        </button>
      </div>
    </div>

    <div className="grid grid-cols-1 lg:grid-cols-12
      {/* Playfield SVG */}
      <div className="lg:col-span-8 bg-[#0B1120] rela
        <svg className="w-full h-full max-w-[650px]"

```

```

        y1={u.y}
        x2={v.x}
        y2={v.y}
        stroke={isActive ? "#818cf8" : "#334155"}
        strokeWidth={isActive ? "3" : "2"}
        markerEnd={`url(#${isActive ? "dag-arrow" : "dag-arrow-2"})`}
        className="transition-all duration-300"
      />
    );
  }
}

```

```

{/* Nodes */}
{dagNodes.map((node) => {
  const isCurrent = step.currentNode === node.id;
  const isVisited = step.visited.has(node.id);
  const isProcessed = step.fullyProcessed.has(node.id);

  let fill = "#1e293b";
  let stroke = "#334155";
  let textColor = "text-slate-400";

  if (isCurrent) {
    fill = "#312e81";
    stroke = "#818cf8";
    textColor = "text-white font-bold";
  } else if (isProcessed) {
    fill = "#022c22";
    stroke = "#059669";
    textColor = "text-emerald-300";
  } else if (isVisited) {
    fill = "#3730a3";
    stroke = "#6366f1";
    textColor = "text-indigo-200";
  }

  return (
    <g key={node.id}>

```

```

    { /* Console column */}
    <div className="lg:col-span-4 bg-slate-950 p-5 00 overflow-y-auto">
      <h4 className="text-xs font-bold text-slate-400">
        Статус сортировки
      </h4>

      <div className="bg-slate-900 border border-slate-800 p-2">
        <p className="text-xs text-indigo-300 min-h-12">
          {step.desc}
        </p>
      </div>

      <div className="flex-1 space-y-4 overflow-y-auto">
        { /* Topological List output */}
        <div>
          <span className="text-[10px] text-slate-500">
            РЕЗУЛЬТАТ (МАТРИЦА ОБУЧЕНИЯ):
          </span>
          <div className="bg-slate-900/60 p-2.5 rounded pr-1 text-slate-300">
            {step.topoList.length === 0 ? (
              <span className="text-slate-600 text-sm">
                ожидаем обработку узлов...
              </span>
            ) : (
              getTopoListString()
            )}
          </div>
        </div>

        { /* Color key */}
        <div className="space-y-1">
          <span className="text-[10px] text-slate-500">
            ВЕРШИНЫ В DFS:
          </span>
          <div className="flex items-center gap-2">
            <span className="w-3.5 h-3.5 rounded bg-slate-800">

```

```
    </div>
  );
};
```

ФАЙЛ 65/78: src/components/vizRegistry.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import React from "react";

import { ArticulationPointsViz } from "./ArticulationPointsViz";
import BellmanFordViz from "./BellmanFordViz";
import BfsViz from "./BfsViz";
import { DPVisualizer } from "./DPVisualizer";
import { DfsBridgesSimulator } from "./DfsBridgesSimulator";
import DijkstraViz from "./DijkstraViz";
import { EulerSimulator } from "./EulerSimulator";
import FloydViz from "./FloydViz";
import { GraphTraversalViz } from "./GraphTraversalViz";
import { HeapViz } from "./HeapViz";
import { JohnsonViz } from "./JohnsonViz";
import { KosarajuViz } from "./KosarajuViz";
import { KruskalSimulator } from "./KruskalSimulator";
import MnemonicCards from "./MnemonicCards";
import { PlanarityDemo } from "./PlanarityDemo";
import { QueueViz } from "./QueueViz";
import { SalmonAutomatonWidget } from "./SalmonAutomatonWidget";
import { SegmentTreeVisualizer } from "./SegmentTreeVisualizer";
import { SplayTreeViz } from "./SplayTreeViz";
import { SplayRotationsViz } from "./SplayRotationsViz";
import { StackViz } from "./StackViz";
import { StringAlgorithmsViz } from "./StringAlgorithmsViz";
import { TopologicalSortViz } from "./TopologicalSortViz";
import { WaterfallAnimationWidget } from "./WaterfallAnimationWidget";
import ChapterImage from "./ChapterImage";
import { Simulator } from "./Simulator";
```

```

    title: "Дерево отрезков",
    hint: "Запросы и обновления на отрезке за  $O(\log n)$ .",
    Component: SegmentTreeVisualizer,
  },
  "sparse-table": {
    title: "Разреженная таблица (1D и 2D)",
    hint: "Наведите курсор на любую ячейку – подсветится.",
    Component: SparseTableViz,
  },
  "prefix-sums-2d": {
    title: "Префиксные суммы (1D и 2D)",
    hint: "Наведите на число – увидите, из чего оно сложилось.",
    Component: PrefixSumViz,
  },
  "dynamic-programming": {
    title: "Динамическое программирование",
    hint: "Таблица подзадач заполняется на глазах.",
    Component: DPVisualizer,
  },
  "splay-tree": {
    title: "Splay-дерево",
    hint: "Сверху – разбор всех трёх поворотов (Zig, Zig-Zig, Zig-Zag).",
    Component: () => (
      <div className="space-y-10">
        <SplayRotationsViz />
        <SplayTreeViz />
      </div>
    ),
  },
  "splay-rotations": {
    title: "Splay: Zig, Zig-Zig и Zig-Zag по шагам",
    hint: "Три случая поворота с покадровой анимацией и звуком.",
    Component: SplayRotationsViz,
  },
  "graph-dfs-bfs": { title: "DFS и BFS на графе", Component: GraphDFS },
  "top-sort": { title: "Топологическая сортировка", Component: TopSort },
  "scc-kosaraju": { title: "Компоненты сильной связности", Component: SCC },
  "graph-articulation": { title: "Точки сочленения", Component: GraphArticulation },

```

```
    title: "Ахо-Корасик",
    hint: "Бор, суффиксные ссылки и BFS-обход по уровням",
    Component: () => (
      <div className="space-y-10">
        <WaterfallAnimationWidget />
        <SalmonAutomatonWidget />
      </div>
    ),
  },
intro: { title: "Мнемокарточки", Component: MnemonicCards },
"everyday-basics": {
  title: "Бытовой тренажёр: стек, очередь, куча",
  hint: "Тарелки, очередь в столовой и мешок гипотез",
  Component: Simulator,
},
};

export const getViz = (id?: string): VizEntry | undefined;
```

ФАЙЛ 66/78: src/components/WaterfallAnimationWidget.tsx
КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 66

```
import React, { useState } from 'react';
import { ArrowRight, CheckCircle2, AlertCircle, Lightbulb } from 'lucide-react';

// Структура заранее подготовленного красивого бора для Waterfall
// Словарь: ["HE", "SHE", "HIS", "HERS"]
interface WNode {
  id: number;
  label: string; // путь от корня
  char: string;
  x: number; // Координаты для SVG водопада
  y: number;
  depth: number;
  fail: number;
```

```

const [activeMode, setActiveMode] = useState<'training' | 'search'>('training');

// === СОСТОЯНИЯ РЕЖИМА ПОИСКА (SEARCH) ===
const [textToScan, setTextToScan] = useState<string>('');
const [currentIndex, setCurrentIndex] = useState<number>(0);
const [currentNode, setCurrentNode] = useState<number>(0);
const [isSearchDone, setIsSearchDone] = useState<boolean>(false);
const [searchLog, setSearchLog] = useState<string>('');
    'Режим поиска: Введите текст и жмите «Следующий шаг»';
);
const [jumpPath, setJumpPath] = useState<{ from: number; to: number }>(null);
const [foundMatches, setFoundMatches] = useState<{ index: number; match: string }>([]);
const [activeSearchCodeLine, setActiveSearchCodeLine] = useState<number>(1);

// === СОСТОЯНИЯ РЕЖИМА ОБУЧЕНИЯ (TRAINING) ===
// Шаги обучения от 0 до 8 (по числу вершин в боре, и т.д.)
const [trainStep, setTrainStep] = useState<number>(0);

const handleTextChange = (e: React.ChangeEvent<HTMLInputElement>) => {
    const clean = e.target.value.toUpperCase().replace(/[^А-Яа-я0-9 ]/g, '');
    setTextToScan(clean);
    resetSearchSimulation();
};

const resetSearchSimulation = () => {
    setCurrentIndex(0);
    setCurrentNode(0);
    setIsSearchDone(false);
    setSearchLog('Симуляция сброшена. Лосось вернулся на базу');
    setJumpPath(null);
    setFoundMatches([]);
    setActiveSearchCodeLine(1);
};

const stepSearchSimulation = () => {
    if (currentIndex >= textToScan.length) {
        setIsSearchDone(true);
        setSearchLog('Мы проплыли весь текст! Лосось доволен');
    }
};

```

```

const currentMatches: { index: number; word: string }[] = [];
const matchedWordsForLog: string[] = [];

let temp = curr;
while (temp !== 0) {
    if (WATERFALL_NODES[temp].is_terminal) {
        WATERFALL_NODES[temp].words.forEach((word) => {
            currentMatches.push({ index: currentIndex, word });
            matchedWordsForLog.push(word);
        });
    }
    temp = WATERFALL_NODES[temp].term_link;
}

if (matchedWordsForLog.length > 0) {
    logMsg += `    БИНГО! Найдено слово: ${matchedWordsForLog.join(' ')}\n`;
    setActiveSearchCodeLine(4);
}

setCurrentNode(curr);
setCurrentIndex(currentIndex + 1);
setSearchLog(logMsg);
if (currentMatches.length > 0) {
    setFoundMatches((prev) => [...prev, ...currentMatches]);
}
};

// ПОЛНАЯ ПОШАГОВАЯ ИНФОРМАЦИЯ О ПОСТРОЕНИИ ДЕРЕВА (В
const TRAINING_STEPS_INFO = [
    {
        targetNodeId: 0,
        title: 'Шаг 0 (Depth 0): Вершина #0 (ROOT)',
        desc: 'Начало алгоритма BFS. Корень (ROOT) не обрабатывается. Другие вершины сразу инициализируются с fail = ROOT',
        codeLine: 1,
        scoutPos: 0,
        inspectedParentFail: 0,
    },

```

```

    title: 'Шаг 4 (Depth 2): Вершина #5 (HI)',
    desc: 'Обрабатываем #5 (HI) на Depth 2. Родитель :
        H" и "S" не подходят. Ветки нет, fail[HI] = ROOT',
    codeLine: 3,
    scoutPos: 0,
    inspectedParentFail: 0,
    checkTargetChar: 'I',
    checkingBranchesFrom: 0,
},
{
    targetNodeId: 8,
    title: 'Шаг 5 (Depth 2): Вершина #8 (SH) — КАК СТАТЬ
    desc: '    ВАЖНЫЙ ВОПРОС! Обрабатываем #8 (SH) на D
        ). Рыба плавает в ROOT и ищет ветку "H". У корня
        суффикс "H"!',
    codeLine: 4,
    scoutPos: 0,
    inspectedParentFail: 0,
    checkTargetChar: 'H',
    checkingBranchesFrom: 0,
},
{
    targetNodeId: 3,
    title: 'Шаг 6 (Depth 3): Вершина #3 (HER)',
    desc: 'Переходим на Depth 3 к #3 (HER). Родитель :
        е подходят. fail[HER] = ROOT.',
    codeLine: 3,
    scoutPos: 0,
    inspectedParentFail: 0,
    checkTargetChar: 'R',
    checkingBranchesFrom: 0,
},
{
    targetNodeId: 6,
    title: 'Шаг 7 (Depth 3): Вершина #6 (HIS)',
    desc: 'Обрабатываем #6 (HIS) на Depth 3. Родитель :
        но ветка "S" → #7 (S) совпадает! fail[HIS] = #7
    codeLine: 4,

```

```

    <span className="text-3xl"> </span>
  <div>
    <h4 className="text-2xl font-bold text-white">
      Анимация водопада: Полное обучение автомата
    </h4>
    <p className="text-sm text-slate-400">
      Визуализация пошагового построения всех ф
    </p>
  </div>
</div>

{/* Переключатель режимов */}
<div className="bg-slate-900 p-1.5 rounded-xl b
  <button
    onClick={() => setActiveMode('training')}
    className={`flex items-center space-x-1.5 p-
      activeMode === 'training'
        ? 'bg-indigo-600 text-white shadow-lg s
        : 'text-slate-400 hover:text-white'
    `}
  >
    <GitBranch className="w-4 h-4" />
    <span>Режим 1: Полное обучение (BFS от корн
  </button>
  <button
    onClick={() => setActiveMode('search')}
    className={`flex items-center space-x-1.5 p-
      activeMode === 'search'
        ? 'bg-blue-600 text-white shadow-lg sha
        : 'text-slate-400 hover:text-white'
    `}
  >
    <Play className="w-4 h-4" />
    <span>Режим 2: Поиск в тексте</span>
  </button>
</div>
</div>

```

```

        ion-all"
        >
        Назад
    </button>
    <button
        onClick={() => setTrainStep((prev) => M
        disabled={trainStep === TRAINING_STEPS_
        className="flex-1 bg-indigo-600 hover:b
    ition-all shadow-lg shadow-indigo-900/50"
        >
        {trainStep === TRAINING_STEPS_INFO.leng
    </button>
</div>
</div>

{/* Блок с кодом построения fail-ссылки и опи
<div className="lg:col-span-2 bg-slate-900/90
    >
    <div>
        <h5 className="text-white font-bold mb-3
            <span className="flex items-center gap-
                <span>❌</span> Код BFS построения fail
            </span>
            <span className="text-xs bg-indigo-950 l
                Вершина: #{targetTrainingNode.id} ({t
            </span>
        </h5>
        <div className="bg-slate-950 p-4 rounded-
            <div className={`p-1.5 rounded flex ite
            -l-4 border-indigo-500 text-indigo-300 font
                <span>1. let u = trie[r][char]; // Те
                {currentTrainInfo.codeLine === 1 && <
            тут</span>}
            </div>
            <div className={`p-1.5 rounded flex ite
            -l-4 border-indigo-400 text-indigo-200 font
                <span>2. let v = fail[r]; // Подъем к
            ainInfo.inspectedParentFail}.label}}</span>

```

```

<div className="grid grid-cols-1 lg:grid-cols-3"
  <div className="bg-slate-900/60 p-5 rounded-x
    <div>
      <h5 className="text-white font-bold mb-2"
        <span>» </span> Текст для сканирования
      </h5>
      <p className="text-xs text-slate-400 mb-4"
        Введи текст, чтобы лосось просканировал
      </p>
      <input
        type="text"
        value={textToScan}
        onChange={handleTextChange}
        maxLength={15}
        placeholder="ВВЕДИТЕ ТЕКСТ..."
        className="w-full bg-slate-800 border b
        cus:outline-none focus:border-blue-500"
      />
    </div>

    <div>
      <div className="mb-4 flex flex-wrap gap-1"
        <span className="text-xs text-slate-400"
          {textToScan.split('').map((char, idx) =>
            <span
              key={idx}
              className={`w-7 h-8 flex items-cent
                idx === currentIndex
                  ? 'bg-blue-600 text-white shadow
                  : idx < currentIndex
                  ? 'bg-emerald-950/60 text-emera
                  : 'bg-slate-800 text-slate-500'
                }`}
            >
              {char}
            </span>
          )}}
        </div>
      </div>
    </div>
  </div>
  {currentIndex >= textToScan.length && (

```

```

        ружок по fail!</span>}
      </div>
      <div className={`p-1.5 rounded flex item
border-blue-400 text-blue-200 font-bold` :
        <span>3. curr = trie[curr].get(char)
        {activeSearchCodeLine === 3 && <span>
ем по течению</span>}
      </div>
      <div className={`p-1.5 rounded flex item
l-4 border-emerald-500 text-emerald-300 font
        <span>4. if (is_terminal[curr]) report
        {activeSearchCodeLine === 4 && <span>
овпадение!</span>}
      </div>
    </div>
  </div>

  <div>
    <h5 className="text-slate-300 font-bold m
    <AlertCircle className="w-3.5 h-3.5 tex
    </h5>
    <div className="bg-slate-800 p-3.5 rounded
tems-center">
      {searchLog}
    </div>
  </div>
</div>
</div>
))

```

```

{ /* SVG Анимация водопада */ }
<div className="bg-gradient-to-b from-blue-950 vi
-hidden shadow-2xl">
  { /* Водопадные фоновые эффекты */ }
  <div className="absolute inset-0 opacity-10 bg-
o-900 to-transparent pointer-events-none"></d
  <div className="absolute top-0 left-1/2 -transl
    <div className="w-8 bg-blue-400 h-full animat

```

```

        strokeDasharray="4,4"
        opacity="0.4"
      />
      <rect x="25" y={level.y - 12} width="14" height="24" />
    />
    <text x="32" y={level.y + 4} fill="#94a0b4" />
      {level.label}
    </text>
  </g>
)}}
```

```

{/* Рендер прямых переходов (потоков воды) */}
Object.entries(TRIE_TRANSITIONS).map(([fromId, toId]) => {
  const fromNode = WATERFALL_NODES[Number(fromId)];
  return Object.entries(children).map(([className, node]) => {
    const toNode = WATERFALL_NODES[toId];

    // Подсветка в режиме поиска
    let isHighlighted = activeMode === 'search' ?
    type === 'normal';

    // Подсветка в режиме обучения (когда ищем ответ)
    let isTrainingExamined = activeMode === 'train' ?
    let isMatchBranch = isTrainingExamined && node.id === toId;

    return (
      <g key={`edge-${fromNode.id}-${toNode.id}`} />
      {
        /* Линия течения воды */
        <line
          x1={fromNode.x}
          y1={fromNode.y}
          x2={toNode.x}
          y2={toNode.y}
          stroke={isHighlighted ? '#3b82f6' : '#ccc'}
          strokeWidth={isHighlighted || isTrainingExamined ? 2 : 1}
          className={isHighlighted || isTrainingExamined ? 'highlighted' : ''}
        />
        {
          /* Символ перехода */

```

```

        let isHighlighted = activeMode === 'search' || activeMode === 'fail';
        let isJustEstablished = activeMode === 'transition' || activeMode === 'target-link';

        // Вычисляем изогнутую кривую для красоты
        const dx = targetNode.x - node.x;
        const dy = targetNode.y - node.y;
        const cx = node.x + dx / 2 + (node.id % 2 === 0 ? -1 : 1) * 30;
        const cy = node.y + dy / 2 - 30;

        return (
          <g key={`fail-${node.id}`}>
            <path
              d={`M ${node.x} ${node.y} Q ${cx} ${cy} ${targetNode.x} ${targetNode.y}`}
              fill="none"
              stroke={isHighlighted ? '#f43f5e' : '#000000'}
              strokeWidth={isHighlighted ? 4 : 2}
              strokeDasharray="6,6"
              className={isHighlighted ? 'highlighted' : ''}
              opacity={isHighlighted ? 1 : 0.5}
            />
            {isJustEstablished && (
              <text x={cx} y={cy - 10} fill="#10b981">
                fail[{node.label}] = {targetNode.label}
              </text>
            )}
          </g>
        );
      }
    }
  }
}

```

```

/* Рендер вершин (бассейнов водопада) */
Object.values(WATERFALL_NODES).map((node) => {
  const isCurrent = activeFishPosNode.id === node.id;
  const isTargetChild = activeMode === 'transition' || activeMode === 'target-link';
  const hasWords = node.words.length > 0;

```

```

        <text x="0" y="2" fontSize="10" f
            ИЩЕМ FAIL
        </text>
    </g>
}}

{/* Индикатор словарных слов */}
{hasWords && {
    <g transform={`translate(${node.x +
        <circle cx="0" cy="0" r="9" fill=
        <text x="0" y="3" fontSize="10" f
            *
        </text>
    </g>
    }}
</g>
});
}}}
```

```

{/* ОДИН ПЛАВНЫЙ АНИМИРОВАННЫЙ ЭХО-ЛОСОСЬ */}
<g>
    {/* Круг-подложка под рыбу */}
    <circle
        cx={activeFishPosNode.x + 15}
        cy={activeFishPosNode.y - 25}
        r="18"
        fill={activeMode === 'training' ? '#8b5
        stroke="#ffffff"
        strokeWidth="2"
        style={{ transition: 'cx 0.8s cubic-bez
        className="shadow-lg"
    />
    {/* Сама рыба */}
    <text
        x={activeFishPosNode.x + 15}
        y={activeFishPosNode.y - 19}
        fontSize="18"
        textAnchor="middle"
```

```
        </div>
      )}
    </div>
  );
};
```

РАЗДЕЛ: ВИЗУАЛИЗАТОРЫ (ВЛОЖЕННЫЕ)

ФАЙЛ 67/78: src/components/visualizers/PrefixSumViz.tsx
КАТЕГОРИЯ: Визуализаторы (вложенные) | СТРОК: 330 | РАЗМЕР: 10.5 КБ

```
import React, { useMemo, useState } from "react";

/**
 * Префиксные суммы: 1D и 2D.
 *
 * Главная фишка – наведение на число:
 * * в 1D наводим на P[i] – подсвечивается кусок массива
 * * в 1D наводим на a[i] – видно, в какие префиксы он входит
 * * в 2D наводим на ячейку – подсвечивается прямоугольник
 * а при выбранном запросе показывается формула включения-исключения
 */

const A1 = [3, 1, 4, 1, 5, 9, 2, 6];

const A2 = [
  [1, 2, 3, 4],
  [5, 6, 7, 8],
  [9, 1, 2, 3],
  [4, 5, 6, 7],
];

const cellBase =
```

```

const [hover, setHover] = useState<{ kind: "pref" | "a" }>(null)
const [range, setRange] = useState<{ l: number; r: number }>({ l: 0, r: 0 })

const covered =
  hover?.kind === "pref"
    ? { from: 0, to: hover.i - 1 }
    : hover?.kind === "a"
      ? { from: hover.i, to: hover.i }
      : null;

const sum = pref[range.r + 1] - pref[range.l];

return (
  <div className="space-y-5">
    <div className="overflow-x-auto pb-1">
      <div className="inline-block min-w-full">
        {/* индексы */}
        <div className="flex gap-1.5 mb-1 pl-[52px]">
          {A1.map((_, i) => (
            <div key={i} className={` ${cellBase} !h-5`}>
              {i}
            </div>
          ))}
        </div>

        {/* массив a */}
        <div className="flex gap-1.5 items-center mb-1">
          <div className="w-[46px] shrink-0 text-right">
            {A1.map((v, i) => {
              const inCover = covered && i >= covered.from && i <= covered.to;
              const inRange = i >= range.l && i <= range.r;
              return (
                <div
                  key={i}
                  onMouseEnter={() => setHover({ kind: "a", i })}
                  onMouseLeave={() => setHover(null)}
                  onClick={() => setRange({r} => (i < r ? i : r))}
                  className={` ${cellBase} cursor-pointer`}>

```

```

        </div>
    );
    }}}
</div>
</div>
</div>

{/* Пояснение под курсором */}
<div className="min-h-[62px] bg-slate-900 border
    {hover?.kind === "pref" ? {
        hover.i === 0 ? {
            <p className="text-slate-300">
                <b className="text-emerald-400">P[0] = 0<
                отрезка, начинающегося с нуля.
            </p>
        ) : {
            <p className="text-slate-300">
                <b className="text-emerald-400">
                    P[{hover.i}] = {pref[hover.i]}
                </b>{" "}
                – это сумма всего, что набежало от начала
                <span className="font-mono text-indigo-300">
                    a[0..{hover.i} - 1] = {A1.slice(0, hove
                </span>
            </p>
        )
    ) : hover?.kind === "a" ? {
        <p className="text-slate-300">
            <b className="text-indigo-400">
                a[{hover.i}] = {A1[hover.i]}
            </b>{" "}
            входит во все префиксы, начиная с{" "}
            <span className="font-mono text-emerald-300">
                подвинуть границу запроса.
            </span>
        </p>
    ) : {
        <p className="text-slate-500">
            Наведите курсор на любое число: y <b classN

```

```

const [hover, setHover] = useState<{ i: number; j: number }>({ i: 0, j: 0 });
const [rect, setRect] = useState({ r1: 1, c1: 1, r2: 1, c2: 1 });

const D = P[rect.r1][rect.c1];
const B = P[rect.r1][rect.c2 + 1];
const Cc = P[rect.r2 + 1][rect.c1];
const Aa = P[rect.r2 + 1][rect.c2 + 1];
const total = Aa - B - Cc + D;

const cornerOf = (i: number, j: number) => {
  if (i === rect.r2 + 1 && j === rect.c2 + 1) return "A";
  if (i === rect.r1 && j === rect.c2 + 1) return "B";
  if (i === rect.r2 + 1 && j === rect.c1) return "C";
  if (i === rect.r1 && j === rect.c1) return "D";
  return null;
};

return (
  <div className="space-y-5">
    <div className="grid gap-5 lg:grid-cols-2">
      {/* Исходная матрица */}
      <div className="overflow-x-auto">
        <div className="text-xs text-slate-400 mb-2">Исходная матрица
        <div className="inline-block">
          {A2.map((row, i) => {
            <div key={i} className="flex gap-1.5 mb-1">
              {row.map((v, j) => {
                const inRect = i >= rect.r1 && i <= rect.r2 && j >= rect.c1 && j <= rect.c2;
                return (
                  <div
                    key={j}
                    onClick={() => setRect((r) => {
                      i < r.r1 || j < r.c1 ? { r1: i, c1: j, r2: i, c2: j } : r
                    })}
                    className={` ${cellBase} cursor-pointer ${inRect ? "bg-slate-200" : ""}`>
                      {v}
                    </div>
                );
              })}
            </div>
          })}
        </div>
      </div>
    </div>
  </div>
);

```

```

        {v}
        {corner && !isHover && {
            <span className="absolute -top-
ht">
                {corner}
            </span>
        }}
    </div>
    );
    }}}
</div>
    )))
</div>
</div>
</div>

<div className="min-h-[54px] bg-slate-900 border
    {hover ? {
        <p className="text-slate-300">
            <b className="text-amber-400">
                 $P[\{hover.i\}][\{hover.j\}] = \{P[hover.i][hov$ 
            </b>{" "}
            – сумма всего прямоугольника от левого верх
        </p>
    } : {
        <p className="text-slate-500">Наведите курсор
    }}
</div>

<div className="bg-slate-900 border border-indigo
    <p className="font-mono text-sm sm:text-base te
        сумма = <span className="text-emerald-400">A<
        <span className="text-rose-400">C</span> + <s
        <span className="text-indigo-300 font-bold">{
    </p>
    <p className="text-[11px] text-slate-500 mt-1">
        Вычли два «хвоста» и вернули дважды вычитенный
    </p>

```

```

    [67, 19, 44, 55, 33, 21, 65, 87],
    [14, 51, 99, 13, 22, 64, 43, 76],
    [89, 32, 54, 71, 15, 88, 29, 60],
    [25, 41, 16, 92, 9, 17, 56, 31],
    [59, 18, 77, 24, 61, 82, 35, 12],
    [73, 8, 38, 85, 47, 95, 19, 58],
    [39, 81, 62, 28, 51, 42, 85, 14]
];

const ST2D: number[][][][] = Array.from({length: 8}, () =>
    Array.from({length: 8}, () =>
        Array.from({length: 4}, () =>
            Array(4).fill(0)
        )
    )
);

for (let r = 0; r < 8; r++) {
    for (let c = 0; c < 8; c++) {
        ST2D[r][c][0][0] = val2D[r][c];
    }
}

for (let kx = 0; kx <= 3; kx++) {
    for (let ky = 0; ky <= 3; ky++) {
        if (kx === 0 && ky === 0) continue;
        for (let r = 0; r + (1 << kx) <= 8; r++) {
            for (let c = 0; c + (1 << ky) <= 8; c++) {
                if (kx > 0) {
                    ST2D[r][c][kx][ky] = Math.min(
                        ST2D[r][c][kx-1][ky],
                        ST2D[r + (1 << (kx-1))][c][kx-1][ky]
                    );
                } else {
                    ST2D[r][c][kx][ky] = Math.min(
                        ST2D[r][c][kx][ky-1],
                        ST2D[r][c + (1 << (ky-1))][kx][ky-1]
                    );
                }
            }
        }
    }
}

```

```

        ) : tab === "2d_build" ? (
          <Viz2DBuild key="2d_build" />
        ) : (
          <Viz2DQuery key="2d_query" />
        )}
      </AnimatePresence>
    </div>
  );
};

const Viz1D: React.FC = () => {
  const [step, setStep] = useState(0);
  const [hover, setHover] = useState<{ i: number; j: number }>({ i: 0, j: 0 });

  // Total steps: we animate computing each element for
  const computeSteps: { i: number; j: number }[] = [];
  for (let j = 1; j < LOG; j++) {
    for (let i = 0; i + (1 << j) <= N; i++) {
      computeSteps.push({ i, j });
    }
  }

  const maxStep = computeSteps.length;
  const covered = hover ? { from: hover.i, to: hover.i + (1 << hover.j) } : null;

  return (
    <div className="flex flex-col gap-5">
      <div className="flex items-center justify-between">
        <div>
          <h3 className="text-lg font-bold text-indigo-600">
            Построение Sparse Table (Min)
          </h3>
          <p className="text-sm text-slate-400">
            Формула:  $ST[i][j] = \min(ST[i][j-1], ST[i + (1 \ll (j-1))][j-1])$ 
          </p>
        </div>
        <div className="flex gap-2">
          <button

```

```

        </div>
    );
    }}}
</div>
<div className="mt-2 text-xs min-h-[36px]">
    {hover && covered ? (
        <p className="text-slate-300">
            <b className="text-sky-400">
                ST[{hover.i}][{hover.j}] = {stID[hover.i]}
            </b>{" "}
            – это минимум на отрезке{" "}
            <span className="font-mono text-sky-300">
                [{covered.from}, {covered.to}]
            </span>{" "}
            длины  $2^{\{hover.j\}}$  = {1 << hover.j}:{ " "}
            <span className="font-mono text-slate-400">
                min({arrID.slice(covered.from, covered.to)}
            </span>
        </p>
    ) : (
        <p className="text-slate-500">
            Наведите курсор (или тапните) на любое чи
        </p>
    )}
</div>
</div>

<div className="overflow-x-auto pb-2">
    <div className="inline-block min-w-full bg-slate-50">
        <div className="grid grid-cols-[auto_repeat(4,1fr)]">
            {/* Headers */}
            <div className="text-slate-500 font-mono text-sm">
                i \ j
            </div>
            <div className="text-center font-bold text-sm">
                2^0 (L=1)
            </div>
        </div>
    </div>

```

```

        if (
            currentStep.j === j + 1 &&
            currentStep.i === i - (1 << (currentStep.j - j))
        ) {
            isSrc2 = true; // wait, no. Src2 is the source of the next step
            // Let's re-eval exact source:
            // We are asking if THIS cell [i][j] is the source of the next step
            if (currentStep.j - 1 === j) {
                if (currentStep.i === i) isSrc1 = true;
                if (currentStep.i + (1 << (currentStep.j - j)) === i)
                    isSrc2 = true;
            }
        }
    }

    return (
        <div key={j} className="relative flex flex-col items-center justify-center"
        {!isValid ? (
            <div className="w-[clamp(30px,9vw,17px)] h-[clamp(11px,3vw,17px)]"
        ) : (
            <motion.div
                onMouseEnter={() => setIsVisible(true)}
                onMouseLeave={() => setIsVisible(false)}
                onClick={() => setIsVisible(true) && setStep(currentStep)}
                initial={{ opacity: 0, scale: 0.8 }}
                animate={{
                    opacity: isVisible ? 1 : 0,
                    scale: isVisible ? 1 : 0.8,
                }}
                className={`w-[clamp(30px,9vw,17px)] h-[clamp(11px,3vw,17px)] font-bold transition-colors duration-200
                    hover && hover.i === i && hover.j === j
                    ? "bg-sky-500 text-white"
                    : isActive
                    ? "bg-indigo-500 text-white"
                    : isSrc1
                    ? "bg-emerald-500 text-white"
                    : isSrc2
                    ? "bg-amber-500 text-white"
                `}
            />
        )}
    )
)

```

```
</div>
```

```
{/* Formula helper text */}
```

```
<div className="bg-slate-900 border border-slate-100">
```

```
  {step > 0 && step <= maxStep ? (
```

```
    (() => {
```

```
      const c = computeSteps[step - 1];
```

```
      const half = 1 << (c.j - 1);
```

```
      return (
```

```
        <p className="text-lg text-slate-300">
```

```
          <span className="text-white font-bold">
```

```
            ST[{c.i}][{c.j}]
```

```
          </span>{" "}
```

```
          = min(
```

```
            <span className="text-emerald-400 font-l">
```

```
              ST[{c.i}][{c.j - 1}]
```

```
            </span>
```

```
            ,
```

```
            <span className="text-amber-400 font-bo">
```

```
              ST[{c.i + half}][{c.j - 1}]
```

```
            </span>
```

```
          ) &rarr; min(
```

```
            <span className="text-emerald-400">{st1D
```

```
            <span className="text-amber-400">
```

```
              {st1D[c.i + half][c.j - 1]}
```

```
            </span>
```

```
          ) ={" "}
```

```
            <span className="text-indigo-400 font-b">
```

```
              {st1D[c.i][c.j]}
```

```
            </span>
```

```
          </p>
```

```
        );
```

```
      ))()
```

```
    ) : (
```

```
      <p className="text-slate-500 italic">
```

```
        Нажмите далее, чтобы увидеть, как блоки сво
```

```
      </p>
```

```
    ))
```

```

rollbar">
{[...Array(k + 1)].map((_, i) => k - i).map((
  const stepL = 1 << step;
  const stepSize = 8 - stepL + 1;
  const isCurr = step === k;

  return (
    <React.Fragment key={step}>
      {idx > 0 && <div className="text-slate
      <div className={`flex flex-col items-c
      <h4 className={`font-bold mb-3 flex
    }>
      <span>Матрица блоков {stepL}x{step
      <span className={`text-[10px] font
rder-indigo-800/40 text-indigo-300' : 'bg-s
      </h4>
      <div className="max-w-full overflow-
      <div className={`grid gap-[2px] p-
go-900/40 shadow-[0_0_25px_rgba(99,102,241,
eColumns: `repeat(${stepSize}, max-content)
      {Array.from({length: stepSize * st
        const r = Math.floor(idx / step
        const c = idx % stepSize;

        let highlightClass = isCurr ? 'b
/50 cursor-pointer' : 'bg-slate-800/50 text
        let zIndex = 'z-0';

        const isActive = !!activeCell &
ll.c + L) && ((r - activeCell.r) % (1 << st

        if (isActive) {
          if (isCurr) {
            highlightClass = `bg-indigo
{locked && locked.r === r && locked.c === c
            zIndex = 'z-10';
          } else {
            const prevL = 1 << (k - 1);

```

```

        }}}
      </div>
    </div>
  </div>
</React.Fragment>
)
  }}}
</div>
</div>

<div className="flex-1 min-w-0 flex flex-col gap-4" >
  <div className="bg-slate-900 p-4 sm:p-6 rounded" >
    <h3 className="text-lg font-bold text-slate-300" >
      {k === 0 ? (
        <div className="text-slate-400 text-sm leading-6" >
          <p className="mb-4" >При <span className="font-weight-normal" >
            и имеют размер <span className="font-weight-normal" >
              <div className="bg-[#0a0f1e] rounded-xl p-4" >
                <span className="text-slate-500" >// Ба
                <span className="text-indigo-400" >ST</span>
              </div>
                <p className="mt-6 italic text-indigo-400" >
                  <Play size={14} className="fill-indigo-400" />
                </p>
              </div>
            ) : (
              <div className="flex flex-col gap-4 animate-pulse" >
                <div className="bg-[#0a0f1e] rounded-xl p-4 text-slate-300 shadow-inner overflow-x-auto" >
                  <span className="text-slate-500" >// Те
                  <span className="text-purple-400" >int</span>
                </div>
                <div className="flex flex-col gap-4" >
                  <span className="text-amber-300" >1</span></div><br/><br/>
                  <span className="text-slate-500" >// Кв
                  <span className="text-indigo-400" >ST</span>
                  <span className="text-rose-400" >1</span>
                </div>
                <div className="flex flex-col gap-4" >
                  <span className="text-blue-400" >1</span>
                  <span className="text-amber-300" >1</span>
                </div>
              </div>
            )
          </div>
        </div>
      </div>
    </div>
  </div>

```

```

                <span className="text
tiveCell.r][activeCell.c + prevL][k-1][k-1]
            </div>
            <div className="flex ite
                <span className="flex
-500 shadow-[0_0_8px_#10b981]"></div> ST[{a
                <span className="text
>{ST2D[activeCell.r + prevL][activeCell.c][
            </div>
            <div className="flex ite
                <span className="flex
shadow-[0_0_8px_#f59e0b]"></div> ST[{active
                <span className="text
[activeCell.r + prevL][activeCell.c + prevL
            </div>
        </>
    )
    }}({})
</div>

    <div className="pt-3 mt-3 border-t
    ) = <span className="text-white
0">{ST2D[activeCell.r][activeCell.c][k][k]}
    </div>
    <p className="mt-4 text-[12px] font
список матриц (слева) вниз, чтобы увидеть,
    </div>
    ) : {
        <div className="bg-slate-950/40 border
mt-2 flex items-center justify-center anim
            Наведите курсор на матрицу {L}x{L
        </div>
    }}
</div>
    )}
</div>
</div>
</div>

```

```

                {val2D[r][c]}
            </div>
        )
    }}}

    <div
        className="absolute border-[3px
ow-[0_0_15px_rgba(244,63,94,0.3)] border-da
        style={{
            top: `${(r1 / 8) * 100}%`,
            left: `${(c1 / 8) * 100}%`,
            marginTop: '8px',
            marginLeft: '8px',
            height: 'calc(' + (H/8*100) +
            width: 'calc(' + (W/8*100) +
        }}
    />

    /* Показываем 4 угла в самой матрице
    <div className="absolute pointer-e
        shadow-[0_0_10px_#f43f5e]" style={{ top: `
        lc(`${(Ly / 8) * 100}% - 16px)`, height: `ca
        <div className="absolute pointer-e
        shadow-[0_0_10px_#3b82f6]" style={{ top: `
        width: `calc(`${(Ly / 8) * 100}% - 16px)`,
        <div className="absolute pointer-e
        ld-400 shadow-[0_0_10px_#10b981]" style={{
        8px)`, width: `calc(`${(Ly / 8) * 100}% - 16
        <div className="absolute pointer-e
        00 shadow-[0_0_10px_#f59e0b]" style={{ top:
        00}% + 8px)`, width: `calc(`${(Ly / 8) * 100
        </div>

    <div className="bg-slate-950 p-4 round
er-slate-800 shadow-[inset_0_2px_10px_rgba(
        <label className="flex items-cen
            <span className="text-slate-
                <div className="flex gap-2">

```

```

        <span className="text-purple-400">int<
ext-amber-300">1</span>); <span className="
        <span className="text-purple-400">int<
"text-slate-500">// Выс: {Lx}</span><br/>
        <span className="text-purple-400">int<
"text-slate-500">// Шир: {Ly}</span><br/><b

        <span className="text-slate-500">// 2.
        <span className="text-indigo-400">int<
        &nbsp;&nbsp;&nbsp;<span className="text-slate-500">
В запроса)</span><br/>
        &nbsp;&nbsp;&nbsp;<span className="text-rose-400">
r1</span>][<span className="text-white bg-r

        &nbsp;&nbsp;&nbsp;<span className="text-slate-500">
во, чтобы правый край уперся в c2)</span><b
        &nbsp;&nbsp;&nbsp;<span className="text-blue-400">
r1</span>][<span className="text-white bg-b

        &nbsp;&nbsp;&nbsp;<span className="text-slate-500">
px, чтобы нижний край уперся в r2)</span><b
        &nbsp;&nbsp;&nbsp;<span className="text-emerald-400">
nded">r2 - Lx + 1</span>][<span className="

        &nbsp;&nbsp;&nbsp;<span className="text-slate-500">
и влево от c2)</span><br/>
        &nbsp;&nbsp;&nbsp;<span className="text-amber-400">
">r2 - Lx + 1</span>][<span className="text
        {'}}'}});
    </div>

    <div className="w-full flex justify-cent
    <div className="flex flex-col items-ce
    <h4 className="text-slate-400 font-bo
        Откуда берутся эти 4 числа: матриц
border-slate-700 shadow-sm">ST[][][{{kx}}][{{k
    </h4>
    <div className="grid gap-[2px] p-2.5

```

```
        wrap bg-[#0a0f1e] py-3 px-2 rounded-lg border
            <span className="text-slate-400 font
                <span className="text-rose-400">
                <span className="text-blue-400 m
            n>
                <span className="text-emerald-400
            span>
                <span className="text-amber-400 m
                &nbsp;<span className="text-slate-400
                <span className="ml-3 text-emerald-
+ 1][c1][kx][ky], ST2D[r2 - Lx + 1][c2 - Ly
            </div>
        </div>
    </div>
</div>
    )
}
```

РАЗДЕЛ: HTML-РАЗМЕТКА (LAYOUT)

ФАЙЛ 69/78: src/layout/footer.html

КАТЕГОРИЯ: HTML-разметка (layout) | СТРОК: 137 | РАЗМЕР

```
        </div>
    </main>
</div>

<script>
    function setMode(mode) {
        document.getElementById('btn-mode-normal').
            ? 'px-3 py-1 rounded text-sm font-bold
```

11:

```
if (theme === 'dark') document.getElementById('dark-mode').checked = true;
if (theme === 'light') document.getElementById('light-mode').checked = true;
if (theme === 'terminal') document.getElementById('terminal-mode').checked = true;
):
```

```
document.getElementById('btn-theme-dark').classList.add('text-white transition-colors' : 'px-3 py-1 rounded transition-colors');
document.getElementById('btn-theme-light').classList.add('text-white transition-colors' : 'px-3 py-1 rounded transition-colors');
document.getElementById('btn-theme-notebook').classList.add('text-white transition-colors' : 'px-3 py-1 rounded transition-colors');
```

```
// Initialize theme and mode
setTheme('dark');
```

```
// Setup intersection observer for TOC highlight
document.addEventListener('DOMContentLoaded', () => {
  const mobileMenuBtn = document.getElementById('mobile-menu');
  const mobileDrawer = document.getElementById('mobile-drawer');
  const closeDrawerBtn = document.getElementById('close-drawer');
  const drawerOverlay = document.getElementById('drawer-overlay');
  const tocList = document.querySelector('ul.toc');
  const tocItems = tocList ? tocList.querySelectorAll('li') : [];
  const observer = new IntersectionObserver((entries) => {
    entries.forEach((entry) => {
      if (entry.isIntersecting) {
        entry.target.classList.add('active');
      }
    });
  });
  tocItems.forEach((item) => {
    observer.observe(item);
  });
  mobileMenuBtn.addEventListener('click', () => {
    mobileDrawer.classList.toggle('show');
    drawerOverlay.classList.toggle('show');
  });
  closeDrawerBtn.addEventListener('click', () => {
    mobileDrawer.classList.remove('show');
    drawerOverlay.classList.remove('show');
  });
  drawerOverlay.addEventListener('click', () => {
    mobileDrawer.classList.remove('show');
    drawerOverlay.classList.remove('show');
  });
});
```

```
function toggleDrawer() {
  const isClosed = mobileDrawer.classList
  if (isClosed) {
    mobileDrawer.classList.remove("-tran
    if (drawerOverlay) {
      drawerOverlay.classList.remove(
      // Timeout allows display:block
      setTimeout(() => drawerOverlay.
    }
    document.body.style.overflow = "hid
```

```
        // Highlight active TOC link lo
        const id = entry.target.id;
        document.querySelectorAll('aside
            if (link.getAttribute('href
                link.classList.add('bg-
            } else {
                link.classList.remove('l
            }
        });
    }
    });
}, observerOptions);

document.querySelectorAll('section[id^="que
    observer.observe(section);
});
});
</script>
</body>
</html>
```

ФАЙЛ 70/78: src/layout/header.html

КАТЕГОРИЯ: HTML-разметка (layout) | СТРOK: 1169 | РАЗМЕР

```
<!DOCTYPE html>
<html lang="ru" class="scroll-smooth">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width,
  <title>Пособие по алгебре: От пиццы к предикатам</t
  <script src="https://cdn.tailwindcss.com"></script>
  <script src="https://polyfill.io/v3/polyfill.min.js
  <script id="MathJax-script" async src="https://cdn.
  <script>
    window.MathJax = {
```

```
mjx-container {
  max-width: 100% !important;
}
mjx-container[display="true"] {
  overflow-x: auto;
  overflow-y: hidden;
  max-width: 100%;
  scrollbar-width: none;
  -ms-overflow-style: none;
  -webkit-overflow-scrolling: touch;
}
mjx-container[display="true"]::-webkit-scrollbar
  display: none;
}

@media (max-width: 640px) {
  html { font-size: 13px; }

  .uno-card { width: 40px; height: 60px; font-size: 10px; }
  .uno-card::before, .uno-card::after { font-size: 10px; }
  .uno-card::before { top: 1px; left: 2px; }
  .uno-card::after { bottom: 1px; right: 2px; }

  .uno-card.mini { width: 28px !important; height: 40px !important; font-size: 4px; }
  .uno-card.mini::before, .uno-card.mini::after { font-size: 4px; }

  .uno-equation .text-2xl { font-size: 1.2rem; }
  .uno-equation.gap-4 { gap: 0.5rem !important; }

  /* Allow horizontal scroll for equations in containers
  .formula-scroll { flex-wrap: nowrap !important; }
  .formula-scroll { width: 100%; height: 100%; }
  }

  .uno-inner {
    position: absolute; width: 110%; height: 75%;
    border-radius: 50%; transform: rotate(-30deg);
  }
```

```
        color: #3b2818 !important; /* brown ink */
        background-image: repeating-linear-gradient(
        background-attachment: local !important;
        font-family: "Comic Sans MS", "Caveat", "Serif";
    }
body.theme-notebook .bg-slate-900, body.theme-notebook .bg-slate-700 {
    background-color: rgba(255, 255, 255, 0.7);
    border-color: #cbd5e1 !important;
    box-shadow: 2px 2px 5px rgba(0,0,0,0.05);
}
body.theme-notebook .text-slate-200, body.theme-notebook .text-slate-400 { color: #cbd5e1 !important; }
body.theme-notebook .text-slate-400 { color: #546e7a !important; }
body.theme-notebook .border-slate-600, body.theme-notebook .border-slate-800 { border-color: #94a3b8 !important; }
body.theme-notebook .font-mono { font-family: "Monospace", "Courier New", "monospace"; }
body.theme-notebook #top-controls { background-color: #f1f3f4; }

/* CREEPY BLINKING */
@keyframes creepy-blink {
    0%, 95%, 98%, 100% { transform: scaleY(1); }
    96.5% { transform: scaleY(0.1); }
}
.creepy-eye {
    transform-origin: center;
    transform-box: fill-box;
    animation: creepy-blink 4s infinite;
}
.creepy-eye-alt {
    transform-origin: center;
    transform-box: fill-box;
    animation: creepy-blink 5.2s infinite;
    animation-delay: 1.5s;
}
.creepy-emoji {
    display: inline-block;
```

```

        <path d="M0,0 L6,3 L0,6 Z" fill="#f97311" stroke="black" stroke-width="1" />
    </marker>
    <marker id="arrow-pink" markerWidth="6" markerHeight="6" />
        <path d="M0,0 L6,3 L0,6 Z" fill="#f472b1" stroke="black" stroke-width="1" />
    </marker>
    <marker id="arrow-cyan" markerWidth="6" markerHeight="6" />
        <path d="M0,0 L6,3 L0,6 Z" fill="#2dd4b1" stroke="black" stroke-width="1" />
    </marker>
    <marker id="arrow-indigo" markerWidth="6" markerHeight="6" />
        <path d="M0,0 L6,3 L0,6 Z" fill="#818cf8" stroke="black" stroke-width="1" />
    </marker>
    <marker id="arrow-lime" markerWidth="6" markerHeight="6" />
        <path d="M0,0 L6,3 L0,6 Z" fill="#a3e63d" stroke="black" stroke-width="1" />
    </marker>
    <marker id="arrow-green" markerWidth="6" markerHeight="6" />
        <path d="M0,0 L6,3 L0,6 Z" fill="#4ade80" stroke="black" stroke-width="1" />
    </marker>

    <!-- ЛЕГО БЛОКИ -->
    <g id="lego-yellow">
        <rect x="0" y="0" width="40" height="20" fill="yellow" />
        <rect x="6" y="-5" width="8" height="6" fill="yellow" />
        <rect x="26" y="-5" width="8" height="6" fill="yellow" />
        <line x1="2" y1="2" x2="38" y2="2" stroke="black" stroke-width="1" />
    </g>
    <g id="lego-red">
        <rect x="0" y="0" width="40" height="20" fill="red" />
        <rect x="6" y="-5" width="8" height="6" fill="red" />
        <rect x="26" y="-5" width="8" height="6" fill="red" />
        <line x1="2" y1="2" x2="38" y2="2" stroke="black" stroke-width="1" />
    </g>
    <g id="lego-rem">
        <rect x="0" y="0" width="40" height="10" fill="red" />
        <rect x="6" y="-5" width="8" height="6" fill="red" />
        <rect x="26" y="-5" width="8" height="6" fill="red" />
        <line x1="2" y1="2" x2="38" y2="2" stroke="black" stroke-width="1" />
    </g>
    <!-- МЯСОРУБКА -->

```

```

<aside id="mobile-drawer" class="fixed inset-y-0
  nslate-x-full transition-transform duration-300
  w-none lg:block lg:w-72 lg:shrink-0 lg:z-auto
  <div class="h-full flex flex-col p-6 overflow-hidden
    lg:rounded-2xl lg:border lg:border-slate-700
      <div class="flex justify-between items-center
        <h3 class="font-black text-transparent bg-clip-text
          ider text-sm">
            Содержание
          </h3>
          <button id="close-drawer-btn" class="p-2
            <svg xmlns="http://www.w3.org/2000/svg"
              <path stroke-linecap="round" stroke-width="2"
            </svg>
          </button>
        </div>
      <nav class="space-y-3 text-sm font-medium">
        <details class="group" open>
          <summary class="list-none cursor-pointer text-blue-500
            <a href="#question-1" class="text-blue-500 pl-3 font-bold text-blue-300">
              1. Алгебраические структуры
            </a>
          </summary>
          <div class="pl-6 space-y-2 text-sm">
            <a href="#section-1-1" class="text-blue-500">1.1
            <a href="#section-1-2" class="text-blue-500">1.2
            <a href="#section-1-3" class="text-blue-500">1.3
            <a href="#section-1-4" class="text-blue-500">1.4
          </div>
        </details>
        <details class="group">
          <summary class="list-none cursor-pointer text-indigo-500
            <a href="#question-2" class="text-indigo-500 pl-3 font-bold text-indigo-300">
              2. Комплексные числа
            </a>
          </summary>
          <div class="pl-6 space-y-2 text-sm">
            <a href="#section-2-1" class="text-indigo-500">2.1
            <a href="#section-2-2" class="text-indigo-500">2.2
            <a href="#section-2-3" class="text-indigo-500">2.3
            <a href="#section-2-4" class="text-indigo-500">2.4
          </div>
        </details>
      </nav>
    </div>
  </div>

```



```

<div class="pl-6 space-y-2 text
  <a href="#q9-1" class="bloc
  <a href="#q9-2" class="bloc
  <a href="#q9-3" class="bloc
  <a href="#q9-4" class="bloc
</div>
</details>

<details class="group">
  <summary class="list-none curso
    <a href="#question-10" clas
der-rose-500 pl-3 font-bold text-rose-400">
      10. Делимость и Схема Г
    </a>
  </summary>
  <div class="pl-6 space-y-2 text
    <a href="#q10-1" class="blo
    <a href="#q10-2" class="blo
    <a href="#q10-3" class="blo
a>
      <a href="#q10-4" class="blo
      <a href="#q10-5" class="blo
    </div>
  </details>

  <details class="group">
    <summary class="list-none curso
      <a href="#question-11" clas
der-cyan-500 pl-3 font-bold text-cyan-400">
        11. НОД и НОК многочлен
      </a>
    </summary>
    <div class="pl-6 space-y-2 text
      <a href="#q11-1" class="blo
      <a href="#q11-2" class="blo
нственности НОД</a>
      <a href="#q11-3" class="blo
ПИРАЛЬ)</a>

```

```
14. Кратность корня. Ко  
    </a>  
</summary>  
<div class="pl-6 mt-2 space-y-2"  
    <a href="#q14-1" class="blo  
    <a href="#q14-2" class="blo  
    <a href="#q14-3" class="blo  
deg)</a>  
    <a href="#q14-4" class="blo  
ition-colors border border-rose-500/30"> K  
    </div>  
</details>  
  
<details class="group">  
    <summary class="list-none curso  
    <a href="#question-15" clas  
der-cyan-500 pl-3 font-bold text-cyan-400">  
    15. Производная. Критер  
    </a>  
</summary>  
<div class="pl-6 mt-2 space-y-2"  
    <a href="#q15-1" class="blo  
    <a href="#q15-2" class="blo  
    <a href="#q15-3" class="blo  
    <a href="#q15-4" class="blo  
    </div>  
</details>  
  
<details class="group">  
    <summary class="list-none curso  
    <a href="#question-16" clas  
der-red-500 pl-3 font-bold text-red-400">  
    16. Неприводимые многоч  
    </a>  
</summary>  
<div class="pl-6 mt-2 space-y-2"  
    <a href="#q16-1" class="blo  
    <a href="#q16-2" class="blo
```

```

        detail.addEventListener('toggle
            if (detail.open) {
                document.querySelectorA
                    if (other !== detail
                        other.removeAtt
                    }
            }
        });
    });
});

// Scroll spy logic
document.addEventListener('DOMConten
    const sections = document.query
    ="q5-"], div[id^="q6-"], div[id^="q7-"], di
    v[id^="q13-"], div[id^="q14-"], div[id^="q1
    let isScrolling = false;

    document.querySelectorAll('.gro
        link.addEventListener('clie
            isScrolling = true;
            setTimeout(() => isScro
        });
    });

    const observer = new Intersection
        if (isScrolling) return;

        let visibleId = null;
        entries.forEach(entry => {
            if (entry.isIntersectin
                visibleId = entry.t

                // Highlight active
                document.querySelec
                    a.style.fontWeig
                    if (a.href.ends
                        a.style.fon

```

```
modal.style.display = 'flex';
modal.style.flexDirection = 'column';
modal.style.alignItems = 'center';
modal.style.justifyContent = 'center';

const box = document.createElement('div');
box.style.width = '80%';
box.style.height = '80%';
box.style.backgroundColor = '#1a1a1a';
box.style.padding = '20px';
box.style.borderRadius = '10px';
box.style.display = 'flex';
box.style.flexDirection = 'column';
box.style.color = '#fff';

const header = document.createElement('div');
header.innerText = 'Экспорт в PDF';
header.style.marginBottom = '10px';

const subHeader = document.createElement('div');
subHeader.innerText = 'Из-за проблем с некоторыми браузерами, для загрузки файла необходимо открыть эту страницу в новой вкладке (через меню)';
subHeader.style.marginBottom = '10px';
subHeader.style.color = '#f87171';
subHeader.style.fontSize = '0.9em';

const textarea = document.createElement('div');
textarea.value = contentRaw;
textarea.style.flex = '1';
textarea.style.width = '100%';
textarea.style.color = '#000';
textarea.style.padding = '10px';
textarea.style.fontFamily = 'monospace';

const btnContainer = document.createElement('div');
btnContainer.style.display = 'flex';
btnContainer.style.gap = '10px';
btnContainer.style.marginTop = '10px';
```

```

        box.appendChild(btnContainer);
        modal.appendChild(box);
        document.body.appendChild(modal)
    }

```

```

function downloadMD() {
    let clone = document.querySelector('#clone')

    // Smart markdown parsing based on marked.js
    // Smart markdown parsing based on marked.js
    clone.querySelectorAll('h2').forEach(el => {
    clone.querySelectorAll('h3').forEach(el => {
    clone.querySelectorAll('.analog').forEach(el => {
    clone.querySelectorAll('li').forEach(el => {
    clone.querySelectorAll('svg').forEach(el => {
        let caption = '';
        if (el.nextElementSibling && el.nextElementSibling.tagName === 'CAPTION') {
            caption = el.nextElementSibling.textContent;
        }
        el.textContent = '\n\n[ Изображение: ' + caption + ' ]';
    });
    clone.querySelectorAll('.img-caption').forEach(el => {
        let text = clone.innerHTML;
        text = text.replace(/\n{3,}/g, '\n\n');

        if (window !== window.top) {
            fallbackModal(text, 'Markdown');
            return;
        }
        const blob = new Blob([text], { type: 'text/markdown' });
        const url = URL.createObjectURL(blob);
        const link = document.createElement('a');
        link.href = url;
        link.download = 'vyisshaya_algebra.md';
        document.body.appendChild(link);
        link.click();
    });
}

```

```

nt; }

body.bg-white [class*="]

nt; }

body.bg-white [class*="]
body.bg-white .analogy-
body.bg-white .img-plac
body.bg-white .info-blo

body.bg-white .analogy-
or: #94a3b8 !important; }
body.bg-white .img-capt

/* Tweak card blocks */
body.bg-white .card-red

body.bg-white aside { b
#334155 !important; }
body.bg-white .text-whi

/* Ensure active links
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a:h

body.bg-white table tr.
body.bg-white table tr.
body.bg-white .text-tea
body.bg-white .text-ros
body.bg-white .text-eme
body.bg-white .text-cya
body.bg-white .text-amb
body.bg-white .text-fuc
body.bg-white .text-lim
body.bg-white .text-blur

```

```

            <span> </span> Word
        </button>
        <a href="algebra.pdf" target="_blank">
    -sm font-bold flex items-center justify-center gap-2
            <span> </span> PDF
        </a>
        <button onclick="downloadMD()" type="button">
    -bold flex items-center justify-center gap-2
            <span> </span> Markdown
        </button>
        <button onclick="downloadHTML()" type="button">
    t-sm font-bold flex items-center justify-center gap-2
            <span> </span> HTML
        </button>
    </div>

    <!-- Смена темы -->
    <div>
        <div class="text-xs text-slate-500">
            <div class="flex items-center gap-2">
                <button id="theme-dark" onclick="toggleTheme('dark')"
    justify-center text-amber-300 hover:text-white
    line-amber-500" title="Темная тема"> </button>
                <button id="theme-light" onclick="toggleTheme('light')"
    r justify-center text-amber-50 hover:text-white
    ">* </button>
                <button id="theme-terminal" onclick="toggleTheme('terminal')"
    -center justify-center text-green-400 hover:text-white
    терминала"> </button>
            </div>
        </div>
    </div>
</div>

</aside>

<!-- ОСНОВНОЙ КОНТЕНТ -->
<main class="flex-1 bg-slate-800 p-4 sm:p-8 md:p-12">

```

```

<!-- ===== WTF CONTAINER =====
<div id="wtf-container" class="hidden">
  <h2 class="text-3xl font-black text-cen

    <div class="overflow-x-auto w-full pb-4"
      <div class="grid grid-cols-3 gap-4"
        <!-- Column headers -->
        <div class="bg-yellow-900/40 bo
оля и кольца, целые числа</div>
        <div class="bg-orange-900/40 bo
ольцо полиномов</div>
        <div class="bg-cyan-900/40 bord
сные числа</div>

        <!-- Row 1: База -->
        <a href="#question-1" onclick="setM
-4 border-yellow-500 hover:bg-slate-700 tra
        <div class="text-xs text-yellow
        <div class="font-bold text-slate
        </a>
        <a href="#question-9" onclick="setM
-4 border-orange-500 hover:bg-slate-700 tra
        <div class="text-xs text-orange
        <div class="font-bold text-slate
        </a>
        <a href="#question-2" onclick="setM
-4 border-cyan-500 hover:bg-slate-700 trans
        <div class="text-xs text-cyan-5
        <div class="font-bold text-slate
        </a>

        <!-- Row 2: Базовые операции / Евклид
        <a href="#question-6" onclick="setM
-4 border-yellow-500 hover:bg-slate-700 tra
        <div class="text-xs text-yellow
        <div class="font-bold text-slate
        </a>
        <a href="#question-10" onclick="setM

```

```

        <div class="font-bold text-slate-500">
    </a>

    <!-- Row 5: Корни / Уравнения -->
    <div class="hidden md:block"></div>
    <a href="#question-13" onclick="setMainQuestion(13)">
- l-4 border-orange-500 hover:bg-slate-700 transition-colors
        <div class="text-xs text-orange-500">
            <div class="font-bold text-slate-500">
    </a>
    <a href="#question-4" onclick="setMainQuestion(4)">
- 4 border-cyan-500 hover:bg-slate-700 transition-colors
        <div class="text-xs text-cyan-500">
            <div class="font-bold text-slate-500">
    </a>

    <!-- Row 6: Кратность -->
    <div class="hidden md:block"></div>
    <a href="#question-14" onclick="setMainQuestion(14)">
- l-4 border-orange-500 hover:bg-slate-700 transition-colors
        <div class="text-xs text-orange-500">
            <div class="font-bold text-slate-500">
    </a>
    <div class="hidden md:block"></div>

    <!-- Row 7: Производная -->
    <div class="hidden md:block"></div>
    <a href="#question-15" onclick="setMainQuestion(15)">
- l-4 border-orange-500 hover:bg-slate-700 transition-colors
        <div class="text-xs text-orange-500">
            <div class="font-bold text-slate-500">
    </a>
    <div class="hidden md:block"></div>

    <!-- КЛАСТЕР 3: НЕПРИВОДИМОСТЬ -->
    <div class="col-span-1 md:col-span-1">
b-2 border-b border-emerald-500/30 pb-2">Кл

```



```
Просто перепиши формулу с буквой $k$.</li>
    <li><span class="text-w
усок $k$, замени его на жареного карася из
```

```
</ul>
```

```
</div>
```

```
<!-- Двойная делимость -->
```

```
<div class="bg-slate-800 p-6
```

```
    <h3 class="text-xl text-ora
```

```
    <p class="text-sm text-slate
```

```
    <ul class="list-disc pl-5 s
```

```
        <li><span class="text-w
```

```
вверх по уравнениям).</li>
```

```
        <li><span class="text-w
```

```
рху вниз, показывая, что  $\Delta \vdots r_i$ 
```

```
        <li><span class="text-w
```

```
х. Твой  $d$  больше.</li>
```

```
    </ul>
```

```
</div>
```

```
<h2 class="text-2xl font-bold b
```

```
оритмы экзекуции)</h2>
```

```
<div class="grid grid-cols-1 md
```

```
    <div class="bg-slate-800 p-
```

```
        <h4 class="text-red-400
```

```
        <p class="text-sm text-
```

```
        <p class="text-xs font-
```

```
со следующим. Если в конце 0 – пациент мерт
```

```
    </div>
```

```
<div class="bg-slate-800 p-
```

```
    <h4 class="text-red-400
```

```
    <p class="text-sm text-
```

```
    <p class="text-xs font-
```

```
 $c$ . Пока получается 0 – корень жив. Как то
```

```
корня минус один.</p>
```

```
-----
        <h4 class="text-cyan">
        <p class="text-sm">
x двух, потом последних. <br>
        <code class="text-x">
        </div>
    </li>

    <li class="bg-slate-800/50">
        <span class="text-2xl">
        <div>
            <h4 class="text-cyan">
            <p class="text-sm">
есть сложение – есть ноль. Если есть умноже
            <code class="text-x">
            </div>
        </li>

    <li class="bg-slate-800/50">
        <span class="text-2xl">
        <div>
            <h4 class="text-cyan">
            <p class="text-sm">
авь их драться. Умножение всегда врывается
            <code class="text-x">
            </div>
        </li>
    </ul>
</div>
</section>
</div>

<div id="questions-container">
<!-- ===== ВОПРОС 1 =====
```

```

fs.mkdirSync(TMP, { recursive: true });
const bundle = path.join(TMP, "content.bundle.mjs");

await build({
  entryPoints: [path.join(ROOT, "src/data/content.ts")]
  bundle: true,
  format: "esm",
  platform: "node",
  outfile: bundle,
  logLevel: "error",
});

const { chapters } = await import(`file://${bundle}`);

// Список интерактивов берём из реестра визуализаторов
const registry = fs.readFileSync(path.join(ROOT, "src/c
const registryBody = registry.slice(registry.indexOf("V
const vizIds = new Set([...registryBody.matchAll(/^\\s{2

const rows = chapters.map((c) => {
  const html = c.content ?? "";
  const analogies = (html.match(/Аналогия/gi) ?? []).length
  return {
    id: c.id,
    title: c.title,
    num: Number.parseInt(c.title.match(/^(\d+)\.\/)?.[1]
    analogies,
    hasAnalogy: analogies > 0,
    inlineSvg: /<svg[\\s>]/i.test(html),
    image: /<img[\\s>]/i.test(html),
    interactive: vizIds.has(c.id),
    chars: html.length,
  };
});

const has2D = (r) => r.interactive || r.inlineSvg || r.
const gapBoth = rows.filter((r) => !r.hasAnalogy && !ha

```

```
/**
 * Проверка покрытия глав всплывающими подсказками:
 * в каждой ли теме находятся термины из глоссария.
 *
 * node scripts/audit-terms.mjs
 */
import { build } from "esbuild";
import path from "node:path";

const ROOT = process.cwd();
await build({
  entryPoints: ["src/data/content.ts", "src/data/glossary.ts"],
  bundle: true, format: "esm", platform: "node",
  outdir: "tmp/audit-terms", outExtension: { ".js": ".mjs" },
  external: ["react", "react-dom", "motion", "lucide-react"],
});

const { chapters } = await import(path.join(ROOT, "tmp/audit-terms.mjs"));
const { GLOSSARY_LABELS, GLOSSARY } = await import(path.join(ROOT, "tmp/audit-terms.mjs"));

const esc = (s) => s.replace(/[\.\*\+\?\^\$\{\}\(\)|[\]\[\]]/g, "\\");
const re = new RegExp(
  `(?<![\\p{L}\\p{N}_-])(?=${GLOSSARY_LABELS.map((l) => esc(l.label))})` +
  "giu"
);

const map = new Map(GLOSSARY_LABELS.map((l) => [l.label, l]));

// те же лимиты, что и в src/hooks/useTermHints.ts
const MAX_PER_SURFACE = 2;
const MAX_PER_TERM = 8;

const bad = [];
const used = new Set();
console.log("тем | подсв. | глава");
for (const c of chapters) {
  const text = c.content
    .replace(/<(script|style|code|pre)[\s\S]*?<\/[\s\S]>/gi, "")
    .replace(/<[^\>]+>/g, " ");
}
```

```
*
* Склеивает отрендеренные страницы tmp/export-pages/pages/
* в единый документ public/export/full_code_all_pages.pdf
*/
import fs from "node:fs";
import path from "node:path";
import PDFDocument from "pdfkit";
import { ROOT, OUT_DIR, PAGES_DIR, PDF_PATH, ensureDir,

// A4 в пунктах PDF
const A4 = { width: 595.28, height: 841.89 };

async function main() {
  if (!fs.existsSync(PAGES_DIR)) {
    throw new Error(`Нет ${path.relative(ROOT, PAGES_DIR)}`);
  }

  const pages = fs
    .readdirSync(PAGES_DIR)
    .filter((f) => f.endsWith(".png"))
    .sort();

  if (pages.length === 0) throw new Error("Не найдено ни одной страницы");

  ensureDir(OUT_DIR);

  const doc = new PDFDocument({
    size: [A4.width, A4.height],
    margin: 0,
    autoFirstPage: false,
    info: {
      Title: "Универсальное пособие – полный исходный код",
      Author: "CI: rebuild-pdf",
      Subject: "Экспорт всего кода проекта (код -> txt)",
      CreationDate: new Date(),
    },
  });
};
```

```
*/
import fs from "node:fs";
import path from "node:path";
import { execFileSync } from "node:child_process";
import { ROOT, OUT_DIR, TXT_PATH, HR, SUBHR, ensureDir,

/** Расширения, которые считаем «кодом» и включаем в да
const CODE_EXT = new Set([
  ".ts", ".tsx", ".js", ".jsx", ".mjs", ".cjs",
  ".css", ".scss", ".html", ".json", ".md", ".yaml", ".yml"
]);

/** Явные исключения: генерируемое, бинарное и просто ш
const EXCLUDE_FILES = new Set(["package-lock.json", "bu
const EXCLUDE_DIRS = ["node_modules", "dist", "build",

/** Человекочитаемые категории для оглавления. */
const CATEGORIES = [
  [/^src\/data\/tickets\/$/, "Билеты (учебный контент)"],
  [/^src\/data\/$/, "Контент и данные пособия"],
  [/^src\/components\/visualizers\/$/, "Визуализаторы (в
  [/^src\/components\/$/, "Интерактивные компоненты и си
  [/^src\/layout\/$/, "HTML-разметка (layout)"],
  [/^src\/utils\/$/, "Утилиты"],
  [/^src\/$/, "Ядро приложения"],
  [/^scripts\/$/, "Скрипты сборки и экспорта"],
  [/^\.github\/$/, "CI / автоматизация"],
  [/^[^\/]+\$/, "Конфигурация проекта"],
];

const categoryOf = (rel) => CATEGORIES.find(([re]) => re

/** Порядок разделов в документе. */
const ORDER = [
  "Конфигурация проекта",
  "Ядро приложения",
  "Контент и данные пособия",
  "Билеты (учебный контент)",
```

```

        const ia = ORDER.indexOf(categoryOf(a));
        const ib = ORDER.indexOf(categoryOf(b));
        return ia !== ib ? ia - ib : a.localeCompare(b);
    });
}

/** Достаём заголовки глав из данных пособия (без запус
function extractChapters(files) {
    const chapters = [];
    const seen = new Set();
    for (const rel of files.filter((f) => f.startsWith("s
        const src = fs.readFileSync(path.join(ROOT, rel), "
        const re = /id:\s*"([^"]+)"\s*,\s*\n?\s*title:\s*"
        let m;
        while ((m = re.exec(src)) !== null) {
            const [, id, title] = m;
            if (seen.has(id)) continue;
            seen.add(id);
            chapters.push({ id, title, file: rel });
        }
    }
    return chapters.sort((a, b) => {
        const na = parseInt(a.title.match(/(\d+)/)?.[0] ??
        const nb = parseInt(b.title.match(/(\d+)/)?.[0] ??
        return na - nb;
    });
}

function main() {
    const files = listFiles();
    const chapters = extractChapters(files);
    ensureDir(OUT_DIR);

    const out = [];
    const push = (s = "") => out.push(s);

    const stamp = new Date().toLocaleString("ru-RU", { ti
    const totalBytes = files.reduce((s, f) => s + fs.stat

```

```
    const cat = categoryOf(rel);
    if (cat !== current) {
        current = cat;
        push(HR);
        push(`РАЗДЕЛ: ${cat.toUpperCase()}`);
        push(HR);
        push();
    }
    const content = fs.readFileSync(path.join(ROOT, rel));
    const lines = content.split("\n");
    push(SUBHR);
    push(`ФАЙЛ ${i + 1}/${files.length}: ${rel}`);
    push(`КАТЕГОРИЯ: ${cat} | СТРОК: ${lines.length} |`);
    push(SUBHR);
    push();
    push(content.replace(/\n+$/, ""));
    push();
    push();
});

push(HR);
push(`КОНЕЦ ДОКУМЕНТА • ${files.length} файлов • сгенерировано`);
push(HR);

const txt = out.join("\n") + "\n";
fs.writeFileSync(TXT_PATH, txt, "utf8");

console.log("[1/3] код -> txt");
console.log(`    файлов:  ${files.length}`);
console.log(`    глав:    ${chapters.length}`);
console.log(`    строк:   ${txt.split("\n").length}`);
console.log(`    выход:   ${path.relative(ROOT, TXT_PATH)}`);
}

main();
```

```
    /** Строк содержимого на странице (+2 служебные: шапка  
    rows: 76,  
};  
  
export const HR = "=".repeat(PAGE.cols);  
export const SUBHR = "-".repeat(PAGE.cols);  
  
export function ensureDir(dir) {  
  fs.mkdirSync(dir, { recursive: true });  
}  
  
export function humanSize(bytes) {  
  if (bytes < 1024) return `${bytes} B`;  
  if (bytes < 1024 * 1024) return `${(bytes / 1024).toFixed(2)} KB`;  
  return `${(bytes / 1024 / 1024).toFixed(2)} MB`;  
}  
  
/** ImageMagick 7 (`magick`) или 6 (`convert`). */  
export function imageMagickCmd() {  
  for (const cmd of ["magick", "convert"]) {  
    try {  
      execFileSync(cmd, ["-version"], { stdio: "ignore" });  
      return cmd;  
    } catch {  
      /* пробуем следующий */  
    }  
  }  
  throw new Error(  
    "ImageMagick не найден. Установите его: sudo apt-get install imagemagick  
  );  
}
```

```

    const pages = [];
    for (let i = 0; i < source.length; i += PAGE.rows) {
        pages.push(source.slice(i, i + PAGE.rows));
    }
    return pages;
}

async function main() {
    if (!fs.existsSync(TXT_PATH)) {
        throw new Error(`Нет ${path.relative(ROOT, TXT_PATH)}`);
    }

    const im = imageMagickCmd();
    const pages = paginate(fs.readFileSync(TXT_PATH, "utf8"));

    fs.rmSync(PAGES_DIR, { recursive: true, force: true });
    ensureDir(PAGES_DIR);

    const total = pages.length;
    const jobs = pages.map((lines, idx) => {
        const num = String(idx + 1).padStart(4, "0");
        const header = `Универсальное пособие • полный исходный код
        Math.max(1, PAGE.cols - 44 - `стр. ${idx + 1} / ${total}`);
    })стр. ${idx + 1} / ${total}`;
    const body = [header, "-".repeat(PAGE.cols), ...lines];
    const txtFile = path.join(PAGES_DIR, `page-${num}.txt`);
    const pngFile = path.join(PAGES_DIR, `page-${num}.png`);
    fs.writeFileSync(txtFile, body + "\n", "utf8");
    return { txtFile, pngFile, num };
});

const args = (job) => [
    "-density", String(PAGE.density),
    "-page", PAGE.size,
    "-font", fs.existsSync(PAGE.fontFile) ? PAGE.fontFile : "serif",
    "-pointsize", String(PAGE.pointsize),
    `text:${job.txtFile}`,
    "-background", "white",

```

```
process.exit(1);  
});
```

РАЗДЕЛ: CI / АВТОМАТИЗАЦИЯ

ФАЙЛ 78/78: `.github/workflows/rebuild-pdf.yml`

КАТЕГОРИЯ: CI / автоматизация | СТРӨК: 128 | РАЗМЕР: 4.0 КБ

name: Rebuild code PDF

Пайплайн пересборки PDF при каждом коммите:

```
#  
# исходный код → full_code_all_pages.txt (script to generate PDF)  
#  
# └───────────┬───────────> page-0001.png ... page-NNNN.png (images)  
#  
# └───────────┬───────────> full_code_all_page.pdf (final PDF)  
#
```

Готовые TXT и PDF коммитаются обратно в ветку (их отдаст деплой)

промежуточные PNG сохраняются как artifacts запуска.

on:

push:

branches:

- "**"

paths-ignore:

чтобы коммит самого workflow не запускал бесконечно

- "public/export/**"

- "**/*.md"

workflow_dispatch:

inputs:

density:

description: "DPI растеризации страниц (по умолчанию 300)"

- ```
 sudo sed -i 's/rights="none" pattern="\{P
 fi
done
convert -version
```
- name: Install dependencies  
run: npm ci --no-audit --no-fund
  - name: Step 1 – код → txt  
run: npm run export:txt
  - name: Step 2 – txt → png  
env:  
 EXPORT\_DENSITY: \${github.event.inputs.density}
run: npm run export:png
  - name: Step 3 – png → pdf  
run: npm run export:pdf
  - name: Type-check приложения  
run: npx tsc --noEmit  
continue-on-error: true
  - name: Summary  
run: |  
 {  
 echo "### Экспорт пересобран"  
 echo ""  
 echo "| Артефакт | Размер |"  
 echo "| --- | --- |"  
 echo "| \public/export/full\_code\_all\_pages  
 echo "| \public/export/full\_code\_all\_pages  
 echo "| PNG-страниц | \${ls tmp/export-pages.  
 } >> "\$GITHUB\_STEP\_SUMMARY"
  - name: Upload artifacts (PDF + TXT)  
uses: actions/upload-artifact@v4  
with: